

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»
Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)
“ ” _____ 2024 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
інформаційних систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Розширення Visual Studio для автоматичної генерації юніт-тестів

Виконав студент IV курсу, групи ІІ-01
(шифр групи)
Коваленко Микита Артемович
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник д.т.н., проф., засл. діяч, Павлов О. А.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент професор кафедри ICT, д.т.н., проф. Корнага Я. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.
Студент _____
(підпис)

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
інформаційних систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ____ ” _____ 2024 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Коваленко Микиті Артемовичу

(прізвище, ім'я, по батькові)

1. Тема проєкту Розширення Visual Studio для автоматичної генерації юніт-тестів

керівник проєкту Павлов О. А., д.т.н., проф., засл. діяч

(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «27» травня 2024 р. №2112-с

2. Термін подання студентом проєкту « 17 » червня 2024 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Передпроєктне обстеження предметної області: основні визначення та терміни, опис предметного середовища, огляд ринку програмних продуктів, постановка задачі.

2) Розроблення вимог до програмного забезпечення: опис варіантів використання, аналіз системних вимог, зроблення функціональних і нефункціональних вимог.

3) Конструювання та розроблення програмного забезпечення: архітектура програмного забезпечення, обґрунтування вибору засобів розробки, конструювання програмного забезпечення.

4) Аналіз якості та тестування програмного забезпечення: аналіз якості ПЗ, опис процесів тестування, опис контрольного прикладу.

5) Розгортання та супровід програмного забезпечення.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань _____

2) Схема структурна компонентів програмного забезпечення _____

3) Креслення вигляду екранних форм _____

4) _____

5) _____

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «11» березня 2024 року _____

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	11.02.2024	
2	Аналіз існуючих методів розв'язання задачі	20.02.2024	
3	Постановка та формалізація задачі	01.03.2024	
4	Розробка інформаційного забезпечення	10.03.2024	
5	Алгоритмізація задачі	14.03.2024	
6	Обґрунтування вибору використаних технічних засобів	26.03.2024	
7	Розробка програмного забезпечення	26.04.2024	
8	Налагодження програми	05.05.2024	
9	Виконання графічних документів	15.05.2024	
10	Оформлення пояснювальної записки	30.05.2024	
11	Подання ДП на попередній захист	03.06.2024	
12	Подання ДП рецензенту	10.06.2024	
13	Подання ДП на основний захист	14.06.2024	

Студент

_____ (підпис)

Микита КОВАЛЕНКО

_____ (ініціали, прізвище)

Керівник

_____ (підпис)

Олександр ПАВЛОВ

_____ (ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з п'яти розділів, містить 29 таблиць, 72 рисунки, 3 формули та 27 джерел – загалом 160 сторінок.

Дипломний проєкт присвячений розробці розширення для автоматизації генерації юніт-тестів у середовищі розробки Visual Studio.

Метою розробки є підвищення ефективності роботи програмістів за рахунок зменшення часу, що витрачається на створення юніт-тестів під час розробки.

У розділі обстеження предметної області були надані основні визначення та терміни, описане предметне середовище, оглянуто ринок програмних продуктів, виконана постановка задачі.

Розділ розроблення вимог до програмного забезпечення формулює варіанти використання, функціональні та нефункціональні вимоги до розроблюваного ПЗ, аналізує системні вимоги.

Розділ конструювання програмного забезпечення описує архітектуру, обґрунтування вибору засобів розробки, конструювання програмного забезпечення.

Розділ аналізу якості та тестування містить опис характеристик якості та процесів тестування програмного забезпечення.

Розділ розгортання та супроводу описує процеси розгортання, публікації та супроводу розробленого програмного забезпечення.

КЛЮЧОВІ СЛОВА: РОЗШИРЕННЯ, VISUAL STUDIO, ЮНІТ-ТЕСТ, АВТОМАТИЗАЦІЯ, МЕТАПРОГРАМУВАННЯ.

ABSTRACT

The explanatory note for the diploma project consists of five sections, including 29 tables, 72 figures, 3 formulas and 27 numbers – 160 pages.

The diploma project is dedicated to the development of an extension to automate the generation of unit tests in the middle of Visual Studio development.

The method of development is to increase the efficiency of work programs by changing the amount of time that is spent on creating unit tests during development.

In the section covering the subject area, the main meanings of the terms were given, the middle of the subject was described, the software market was reviewed, and the problem statement was defined.

The section of requirement formulation describes requirements for software development, functional and non-functional capabilities prior to the development of the software, and analyzes system capabilities.

The software design section describes the architecture, selection of development methods, and software design.

The quality analysis and testing section contains a description of the quality characteristics and software testing processes.

The section on the development and maintenance of software describes the processes of development, publication and maintenance of fragmented software.

KEY WORDS: EXTENSION, VISUAL STUDIO, UNIT TEST, AUTOMATION, METAPROGRAMMING.

[illegible]

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗШИРЕННЯ VISUAL STUDIO ДЛЯ АВТОМАТИЧНОЇ ГЕНЕРАЦІЇ
ЮНІТ-ТЕСТІВ**

Технічне завдання

КПІ.ПІ-0114.045300.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Микита КОВАЛЕНКО

Київ – 2024

ЗМІСТ

1	НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ.....	3
2	ПІДСТАВА ДЛЯ РОЗРОБКИ.....	4
3	ПРИЗНАЧЕННЯ РОЗРОБКИ.....	5
4	ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	6
4.1	Вимоги до функціональних характеристик	6
4.1.1	Користувачького інтерфейсу.....	6
4.1.2	Для користувача:.....	8
4.1.3	Для адміністратора системи (якщо він передбачений):	8
4.1.4	Додаткові вимоги:.....	8
4.2	Вимоги до надійності	9
4.3	Умови експлуатації.....	9
4.3.1	Вид обслуговування	9
4.3.2	Обслуговуючий персонал	9
4.4	Вимоги до складу і параметрів технічних засобів	9
4.5	Вимоги до інформаційної та програмної сумісності	9
4.5.1	Вимоги до вхідних даних.....	10
4.5.2	Вимоги до вихідних даних	10
4.5.3	Вимоги до мови розробки.....	10
4.5.4	Вимоги до середовища розробки	10
4.5.5	Вимоги до представлення вихідних кодів	10
4.6	Вимоги до маркування та пакування.....	10
4.7	Вимоги до транспортування та зберігання	10
4.8	Спеціальні вимоги	10
5	ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ	11
5.1	Попередній склад програмної документації	11
5.2	Спеціальні вимоги до програмної документації	11
6	СТАДІЇ І ЕТАПИ РОЗРОБКИ.....	12
7	ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ	13

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Розширення Visual Studio для автоматичної генерації юніт-тестів.

Галузь застосування: розробка програмного забезпечення з використанням інтегрованого середовища розробки Visual Studio

Наведене технічне завдання поширюється на розробку розширення для середовища Visual Studio, котре використовується для автоматичної генерації юніт-тестів та призначене для програмістів, що використовують мову C# і середовище Visual Studio для розробки і супроводу програмного забезпечення.

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки розширення Visual Studio для автоматичної генерації юніт-тестів є завдання на дипломне проектування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для автоматизації генерації та полегшення написання юніт-тестів у середовищі розробки Visual Studio.

Метою розробки є підвищення ефективності роботи програмістів за рахунок зменшення часу, що витрачається на створення юніт-тестів під час розробки.

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Вимоги до функціональних характеристик

Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1 Користувацького інтерфейсу

Користувацький інтерфейс та екранні форми розширення мають відповідати загальному тематичному оформленню середовища Visual Studio і відчуватися природньо.

- спливаюче вікно для генерації тестів для заданого типу (рис. 4.1);
- спливаюче вікно для генерації тестового проєкту (рис. 4.2);
- окрема сторінка у вікні налаштувань середовища для конфігурації розширення (рис. 4.3).

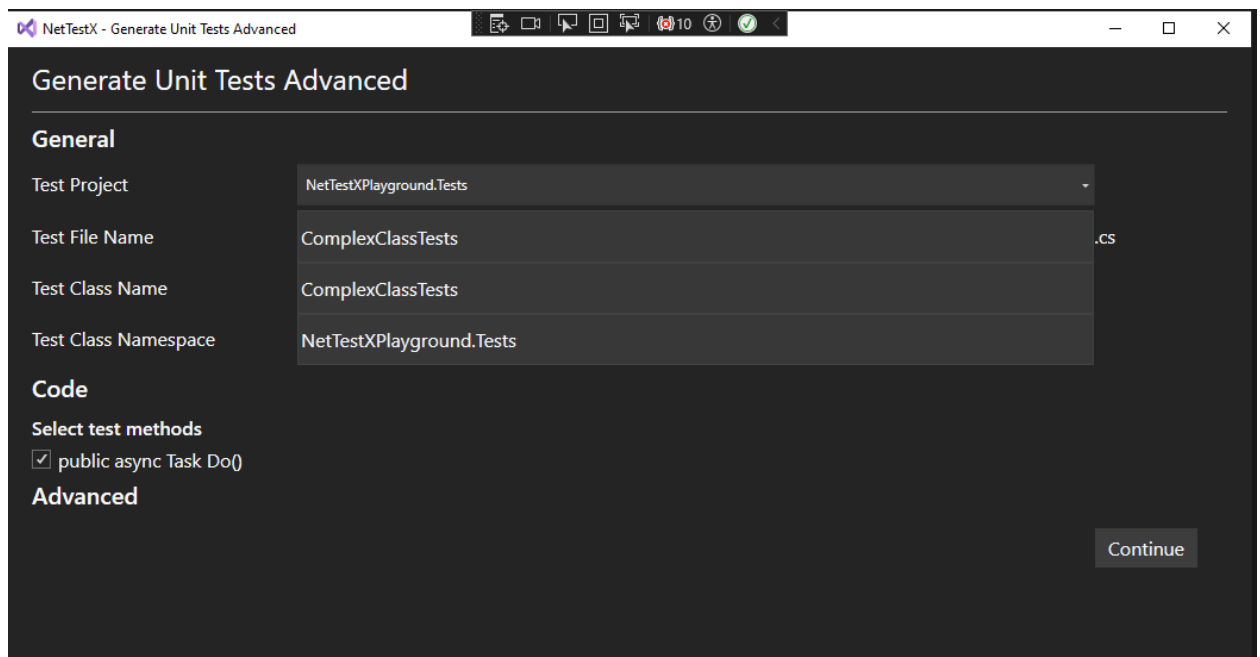


Рисунок 4.1 – Сторінка генерації тестів для заданого типу

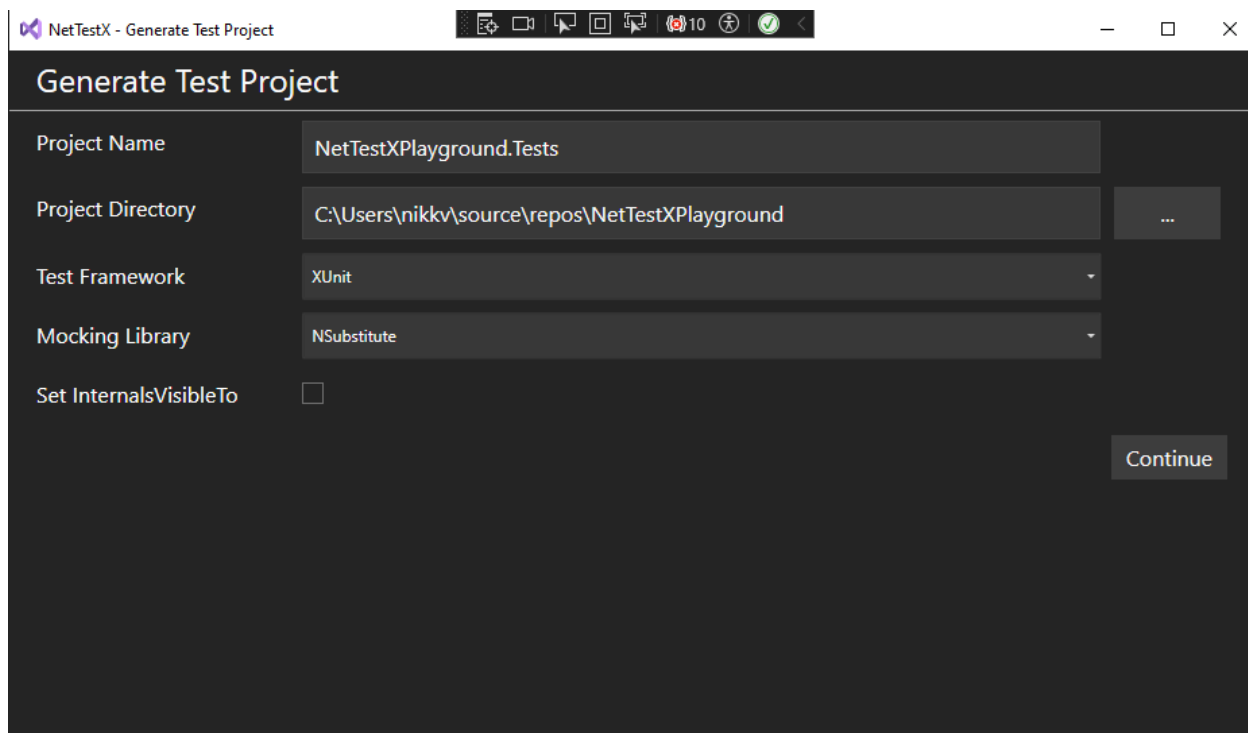


Рисунок 4.2 – Сторінка генерації тестового проєкту

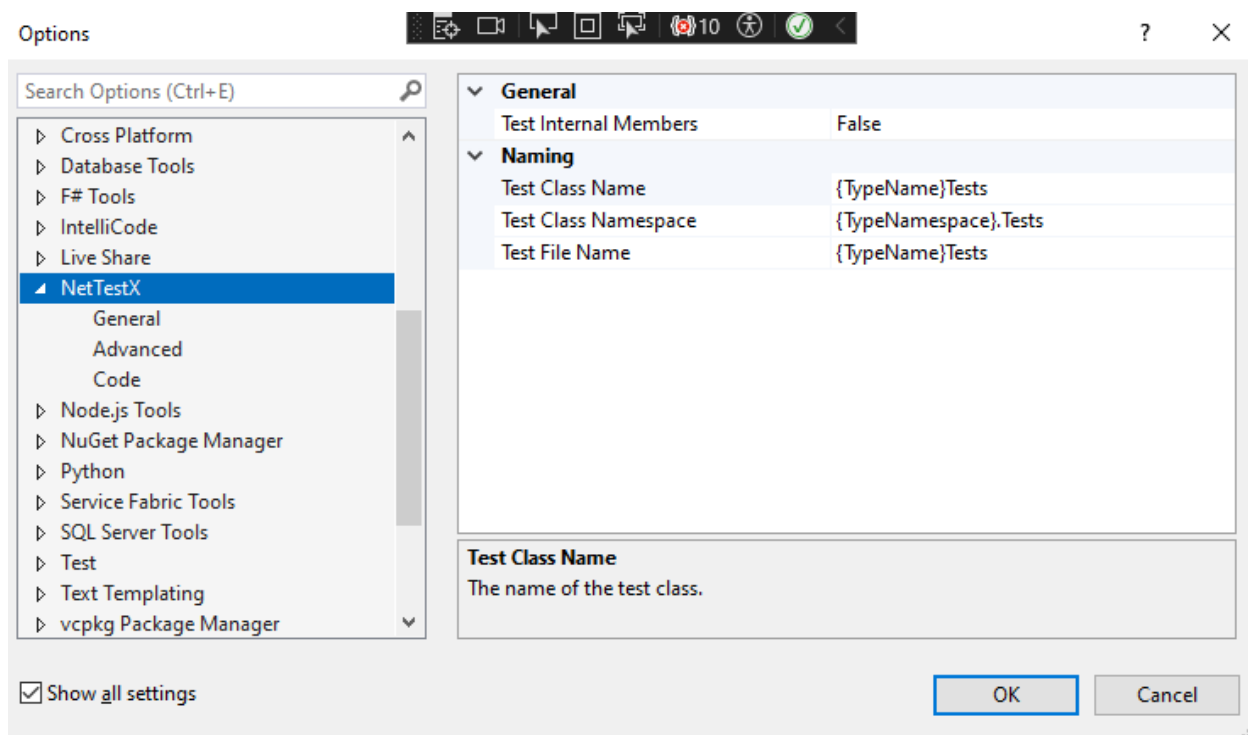


Рисунок 4.3 – Сторінка конфігурації розширення у вікні налаштувань середовища

4.1.2 Для користувача:

- можливість згенерувати тести для обраного типу (типів) в окремому файлі (файлах) в тестовому проєкті:
 - через вікно редактору коду (Code Editor):
 - лише задавши тестовий проєкт, використовуючи налаштування за замовчуванням;
 - задавши тестовий проєкт і власні налаштування (рис. 4.1);
 - через вікно дослідника рішень (Solution Explorer):
 - лише задавши тестовий проєкт, використовуючи налаштування за замовчуванням;
 - задавши тестовий проєкт і власні налаштування (рис. 4.1);
- можливість згенерувати тестовий проєкт для обраного проєкту (рис. 4.2);
- можливість змінити налаштування розширення у меню опцій (Options) середовища Visual Studio (рис. 4.3).

4.1.3 Для адміністратора системи (якщо він передбачений):

Вимоги до адміністратора системи не висуваються.

4.1.4 Додаткові вимоги:

- розширення має генерувати максимальну кількість шаблонного коду, лишаючи користувачеві дописати лише кілька рядків коду;
- розширення має бути спрямоване на генерацію коду, що завжди компілюється.

4.2 Вимоги до надійності

Передбачити контроль введення інформації та захист від некоректних дій користувача. Забезпечити функціонал генерації тестів лише для файлів із відповідним розширенням *.cs.

4.3 Умови експлуатації

Умови експлуатації згідно СанПін 2.2.2.542 – 96.

4.3.1 Вид обслуговування

Вимоги до виду обслуговування не висуваються.

4.3.2 Обслуговуючий персонал

Вимоги до обслуговуючого персоналу не висуваються.

4.4 Вимоги до складу і параметрів технічних засобів

Програмне забезпечення повинно функціонувати на персональних комп'ютерах архітектури x64.

Мінімальна рекомендована версія Visual Studio для роботи: Visual Studio 2022. Рекомендовані вимоги до технічних засобів для середовища розробки:

- тип процесору: Intel Core i5;
- об'єм ОЗП: 8 Гб;
- об'єм вільного дискового простору: 20 Гб.

4.5 Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційних систем Windows 10 або Windows 11.

На системі має бути встановлено середовище розробки Visual Studio 2022.

4.5.1 Вимоги до вхідних даних

Вхідні дані повинні бути представлені в наступному форматі:

- для генерації тестів: файл із розширенням *.cs;
- для генерації тестових проєктів: файл із розширенням *.csproj.

4.5.2 Вимоги до вихідних даних

Результати повинні бути представлені в наступному форматі:

- для генерації тестів: файл із розширенням *.cs;
- для генерації тестових проєктів: файл із розширенням *.csproj.

4.5.3 Вимоги до мови розробки

Розробку виконати на мові програмування C#.

4.5.4 Вимоги до середовища розробки

Розробку виконати за допомогою середовища Visual Studio 2022

4.5.5 Вимоги до представлення вихідних кодів

Вихідний код програми має бути представлений у вигляді текстових файлів із розширенням *.cs.

4.6 Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7 Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8 Спеціальні вимоги

Згенерувати інсталяційну версію програмного забезпечення. Викласти розширення на порталі Visual Studio Marketplace.

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1 Попередній склад програмної документації

У склад супроводжувальної документації повинні входити наступні документи на аркушах формату А4:

- технічне завдання;
- пояснювальна записка;
- текст програми;
- програма та методика тестування;
- керівництво користувача;

Графічна частина повинна бути виконана на аркушах формату А3 та містити наступні документи:

- схема структурна варіантів використання;
- схема структурна компонентів програмного забезпечення;
- креслення вигляду екранних форм.

5.2 Спеціальні вимоги до програмної документації

Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі.

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення рекомендованої літератури	11.02.2024	
2.	Аналіз існуючих методів розв'язання задачі	20.02.2024	
3.	Постановка та формалізація задачі	01.03.2024	Технічне завдання
4.	Розробка інформаційного забезпечення	10.03.2024	
5.	Алгоритмізація задачі	14.03.2024	
6.	Обґрунтування вибору використаних технічних засобів	26.03.2024	
7.	Розробка програмного забезпечення	26.04.2024	Тексти програмного забезпечення
8.	Налагодження програми	05.05.2024	Тести, результати тестування
9.	Виконання графічних документів	15.05.2024	Схема структурна класів програмного забезпечення , специфікація пакетів і компонентів
10	Оформлення пояснювальної записки	30.05.2024	Пояснювальна записка

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту виконується відповідно до “Програми та методики тестування”.

**Пояснювальна записка
до дипломного проєкту**

на тему: Розширення Visual Studio для автоматичної генерації юніт-тестів

КПІ.ПІ-0114.045300.02.81

Київ – 2024

ЗМІСТ

ВСТУП	5
1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ	6
1.1 Аналіз предметної області	6
1.2 Аналіз існуючих рішень.....	9
1.2.1 Аналіз відомих програмних продуктів.....	10
1.2.2 Аналіз відомих алгоритмічних та технічних рішень.....	17
1.3 Опис бізнес-процесів.....	19
1.4 Постановка задачі	22
Висновки до розділу	23
2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	24
2.1 Варіанти використання програмного забезпечення.....	24
2.2 Аналіз системних вимог.....	30
2.3 Розроблення функціональних вимог	30
2.4 Розроблення нефункціональних вимог	37
Висновки до розділу	38
3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	39
3.1 Архітектура програмного забезпечення.....	39
3.2 Обґрунтування вибору засобів розробки	45
3.3 Конструювання програмного забезпечення.....	49
3.3.1 Опис проєкту NetTestX.Razor.....	49
3.3.2 Опис проєкту NetTestX.CodeAnalysis.....	50
3.3.3 Опис проєкту NetTestX.CodeAnalysis.Workspaces	51
3.3.4 Опис проєкту NetTestX.VSIX.....	52
3.3.5 Опис алгоритму підстановки аргументів типу.....	54
3.3.6 Опис утиліт, бібліотек та іншого стороннього програмного забезпечення	58
3.4 Аналіз безпеки даних	60
Висновки до розділу	60

4	АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	
61		
4.1	Аналіз якості ПЗ.....	61
4.1.1	Індекс підтримуваності.....	61
4.1.2	Цикломатична складність.....	62
4.1.3	Глибина наслідування.....	63
4.1.4	Зв'язність коду.....	64
4.1.5	Оцінка ефективності (економія часу).....	64
4.1.6	Оцінка ефективності (покриття тестами).....	65
4.2	Опис процесів тестування.....	66
4.3	Опис контрольного прикладу.....	74
	Висновки до розділу	83
5	РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	84
5.1	Розгортання програмного забезпечення.....	84
5.2	Супровід програмного забезпечення.....	89
	Висновки до розділу	89
	ВИСНОВКИ.....	90
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	92
	ДОДАТКИ.....	94
	Додаток А. Звіт подібності Unichack	94

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

IDE	– Integrated Development Environment – інтегроване середовище розробки.
API	– Application programming interface, прикладний програмний Інтерфейс
SDK	– Software development kit
TDD	– Test-driven development
BDD	– Behavior-driven development
SUT	– System under test
GUI	– Graphical user interface
CLI	– Command-line interface
AAA	– Arrange-Act-Assert
GWT	– Given-When-Then
UX	– User experience
CLR	– Common Language Runtime
BCL	– Base Class Library
DTE	– Design Time Extensibility
CLI	– Common Language Infrastructure
CIL	– Common Intermediate Language
MVVM	– Model-View-Viewmodel
CD	– Continuous Deployment

ВСТУП

Тестування програмного забезпечення є невід’ємною складовою життєвого циклу ПЗ і допомагає перевірити відповідність заявлених до продукту вимог фактичній реалізованій функціональності. Незважаючи на те, що команди зазвичай мають окремих спеціалістів, що проводять тестування, розробники також часто беруть у ньому участь шляхом створення юніт-тестів (модульних тестів).

Написання юніт-тестів програмістами часто може бути рутинним і нудним завданням із великою кількістю шаблонного коду, особливо враховуючи популярність технологій розробки на зразок TDD. Розробникам було би добре мати певний механізм генерації коду для юніт-тестів, що буде виконувати більшу частину роботи, лишаючи дописати лише кілька рядків для перевірки і порівняння результатів.

Враховуючи популярність платформи .NET і мови C# [1], а також середовища розробки Visual Studio, що підтримує створення власних плагінів, розробка розширення для автоматичної генерації юніт-тестів на мові C# стає вкрай актуальною. Наразі існують кілька програмних рішень, що дозволяють автоматизувати певні аспекти (ReSharper [2], Unitverse [3] тощо), проте їх функціонал можна було б розширити: деякі генерують дуже мало коду, інші - код, що не компілюється тощо.

Метою розробки є підвищення ефективності роботи програмістів за рахунок зменшення часу, що витрачається на створення юніт-тестів під час розробки.

Сферою застосування розширення є розробка програмного забезпечення на платформі .NET і мові C#, як-от бібліотек, веб-сервісів, десктопних, ігрових, консольних застосунків тощо.

1 ПЕРЕДПРОЄКТНЕ ОБСТЕЖЕННЯ ПРЕДМЕТНОЇ ОБЛАСТІ

1.1 Аналіз предметної області

Модульне тестування не є новим концептом у розробці програмного забезпечення. Воно існує ще з 1970-х років і все ще є одним із найкращих способів для розробника покращити якість коду і описати функціональні вимоги до класу чи методу [4]. Вперше було запропоноване Кентом Беком для мови Smalltalk у 1970-х роках і з часом поширилось на багато інших мов програмування, ставши дуже корисним [4][5].

Модульний тест (юніт-тест) – це фрагмент коду, що виконує певну послідовність подій (зазвичай викликає методи) і перевіряє їхній результат на відповідність певним припущенням [4][5]. Якщо припущення щодо результату виявляються неправильними, тест вважається проваленим. Межі юніт-тесту можуть бути різними, від одного методу до кількох різних типів [4]. Предмет тестування часто описується терміном SUT (System under test).

Однією із основних характеристик юніт-тестів є “ізолюваність” або “незалежність”. Це означає, що тест виконується без справжніх залежностей і може бути повністю контрольованим [4]. При цьому залежності імітуються за допомогою “моків” або віртуалізованих об’єктів [6]. Такий підхід спрощує тести, робить їх швидше, а результати - більш послідовними, оскільки вони використовують задані наперед значення замість, наприклад, поточної дати та часу [4].

Зазвичай написанням юніт-тестів займаються самі програмісти [4][5][6]. Проте лише модульного тестування (так само як і, наприклад, інтеграційного тестування) недостатньо для забезпечення якості ПЗ розробником. Для цього також потрібно слідувати TDD, переглядати код, робити рефакторинг тощо [6].

Юніт-тест має відповідати наступним характеристикам [4]:

- має бути автоматизованим і повторюваним;
- має бути легким в написанні;

- має бути доречним завтра;
- має легко запускатися натиском однієї кнопки;
- має бути швидким;
- має мати стійкий незмінний результат;
- має мати повний контроль над SUT;
- має бути ізолюваним;
- при проваленні тесту має бути легко визначити що очікувалось і як знайти проблему.

Під час роботи у програмістів може виникнути питання щодо того, коли саме писати юніт-тести. Багато розробників вважають, що це найкраще робити після того як програмне забезпечення вже написано, проте все більше віддають перевагу написанню юніт-тестів перед написанням самого коду. Цей підхід має назву TDD (Test-driven development) [4][5][6]. TDD відрізняється від більшості підходів до розробки тим, що спочатку пишеться тест, що не проходить, потім пишеться власне код, після чого тест починає проходити.

Важливо розуміти, що використання TDD не гарантує успіху проєкту чи якості написаних юніт-тестів. Якщо використовувати TDD правильно, то можна значно покращити якість і надійність коду, зменшити кількість багів, полегшити їх виправлення. З іншого боку, неправильне використання цього підходу може призвести до марної втрати часу, зниження мотивації та якості коду [4].

Однією із суттєвих і водночас маловідомих переваг TDD є той факт, що спостерігаючи за тим, як тест спочатку не проходить, а потім проходить успішно, програміст фактично тестує сам тест [4]. Якщо, наприклад, тест, що був написаний перед безпосередньою реалізацією системи, проходить одразу, то це свідчить про проблеми в самому тесті.

Зазвичай для написання юніт-тестів використовуються тестові фреймворки. Тестовий фреймворк – це набір бібліотек, що надає розробникам API для написання юніт-тестів. Також за допомогою таких бібліотек можна автоматизувати їх виконання, наприклад, як один із кроків при розгортанні

програмного забезпечення [4]. Насправді тести можна писати повністю вручну без їх використання, але таким чином розробник втрачає можливість до автоматизації і буде змушений запускати тести кожен раз, як робляться зміни до кодової бази [4]. Ще одним недоліком такого підходу є більша кількість шаблонного коду, що треба писати при створенні кожного тесту.

API тестового фреймворку містить:

- базові класи або інтерфейси для тестових класів;
- атрибути (декоратори), що позначають тестові методи;
- спеціальні класи, що містять методи для перевірки результатів тестів.

Тестовий фреймворк також містить сервіс, що запускає і виконує тести (test runner) [4]. Цей сервіс може бути як консольним, так і мати GUI. Він:

- знаходить тести в коді;
- автоматично виконує тести;
- надає статус тестів під час виконання;
- може бути автоматизований через CLI.

Як вже було зазначено, залежності в юніт-тестах зазвичай імітуються шляхом створення моків. Мок – це фіктивний об’єкт, що допомагає визначити, чи тест пройшов успішно шляхом перевірки його взаємодії із системою [4]. Класичним прикладом є база даних: вона потребує багато часу для запуску, її треба тримати пустою (для уникнення побічних ефектів) і, якщо вона розташована на віддаленому сервері, це прив’язує тести до фізичної локації в мережі [5]. Таким чином, легше створити мок замість справжньої бази даних, що буде розташований в оперативній пам’яті.

Для створення моків для юніт-тестів часто використовуються спеціальні бібліотеки моків, або ізоляційні бібліотеки. Бібліотека моків – це набір API, що значно полегшує створення фіктивних об’єктів [4]. Якщо такі бібліотеки були написані правильно, то вони здатні зекономити час програміста на написання повторюваного коду для створення моків. Часто тестові

фреймворки та бібліотеки моків використовуються разом. Бібліотеки моків також добре поєднуються зі структурою AAA (arrange-act-assert) [4].

AAA (arrange-act-assert) – це підхід до структурування юніт-тестів, що допомагає розділити тест на окремі логічні компоненти. Перша частина, arrange, створює необхідні об'єкти для тесту (наприклад, SUT і моки). Друга частина, act, виконує необхідні дії та отримує результат. Третя частина, assert, перевіряє результат на відповідність очікуваному [4]. Схожий підхід, що застосовується у BDD, має назву GWT (Given-When-Then) [6].

Проблема автоматизованої генерації тестів не є новою, і програмні рішення для неї у різних формах існують для більшості популярних мов, як-от C# [2] [3] [7] [8], JavaScript [9], Python [10], Java [11]. Проте їх використання в реальній розробці наразі не набуло значної популярності [1]. У той час як більшість з них підтримує безпосередню генерацію тестів, деякі [2] [12] популярні рішення не генерують ніякої логіки для самих тестів, а виключно створюють тестовий файл (клас), при цьому лишаючи всю роботу програмісту. Також у деяких застосунках відсутні або дуже обмежені можливості конфігурації (налаштування) [8] [9] [12], а в інших (більшості) – відсутня можливість створення тестового проєкту, що міститиме згенеровані тести [2] [7] [9] [12]. Описані вище проблеми будуть вирішені в дипломному проєкті, надаючи користувачеві можливість конфігурації як глобальної, так і окремих тестів, а також можливість генерації тестових проєктів, при цьому обравши потрібний тестовий фреймворк, бібліотеку моків тощо.

1.2 Аналіз існуючих рішень

Проаналізуємо відоме на сьогодні алгоритмічне забезпечення у даній області та технічні рішення, що допоможуть у реалізації розширення для автоматичної генерації юніт-тестів. Далі будуть розглянуті готові програмні рішення, допоміжні програмні засоби та засоби розробки.

1.2.1 Аналіз відомих програмних продуктів

Зрозуміло, що при розробці розширення для генерації юніт-тестів першим і головним вибором є вибір середовища розробки, для якого воно пишеться. Наразі основними та найбільш популярними IDE для розробки на платформі .NET і мові C# є Visual Studio, Rider та Visual Studio Code.

Visual Studio – це середовище розробки від Microsoft, що є “рідним” для .NET і найбільш поширене серед .NET розробників [12]. Окрім мови C# воно також часто використовується для розробки на мовах F#, VB, C++, JavaScript, Python та ін. Також Visual Studio має багато можливостей для відлагодження, статичного та динамічного аналізу програм і суміжних засобів, що допомагають розробці, наприклад, можливість згенерувати діаграми класів [12]. Підтримує створення розширень, дозволяючи взаємодію з майже будь-якою частиною IDE, включаючи редактор коду, вікна, налаштування тощо. Велика популярність Visual Studio робить її привабливим варіантом для розробки розширення. Погана документація може ускладнити цей процес, проте велика кількість існуючих розширень робить його значно легшим [13].

Rider є ще одним популярним варіантом IDE від JetBrains, здебільшого орієнтованим саме на розробку на мові C#, хоча інші мови також підтримуються, наприклад, C++ [14]. Rider має кілька суттєвих переваг над Visual Studio. Основними з них є менше використання системних ресурсів (процесору та оперативної пам’яті) та значно кращий статичний аналіз коду [14]. Фактично функціонал ReSharper, дуже поширеного розширення для Visual Studio від JetBrains, “вбудований” у Rider “з коробки”. Це середовище також підтримує розробку плагінів, проте їх кількість значно менша, ніж у Visual Studio, і розробнику дуже важко знайти будь-яку документацію з їх розробки.

Visual Studio Code є поширеним текстовим редактором від Microsoft з частково відкритим кодом [1]. Загалом він не прив’язаний до мови, але часто використовується для розробки на мовах JavaScript, TypeScript, Python, а також для HTML і CSS [15]. Підтримує написання розширень і має хорошу

документацію для цього, як наслідок – дуже велика кількість написаних плагінів. Для написання розширень здебільшого використовується мова TypeScript [15]. Незважаючи на високу популярність і легкість створення розширень, саме для мови C# Visual Studio Code не є хорошим варіантом. Підтримка мови може бути додана, якщо встановити спеціальний плагін, але навіть з ним відсутні багато критичних засобів розробки, що унеможливорює Visual Studio Code для розробки .NET проєктів середнього чи великого розміру.

Беручи до уваги переваги та недоліки всіх перелічених IDE для розробки на C#, вибір був зупинений на середовищі Visual Studio як найбільш популярному та багатофункціональному, а розробка розширення для нього буде корисною максимальній кількості розробників.

Існує кілька засобів для середовища Visual Studio, що допомагають автоматизувати написання юніт-тестів. У якості аналогів для порівняння були обрані розширення Unitverse [3], Unit Test Boilerplate Generator [7], ReSharper [2]. Також для порівняння було обрано вбудовану у Visual Studio команду Create unit tests для генерації тестів [12].

Visual Studio містить вбудовану можливість генерації юніт-тестів шляхом обрання команди Create unit tests для обраного типу [12] (рис. 1.1). Вона містить базові можливості налаштування тестів, наприклад, задання назви для згенерованого класу та методів. Також є можливість генерації тестового проєкту. Основною великою її перевагою є підтримка плагінів для тестових фреймворків. Розробник може створити розширення, що додасть підтримку для фреймворків, які не були доступні за замовченням (вбудована підтримка лише для MSTest, який використовується досить рідко). Розширення Test Generator NUnit, що додає підтримку фреймворку NUnit, є прикладом такого плагіну. Незважаючи на це, вбудована команда Create unit tests є досить рудиментарною за функціональністю, генерує мало коду (кожен тест по факту є заглушкою) і не підтримує кілька досить поширених функцій, як-от генерацію тестів для типів чи методів із модифікатором доступу internal.

UX також погіршується тим, що неможливо згенерувати тести для кількох типів одночасно і що команда не містить можливостей конфігурації у вікні Tools -> Options у Visual Studio, тобто всі налаштування мають задаватися безпосередньо під час створення тестів.

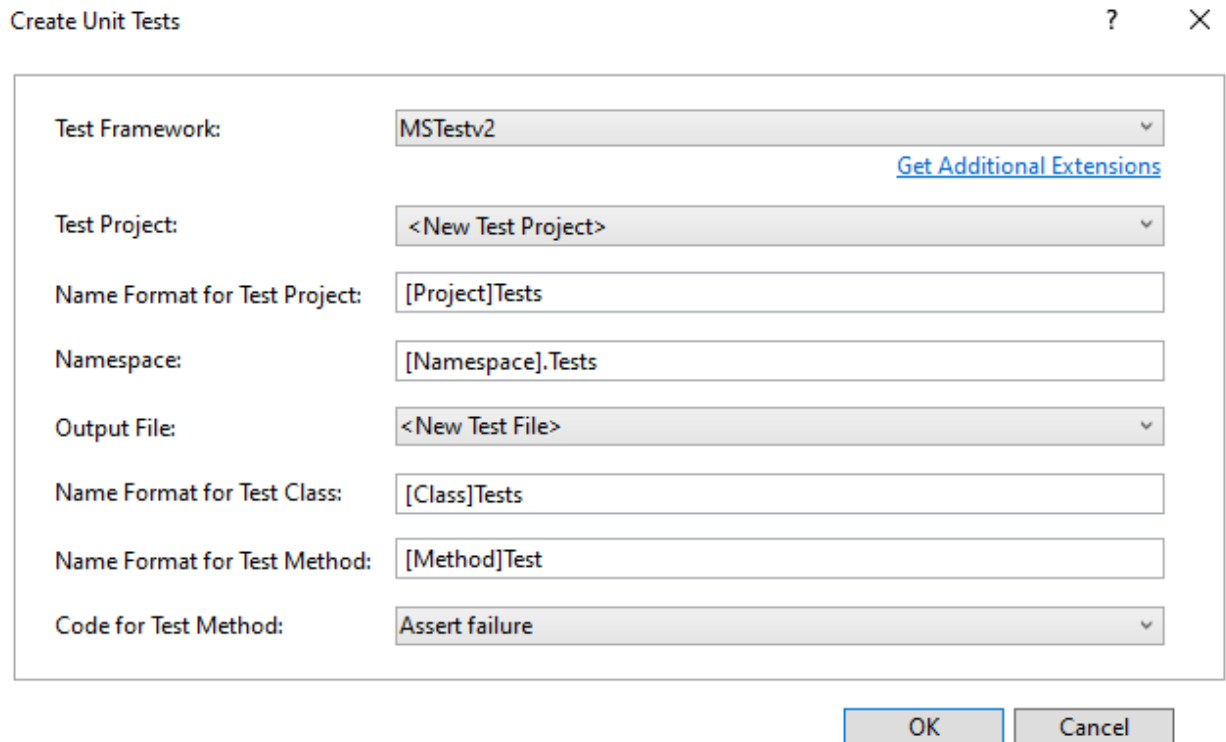


Рисунок 1.1 – Вікно генерації тестів, вбудоване у Visual Studio

Unitverse є, мабуть, найбільш просунутим розширенням подібного класу для генерації тестів [3]. Воно підтримує можливості генерації тестів як для файлів, так і для окремих типів (рис. 1.2), а також генерацію тестових проєктів (рис. 1.3), при цьому надає можливість обрання тестового фреймворку та мок-бібліотеки [3]. Перевагою є те, що розширення також містить багато налаштувань і можливостей конфігурації. З одного боку це добре, оскільки функціонал можна змінити під себе, проте деякі з цих опцій є трохи недоречними і майже ніколи не використовуються, особливо у вікні генерації тестів. Найбільшим недоліком розширення, мабуть, є те, що згенерований код не завжди компілюється та часто потребує корекцій. Особливо це помітно для generic типів та методів, а також параметрів, що передаються за посиланням (із використанням ref, in, out чи ref readonly). Загалом розширення є хорошим

варіантом для автоматичної генерації юніт-тестів, особливо враховуючи багатий функціонал і можливості конфігурації, при цьому варто пам'ятати про його недоліки, деякі з яких неможливо виправити. Проте розширення має відкритий код і все ще підтримується та оновлюється, тому можливо попросити виправити проблеми чи зробити внесок, виправивши їх власноруч.

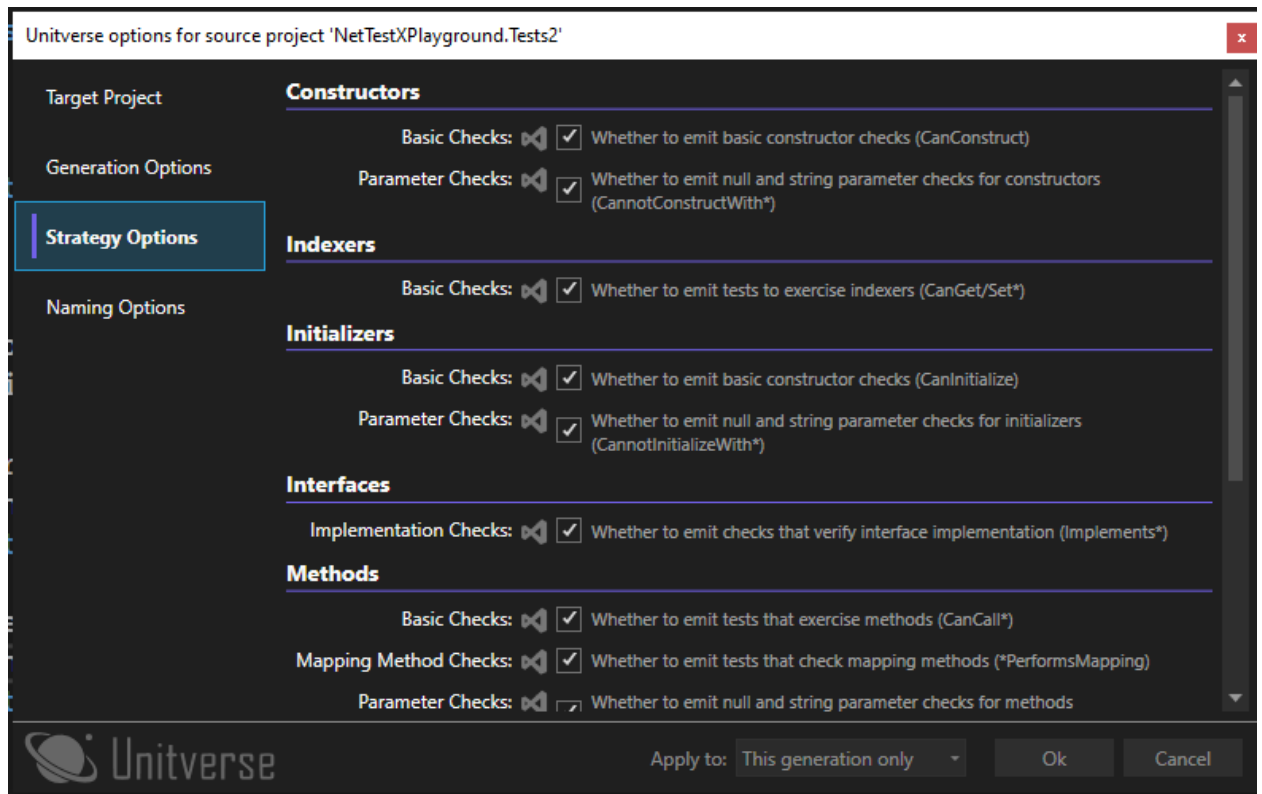


Рисунок 1.2 – Вікно генерації тестів Unitverse

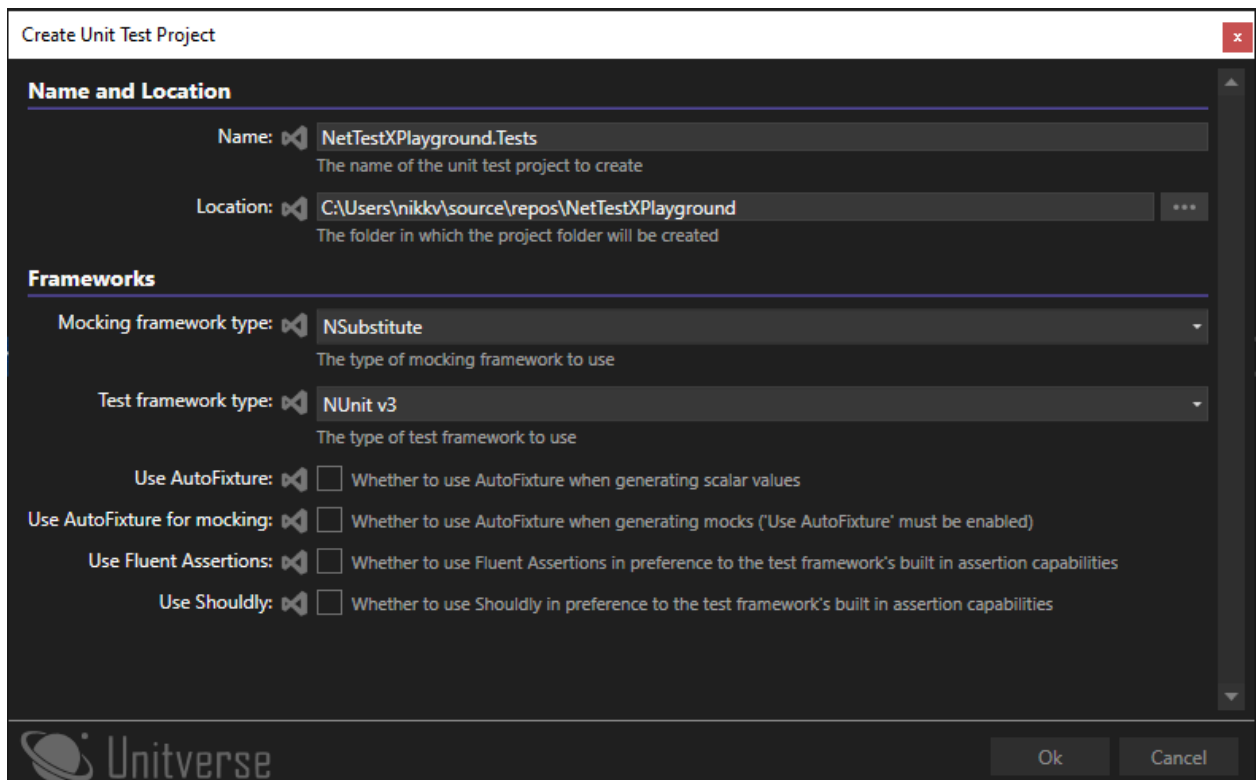


Рисунок 1.3 – Вікно генерації тестового проєкту Unitverse

Unit Test Boilerplate Generator є ще одним досить поширеним варіантом для генерації юніт-тестів у Visual Studio [7]. Підтримує генерацію юніт-тестів для заданого файлу (чи файлів) (рис. 1.4). Розширення має багато можливостей для конфігурації згенерованого коду і навіть дозволяє змінювати шаблон усього згенерованого файлу [7]. Шаблон може використовувати спеціальні змінні, визначені наперед, опис яких наведений у документації, що є дуже зручним для продвинутів варіантів використання. Наприклад, якщо тести генеруються для веб API, користувач зможе використовувати змінну `$HttpType$`, що містить значення HTTP методу, що тестується [7]. Розширення також має певні суттєві недоліки. Воно не містить функціоналу для створення тестового проєкту, а також не надає можливості генерувати тести для окремого типу (лише для цілих файлів). Окрім налаштувань генерації коду можливості конфігурації розширення дуже обмежені, наприклад, не можна вказати, для яких конкретно членів типу генерувати тести. Згенерований код часто може не компілюватися. Деякі аргументи методів замінені на “TODO”, generic типи та методи не підтримуються. Також варто зазначити, що інтерфейс розширення є досить примітивним і не відповідає загальній темі

Visual Studio. Загалом, Unit Test Boilerplate Generator є непоганим варіантом для ситуацій, коли потрібний повний контроль над шаблоном згенерованого файлу, при цьому треба врахувати всі перелічені недоліки. Проєкт має відкритий код і все ще підтримується, тому є можливість створити запит на виправлення помилок чи виправити їх власноруч.

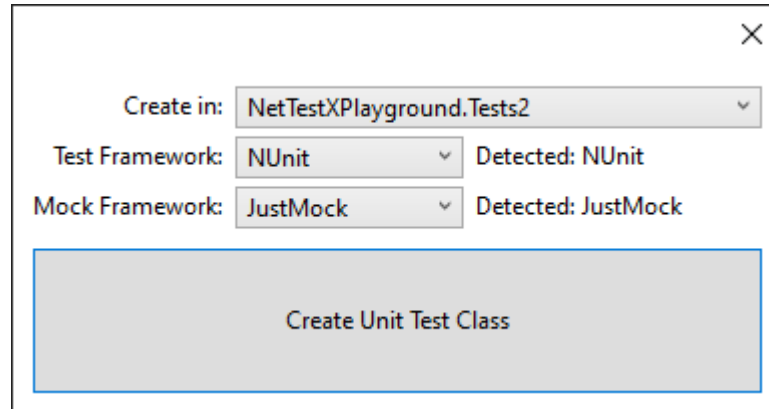


Рисунок 1.4 – Вікно генерації тестів Unit Test Boilerplate Generator

ReSharper є дуже відомим пропрієтарним розширенням для Visual Studio від компанії JetBrains [2]. Воно містить велику кількість засобів для підвищення продуктивності та якості повсякденної роботи, в тому числі статичні та динамічні аналізатори коду, засоби рефакторингу та навігації, засоби декомпіляції, засоби тестування [2]. Однією з можливостей також є генерація юніт-тестів (рис. 1.5). ReSharper дозволяє створення власних шаблонів для генерації файлів, при цьому їх може бути декілька, тобто при генерації тестів для заданого типу можна обрати потрібний шаблон. Також присутні базові можливості кастомізації тестів. Проте найважливішою перевагою використання ReSharper є можливість використання ІІІ для генерації тестів в останніх версіях розширення [2]. Якщо без цього згенеровані файли містили дуже мало коду (навіть менше, ніж вбудована у Visual Studio команда Create unit tests), то з увімкненим ІІІ коду стає набагато більше і розробнику здебільшого лишається дописати перевірку результату. Незважаючи на це, іноді код не компілюється, частково через неправильну генерацію ІІІ, а частково через банальні баги. Розширенню також не вистачає опції генерації тестових проєктів та більших можливостей до конфігурації. В цілому можна сказати, що ReSharper є відмінним розширенням для

покращення роботи розробників, і в тому числі для генерації тестів. Незважаючи на це, суттєвими недоліками є необхідність придбання ліцензії, неможливість генерації тестових проєктів, невелика кількість опцій для генерації та довгий час на генерацію при використанні ІІІ.

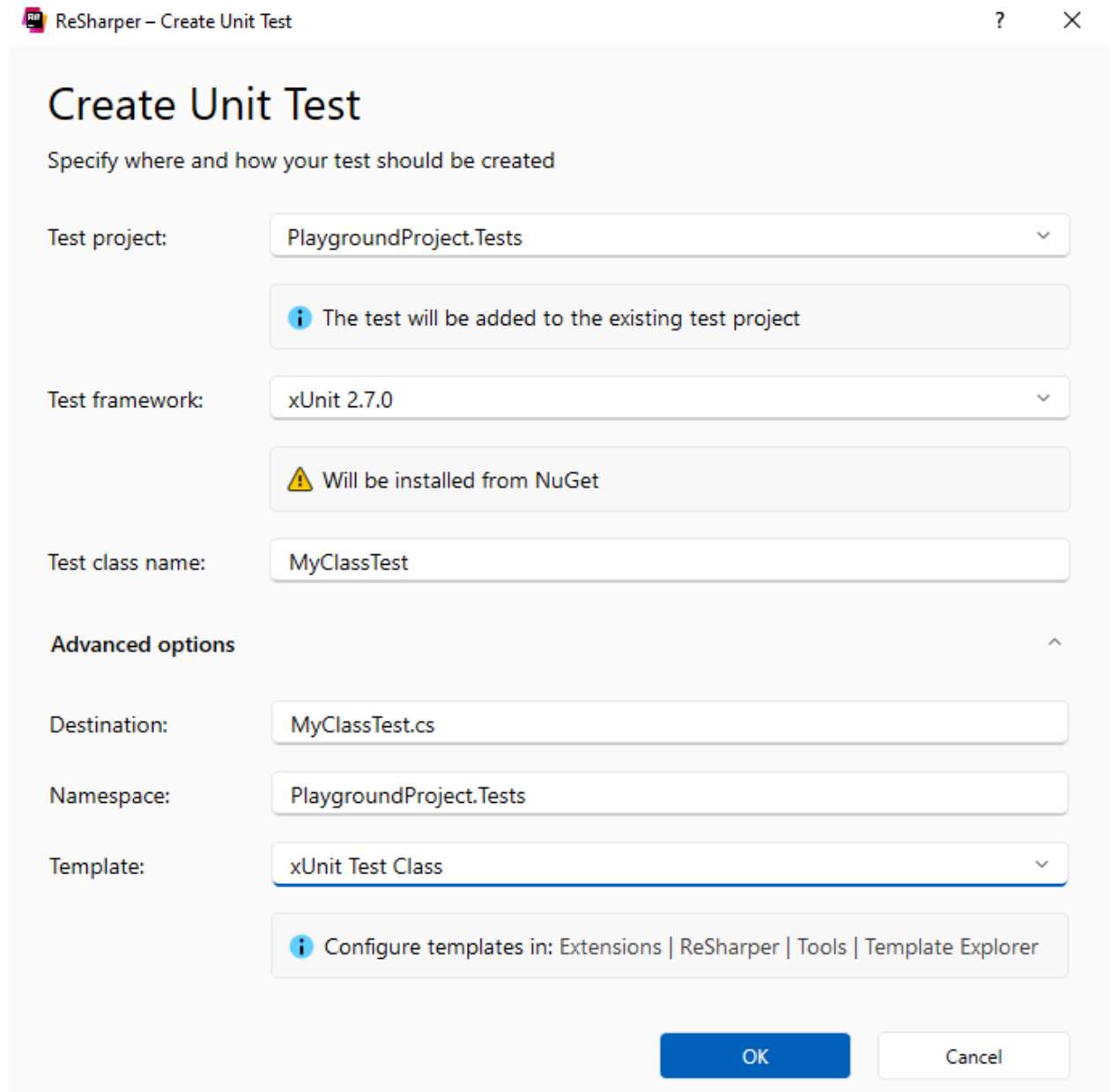


Рисунок 1.5 – Вікно генерації тестів ReSharper

Для порівняння проєкту з аналогом можна скористатись таблицею 1.1.

Таблиця 1.1 – Порівняння з аналогом

Функціонал	Дипломний проєкт	Unitverse	Unit Test Boilerplate Generator	ReSharper	Visual Studio
Кількість згенерованого коду	Велика	Велика	Середня	Середня	Мала
Чи компілюється згенерований код	Часто	Іноді	Іноді	Часто	Часто
Генерація тестових проєктів	Так	Так	Ні	Ні	Ні
Можливості персоналізації	Високі	Високі	Середні	Середні	Низькі
Зручність генерації	Висока	Висока	Низька	Середня	Низька
Модель поширення	Вільна	Вільна	Вільна	Пропрієтарна	Пропрієтарна

1.2.2 Аналіз відомих алгоритмічних та технічних рішень

Технічно задача генерації юніт-тестів може бути вирішена багатьма способами: у вигляді сайту [9], команди CLI [10], окремого застосунку [11] чи розширення для IDE [2] [3] [7]. Кожен із наведених способів має свої переваги та недоліки.

Генерація тестів на сайті дозволяє не встановлювати ніяке програмне забезпечення, при цьому очевидним мінусом є неможливість офлайн-використання та відсутність контексту: якщо завантажити на сайт лише один

файл або клас, для якого треба згенерувати тести, то неможливо отримати повну семантичну інформацію про код. З іншого боку, завантаження всіх файлів проєкту на сайт значно погіршує UX і може бути повільним для великих проєктів.

Використання команди CLI або десктопного рішення для генерації тестів може бути хорошим рішенням, якщо тести генеруються для вже зібраної бібліотеки (*.dll або *.exe), проте при використанні такого підходу деяка інформація про типи, що була доступна під час компіляції, втрачається [16], що може призвести до меншої ефективності або точності в генерації тестів. Також недоліком є необхідність встановлення окремого застосунку та його застосування поза IDE.

Реалізація рішення як розширення для IDE має набір переваг. По-перше, UX суттєво покращується за рахунок інтеграції з середовищем розробки: розширення можна встановити і використовувати прямо в IDE. По-друге, розширення мають безпосередній доступ до всього проєкту, що відкритий у середовищі, та можуть застосовувати синтаксичний та семантичний аналіз коду для генерації тестів. По-третє, вони можуть працювати без інтернету. У якості недоліку можна виділити специфіку до IDE: зазвичай розширення пишуться для конкретного IDE, та їх використання в інших середовищах неможливе. Проте цей недолік можна виправити, якщо розширення добре спроектовано: логіка роботи є спільною для всіх IDE, а інтерфейс взаємодії з середовищем є окремим тонким шаром.

Порівняння різних технічних рішень для генерації юніт-тестів наведено в таблиці 1.2.

Таблиця 1.2 – Порівняння технічних рішень

Характеристика	Розширення	Сайт	CLI	Десктопне рішення
Необхідність встановлення	Іноді	Ні	Так	Так

Продовження таблиці 1.2

Характеристика	Розширення	Сайт	CLI	Десктопне рішення
Інтеграція з IDE	Так	Ні	Ні	Ні
Можливості генерації	Високі	Середні	Середні	Середні
Офлайн-використання	Так	Ні	Так	Так
Загальний UX	Хороший	Середній	Середній	Середній

1.3 Опис бізнес-процесів

Процеси створення тестового проєкту та файлів з тестами описані за допомогою BPMN моделей.

На рисунку 1.6 зображений процес генерації тестового проєкту. Якщо розширення встановлене у Visual Studio, то контекстне меню для обраного існуючого проєкту у вікні дослідника рішень (Solution Explorer) містить команду “Generate Test Project” для генерації тестового проєкту. При обранні цієї команди користувачу буде показане вікно для задання необхідних параметрів генерації, зокрема назви проєкту та папки, що його міститиме. Усі параметри наперед заповнюються автоматично для покращення UX. Якщо користувач закриває вікно, то процес закінчується. Якщо він продовжує, то вікно закривається і система створює файл проєкту *.csproj і записує в нього згенерований XML-код. Після цього створений проєкт додається в рішення (файл *.sln), таким чином роблячи його доступним в IDE.

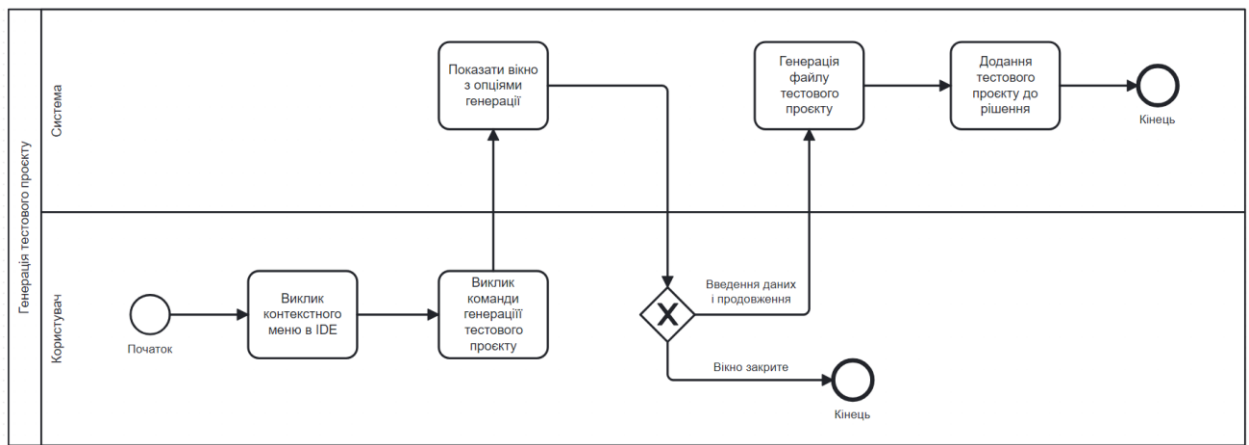


Рисунок 1.6 – Процес генерації тестового проєкту

На рисунках 1.7 та 1.8 зображений процес генерації юніт-тестів з контекстного меню. Таким може бути меню всередині редактора коду (Code Editor) або меню дослідника рішень (Solution Explorer). Якщо розширення встановлене у Visual Studio, то при виклику меню до нього можуть бути додані команди генерації юніт-тестів (“Generate Unit Tests...”) та розширеної генерації юніт-тестів (“Generate Unit Tests Advanced”). Ці команди видимі лише за умови, що обрані елементи містять хоч один тип, для якого можна згенерувати тести. Наприклад, якщо користувач обрав у вікні редактора коду публічний клас, то команди будуть видимі. Якщо клас приватний, то вони будуть вимкнені.

Кнопка генерації тестів (“Generate Unit Tests...”) дозволяє користувачу згенерувати тести “в один клік”, без спливаючих вікон та без перевизначення параметрів. Єдине, що він має задати – тестовий проєкт, до якого будуть додані тести. Якщо користувач обирає створити новий тестовий проєкт, то він слідує процесу генерації тестового проєкту, описаному на рис. 1.6. Після цього система генерує файл (або файли) *.cs, що містить згенеровані тести, у заданому проєкті.

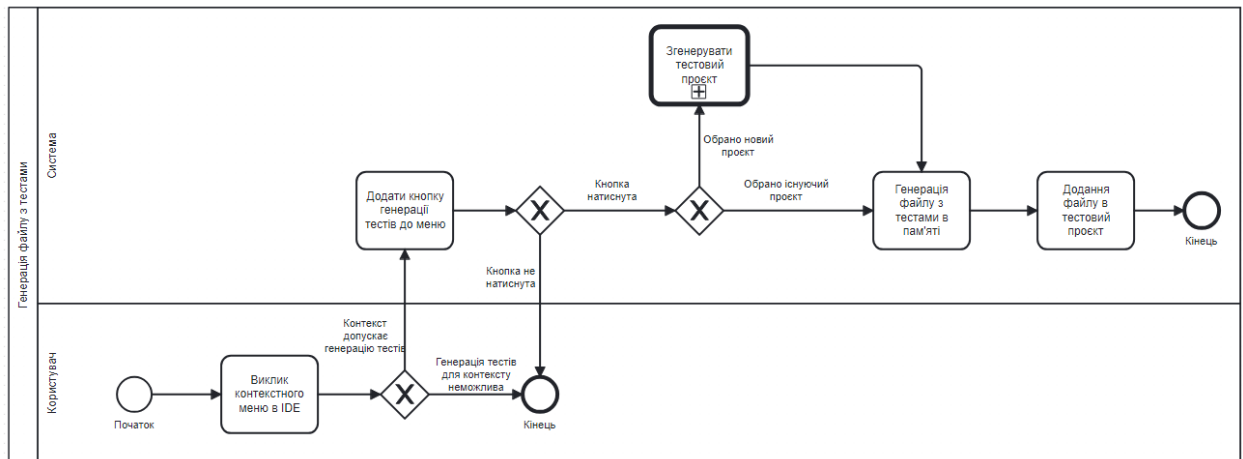


Рисунок 1.7 – Процес генерації юніт-тестів

Процес розширеного створення юніт-тестів дозволяє користувачу змінити налаштування за замовченням і кастомізувати процес генерації. При обранні команди “Generate Unit Tests Advanced” йому буде показане вікно, що містить список параметрів для генерації тестів, зокрема назва файлу, класу, простору імен тощо. Також користувач може власноруч обрати, для яких символів генерувати тести. Усі параметри наперед заповнюються автоматично для покращення UX. Після заповнення даних і продовження процес аналогічний до процесу звичайної генерації тестів. Розширений процес генерації тестів наведений на рис. 1.8.

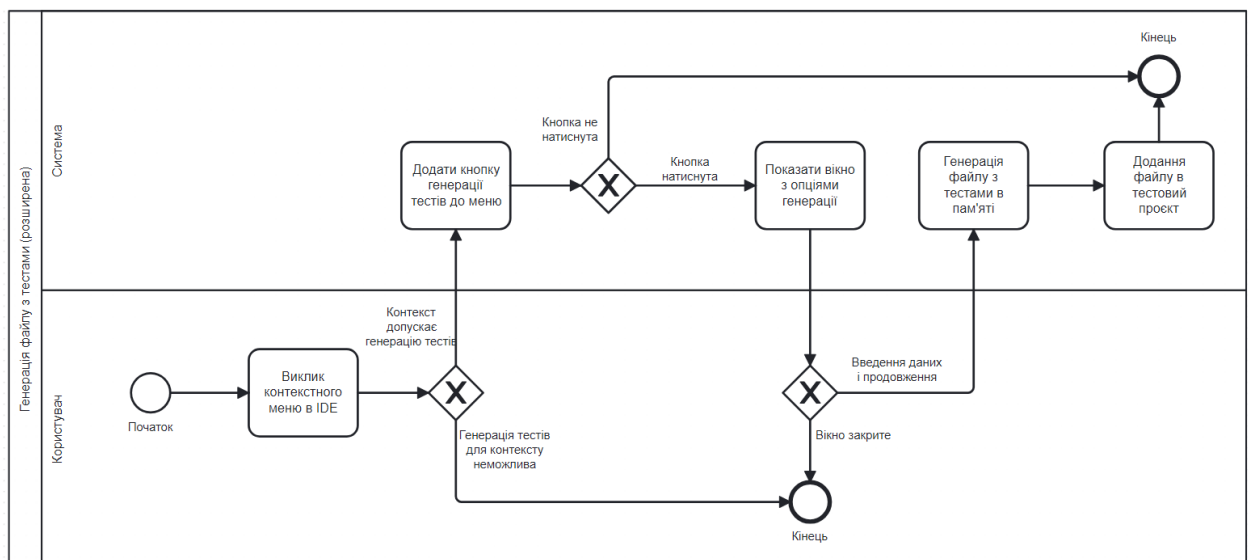


Рисунок 1.8 – Розширений процес генерації юніт-тестів

1.4 Постановка задачі

Задачею дипломного проєкту є розробка розширення для середовища Visual Studio для автоматичної генерації юніт-тестів. Метою розробки є підвищення ефективності роботи програмістів за рахунок зменшення часу, що витрачається на створення юніт-тестів під час розробки.

Розширення має підтримувати можливість згенерувати тести для обраного типу (типів) в окремому файлі *.cs (файл C#) в тестовому проєкті. Цей процес має бути інтуїтивним, а тому типи для генерації можуть обиратися як за допомогою текстового редактору (Code Editor), так і за допомогою вікна дослідника рішень (Solution Explorer). Також має бути присутня можливість створення власне тестового проєкту. Розширення має містити сторінку налаштувань у вікні Options середовища Visual Studio, при цьому ці налаштування використовуватимуться за замовченням, якщо користувач не перевизначив їх при генерації тестів.

Розширення має генерувати максимальну кількість шаблонного коду, лишаючи користувачеві дописати лише кілька рядків коду (здебільшого це логіка перевірки результату). Також воно має бути спрямоване на генерацію коду, що завжди компілюється (не містить помилок компіляції).

Розширення має вписуватись у загальний стиль оформлення Visual Studio і відчуватись природньо. Екранні форми мають відповідати темі Visual Studio.

Має бути згенерована інсталяційна версія програмного забезпечення. Розширення має бути викладено на порталі Visual Studio Marketplace.

Виходячи з наведеного вище можемо сформулювати повний список функціональних задач розробки:

- створення тестового проєкту;
- генерація юніт-тестів з вікна дослідника рішень;
- генерація юніт-тестів з вікна редактора коду;
- генерація юніт-тестів розширено з вікна дослідника рішень;
- генерація юніт-тестів розширено з вікна редактора коду;

— зміна налаштувань розширення.

Висновки до розділу

Перший розділ пояснювальної записки мав на меті передпроектне обстеження предметної області. Було проведено аналіз предметної області: описано поняття юніт-тесту, наведено його характеристики й особливості, зазначено роль юніт-тестів у розробці програмного забезпечення, використання підходу TDD, його переваги та недоліки. Були описані тестові фреймворки і бібліотеки моків, а також поняття AAA і GWT.

Було проведено аналіз кількох середовищ для розробки на мові C# та платформі .NET: Visual Studio, Rider, Visual Studio Code, описано їх переваги та недоліки. В результаті було обрано середовище Visual Studio для розробки розширення. Було проаналізовано набір існуючих розширень для генерації юніт-тестів, виділено сильні та слабкі сторони кожного з них, наведено порівняльну таблицю із розширенням на дипломний проєкт.

Було описано та наочно представлено бізнес-процеси розробки за допомогою BPMN-діаграм: процес генерації юніт-тестів (звичайний та розширений сценарій), процес генерації тектових проєктів.

Було формально визначено завдання: наведена його мета, зазначені цілі та задачі, що мають бути вирішені.

2 РОЗРОБЛЕННЯ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Варіанти використання програмного забезпечення

Головною функцією програмного забезпечення є генерація юніт-тестів. Діаграма варіантів використання наведена у графічному матеріалі, креслення 1.

В таблицях 2.1 - 2.9 наведені варіанти використання програмного забезпечення.

Таблиця 2.1 - Варіант використання UC-1

Use case name	Створити тестовий проєкт
Use case ID	UC-01
Goals	Створення тестового проєкту та додавання його до відкритого рішення
Actors	Користувач
Trigger	Користувач бажає створити тестовий проєкт
Pre-conditions	-
Flow of Events	Користувач обирає існуючий проєкт у вікні дослідника рішень і відкриває контекстне меню. У ньому він обирає команду “Generate Test Project...”.
Extension	Якщо у вікні дослідника рішень обрано щось інше, окрім проєкту C#, то команда “Generate Test Project...” не з’являється у контекстному меню.
Post-Condition	Відкриття вікна створення проєкту

Таблиця 2.2 - Варіант використання UC-2

Use case name	Показати вікно створення тестового проєкту
Use case ID	UC-02
Goals	Задання параметрів для нового тестового проєкту, його створення та додавання до відкритого рішення
Actors	Користувач
Trigger	Користувач бажає створити тестовий проєкт

Продовження таблиці 2.2

Pre-conditions	Користувач обрав команду “Generate Test Project...” з контекстного меню обраного проєкту у вікні дослідника рішень або з переліку проєктів при генерації тестів
Flow of Events	Користувачу показується вікно створення тестового проєкту. Значення параметрів задані за замовченням, проте користувач може їх змінити. Задаються назва тестового проєкту, директорія, в якій буде створено проєкт, тестовий фреймворк і бібліотека моків, необхідність задання InternalsVisibleTo. Після задання параметрів користувач натискає кнопку “Continue”
Extension	У випадку введення некоректних даних кнопка “Continue” стає неактивною. Якщо якесь конкретне поле введено некоректно, то воно підсвічується червоним надписом.
Post-Condition	Створення тестового проєкту, додання його до відкритого рішення

Таблиця 2.3 - Варіант використання UC-3

Use case name	Згенерувати тести з вікна дослідника рішень
Use case ID	UC-03
Goals	Генерація файлів C# з юніт-тестами та додання їх до заданого тестового проєкту
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести для заданого файлу C#
Pre-conditions	-
Flow of Events	Користувач обирає файл C# з проєкту у вікні дослідника рішень і відкриває контекстне меню. У ньому він обирає меню “Generate Unit Tests”, після чого обирає тестовий проєкт зі списку доступних проєктів у рішенні. Якщо обрана команда “Generate Test Project...”, то користувач слідує варіанту використання UC-2. Якщо обраний файл (файли) містить більше одного типу, для якого можливо згенерувати тести, то користувачу може бути показано попередження про це з можливістю відмінити генерацію.

Продовження таблиці 2.3

Extension	Якщо обраний елемент у вікні дослідника рішень не є файлом C#, то команда “Generate Unit Tests” не з’являється у контекстному меню. Якщо обраний файл не містить типів, для яких можливо згенерувати тести, то команда “Generate Unit Tests” стає неактивною.
Post-Condition	Генерація файлів C# з тестами, додання їх до заданого тестового проєкту

Таблиця 2.4 - Варіант використання UC-4

Use case name	Згенерувати тести з вікна редактора коду
Use case ID	UC-04
Goals	Генерація файлів C# з юніт-тестами та додання їх до заданого тестового проєкту
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести для заданого типу
Pre-conditions	-
Flow of Events	Користувач наводить курсор на символ у вікні редактора коду і відкриває контекстне меню. У ньому він обирає меню “Generate Unit Tests”, після чого обирає тестовий проєкт зі списку доступних проєктів у рішенні. Якщо обрана команда “Generate Test Project...”, то користувач слідує варіанту використання UC-2.
Extension	Якщо обраний символ у вікні редактора коду не є типом, то команда “Generate Unit Tests” не з’являється у контекстному меню. Якщо для обраного типу можливо згенерувати тести, то команда “Generate Unit Tests” стає неактивною.
Post-Condition	Генерація файлу C# з тестами, додання його до заданого тестового проєкту

Таблиця 2.5 - Варіант використання UC-5

Use case name	Згенерувати тести розширено з вікна дослідника рішень
Use case ID	UC-05

Продовження таблиці 2.5

Goals	Генерація файлів C# з юніт-тестами та додання їх до заданого тестового проєкту
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести для заданого файлу C# та змінити параметри за замовченням
Pre-conditions	-
Flow of Events	Користувач обирає файл C# з проєкту у вікні дослідника рішень і відкриває контекстне меню. У ньому він обирає меню “Generate Unit Tests Advanced...”. Якщо обраний файл (файли) містить більше одного типу, для якого можливо згенерувати тести, то користувачу може бути показано попередження про це з можливістю відмінити генерацію.
Extension	Якщо обраний символ у вікні редактора коду не є типом, то команда “Generate Unit Tests Advanced...” не з’являється у контекстному меню. Якщо для обраного типу можливо згенерувати тести, то команда “Generate Unit Tests Advanced...” стає неактивною.
Post-Condition	Відкриття вікна розширеної генерації тестів

Таблиця 2.6 - Варіант використання UC-6

Use case name	Згенерувати тести розширено з вікна редактора коду
Use case ID	UC-06
Goals	Генерація файлу C# з юніт-тестами та додання його до заданого тестового проєкту
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести для заданого типу та змінити параметри за замовченням
Pre-conditions	-
Flow of Events	Користувач наводить курсор на символ у вікні редактора коду і відкриває контекстне меню. У ньому він обирає меню “Generate Unit Tests Advanced”.

Продовження таблиці 2.6

Extension	Якщо обраний символ у вікні редактора коду не є типом, то команда “Generate Unit Tests Advanced” не з’являється у контекстному меню. Якщо для обраного типу можливо згенерувати тести, то команда “Generate Unit Tests Advanced” стає неактивною.
Post-Condition	Відкриття вікна розширеної генерації тестів

Таблиця 2.7 - Варіант використання UC-7

Use case name	Показати вікно розширеної генерації тестів
Use case ID	UC-07
Goals	Генерація файлу C# з юніт-тестами та додання його до заданого тестового проєкту
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести для заданого типу та змінити параметри за замовченням
Pre-conditions	Користувач обрав команду “Generate Unit Tests Advanced...” з контекстного меню символу у вікні редактора коду або з контекстного меню файлу у вікні дослідника рішень
Flow of Events	Користувачу показується вікно розширеної генерації тестів. Значення параметрів задані за замовченням, проте користувач може їх змінити. Задаються тестовий проєкт, назва файлу з тестами, назва класу з тестами, простір імен класу, а також конкретні тести, які треба згенерувати. Після цього користувач натискає кнопку “Continue”.
Extension	У випадку введення некоректних даних кнопка “Continue” стає неактивною. Якщо якийсь конкретне поле введено некоректно, то воно підсвічується червоним надписом.
Post-Condition	Генерація файлів C# з тестами, додання їх до заданого тестового проєкту

Таблиця 2.8 - Варіант використання UC-8

Use case name	Попередити про генерацію кількох типів
Use case ID	UC-08

Продовження таблиці 2.8

Goals	Попередження користувача про те, що в результаті генерації може бути згенеровано кілька файлів з тестами для кількох типів
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести
Pre-conditions	Користувач обрав команду “Generate Unit Tests” або “Generate Unit Tests Advanced...” з контекстного меню символу у вікні редактора коду або з контекстного меню файлу у вікні дослідника рішень
Flow of Events	Користувачу показується попередження з повідомленням про те, що обраний файл чи файли містять кілька можливих типів для генерації тестів, і що для кожного з них буде згенеровано окремий файл з тестами. Користувач може або натиснути “Yes” для підтвердження, або “No” для відміни.
Extension	У випадку, якщо користувач у налаштуваннях розширення задав опцію не показувати попередження про генерацію кількох типів, то попередження не покажеться, а поведінка розширення буде такою самою, як якби користувач натиснув кнопку “Yes”.
Post-Condition	Генерація файлів C# з тестами, якщо користувач обрав опцію “Yes”; зупинка генерації, якщо користувач обрав “No”

Таблиця 2.9 - Варіант використання UC-9

Use case name	Змінити налаштування
Use case ID	UC-09
Goals	Зміна налаштувань розширення
Actors	Користувач
Trigger	Користувач бажає згенерувати юніт-тести
Pre-conditions	Користувач відкрив вікно Tools -> Options у Visual Studio
Flow of Events	Користувач відкриває вкладку налаштувань розширення. На сторінках “General”, “Code”, “Advanced” користувач задає різні параметри для генерації тестів, наприклад, назву файлу з тестами, тестового класу, простору імен (використовуючи шаблон), або чи додавати AAA-коментарі в згенеровані тести.

Продовження таблиці 2.9

Extension	-
Post-Condition	Налаштування користувача зберігаються середовищем Visual Studio та використовуватимуться як параметри за замовченням при генерації тестів

2.2 Аналіз системних вимог

Системні вимоги аналогічні до вимог для середовища Visual Studio 2022.

Програмне забезпечення повинно функціонувати на персональних комп'ютерах архітектури x64.

Мінімальна рекомендована версія Visual Studio для роботи: Visual Studio 2022. Рекомендовані вимоги до технічних засобів для середовища розробки [12]:

- тип процесору: Intel Core i5;
- об'єм ОЗП: 8 Гб;
- об'єм вільного дискового простору: 20 Гб.

2.3 Розроблення функціональних вимог

Програмне забезпечення розділене на модулі. Кожен модуль має свій певний набір функцій. В таблиці 2.10 наведено загальну модель вимог, а в таблиці 2.11 наведений опис функціональних вимог до програмного забезпечення. Матрицю трасування вимог можна побачити на рисунку 2.1.

Таблиця 2.10 – Загальна модель вимог

№	Назва	ID Вимоги	Пріоритети	Ризики
1	Створення тестового проєкту	FR-1	Високий	Високий
1.1	Перегляд сторінки створення тестового проєкту	FR-2	Високий	Низький

Продовження таблиці 2.10

№	Назва	ID Вимоги	Пріоритети	Ризики
1.1.1	Перевірка валідності назви тестового проєкту	FR-3	Середній	Низький
2	Генерація юніт-тестів з вікна дослідника рішень	FR-4	Високий	Високий
2.1	Перевірка обраних елементів у вікні дослідника рішень	FR-5	Середній	Середній
2.2	Обрання тестового проєкту	FR-6	Високий	Низький
2.3	Попередження про генерацію кількох типів	FR-7	Низький	Низький
3	Генерація юніт-тестів з вікна редактора коду	FR-8	Високий	Високий
3.1	Перевірка обраного символу у вікні дослідника рішень	FR-9	Середній	Середній
3.2	Обрання тестового проєкту	FR-6	Високий	Низький
3.3	Попередження про генерацію кількох типів	FR-7	Низький	Низький
4	Генерація юніт-тестів розширено з вікна дослідника рішень	FR-10	Високий	Високий
4.1	Перевірка обраних елементів у вікні дослідника рішень	FR-5	Середній	Середній
4.2	Попередження про генерацію кількох типів	FR-7	Низький	Низький
4.3	Перегляд сторінки розширеної генерації тестів	FR-11	Високий	Низький

Продовження таблиці 2.10

№	Назва	ID Вимоги	Пріоритети	Ризики
4.3.1	Обрання тестового проєкту	FR-6	Високий	Низький
4.3.2	Перевірка валідності назви файлу з тестами	FR-12	Середній	Низький
4.3.3	Перевірка валідності назви тестового класу	FR-13	Середній	Низький
4.3.4	Перевірка валідності назви простору імен тестового класу	FR-14	Середній	Низький
4.3.5	Обрання окремих юніт-тестів для генерації	FR-15	Середній	Низький
5	Генерація юніт-тестів розширено з вікна редактора коду	FR-16	Високий	Високий
5.1	Перевірка обраного символу у вікні дослідника рішень	FR-9	Середній	Середній
5.2	Попередження про генерацію кількох типів	FR-7	Низький	Низький
5.3	Перегляд сторінки розширеної генерації тестів	FR-11	Високий	Низький
5.3.1	Обрання тестового проєкту	FR-6	Високий	Низький
5.3.2	Перевірка валідності назви файлу з тестами	FR-12	Середній	Низький
5.3.3	Перевірка валідності назви тестового класу	FR-13	Середній	Низький
5.3.4	Перевірка валідності назви простору імен тестового класу	FR-14	Середній	Низький

Продовження таблиці 2.10

№	Назва	ID Вимоги	Пріоритети	Ризики
5.3.5	Обрання окремих юніт-тестів для генерації	FR-15	Середній	Низький
6	Зміна налаштувань розширення	FR-17	Високий	Низький
6.1	Задання шаблону назви тестового файлу	FR-18	Високий	Низький
6.2	Задання шаблону назви класу з тестами	FR-19	Високий	Низький
6.3	Задання шаблону назви простору імен класу з тестами	FR-20	Високий	Низький
6.4	Задання необхідності тестування internal типів	FR-21	Середній	Низький
6.5	Задання необхідності генерації AAA-коментарів у згенерованих тестах	FR-22	Середній	Низький
6.6	Задання необхідності підстановки аргументів типів для універсальних типів / членів типів	FR-23	Середній	Низький
6.7	Задання необхідності попередження про генерацію кількох типів	FR-24	Низький	Низький

Таблиця 2.11 – Перелік функціональних вимог

Назва	Опис
FR-1	Створення тестового проєкту Обрання команди “Generate Test Project...” у контекстному меню проєкту у вікні дослідника рішень.
FR-2	Перегляд сторінки створення тестового проєкту Можливість задання назви проєкту, директорії з проєктом (через діалог Windows), тестового фреймворку, бібліотеки моків, задання InternalsVisibleTo.
FR-3	Перевірка валідності назви тестового проєкту Назва проєкту має відповідати правилам іменування збірок .NET.
FR-4	Генерація юніт-тестів з вікна дослідника рішень Можливість генерації тестів, обравши команду “Generate Unit Tests” у вікні дослідника рішень.
FR-5	Перевірка обраних елементів у вікні дослідника рішень Перевірка файлів (або інших елементів списку) дослідника рішень, які обрав користувач, щоб вирішити, чи додавати до контекстного меню команди генерації тестів.
FR-6	Обрання тестового проєкту Обрання існуючого тестового проєкту або створення нового зі списку проєктів.
FR-7	Попередження про генерацію кількох типів Показати попередження про те, що для обраних елементів буде згенеровано декілька файлів з тестами. Користувач може або відмінити генерацію, або продовжити.
FR-8	Генерація юніт-тестів з вікна редактора коду Можливість генерації тестів, обравши команду “Generate Unit Tests” у вікні редактора коду.

Продовження таблиці 2.11

Назва	Опис
FR-9	Перевірка обраного символу у вікні дослідника рішень Перевірка, на який символ вказує каретка, щоб вирішити, чи додавати до контекстного меню команди генерації тестів.
FR-10	Генерація юніт-тестів розширено з вікна дослідника рішень Можливість розширеної генерації тестів із заданням власних параметрів, обравши команду “Generate Unit Tests Advanced...” у вікні дослідника рішень.
FR-11	Перегляд сторінки розширеної генерації тестів Можливість задання тестового проєкту, назви файлу з тестами, назви класу з тестами, простору імен класу, а також конкретних тестів, які треба згенерувати.
FR-12	Перевірка валідності назви файлу з тестами Назва файлу з тестами має бути не порожньою та відповідати правилам іменування файлів у Windows.
FR-13	Перевірка валідності назви тестового класу Назва тестового класу має відповідати правилам іменування ідентифікаторів у .NET.
FR-14	Перевірка валідності назви простору імен тестового класу Назва тестового класу має відповідати правилам іменування просторів імен у .NET.
FR-15	Обрання окремих юніт-тестів для генерації Можливість обрання конкретних юніт-тестів, що будуть згенеровані, зі списку тестів.
FR-16	Генерація юніт-тестів розширено з вікна редактора коду Можливість розширеної генерації тестів із заданням власних параметрів, обравши команду “Generate Unit Tests Advanced...” у вікні редактора коду.

Продовження таблиці 2.11

Назва	Опис
FR-17	Зміна налаштувань розширення Можливість задання налаштувань, що будуть використані за замовченням, у меню Tools -> Options середовища Visual Studio
FR-18	Задання шаблону назви тестового файлу Налаштування шаблону, що використовуватиметься як назва тестового файлу за замовченням. Шаблон може містити спеціальні назви, що будуть підстановлені в залежності від контексту, наприклад, {TypeName}Tests буде замінено на MyClassTests.
FR-19	Задання шаблону назви класу з тестами Налаштування шаблону, що використовуватиметься як назва класу з тестами за замовченням. Шаблон може містити спеціальні назви, що будуть підстановлені в залежності від контексту, наприклад, {TypeName}Tests буде замінено на MyClassTests.
FR-20	Задання шаблону назви простору імен класу з тестами Налаштування шаблону, що використовуватиметься як назва простору імен класу з тестами за замовченням. Шаблон може містити спеціальні назви, що будуть підстановлені в залежності від контексту, наприклад, {TypeNamespace}Tests буде замінено на MyClassNamespaceTests.
FR-21	Задання необхідності тестування internal типів Можливість вказати, чи треба тестувати internal типи / члени типів.
FR-22	Задання необхідності генерації AAA-коментарів у згенерованих тестах Можливість вказати, чи треба додавати AAA-коментарі (Arrange - Act - Assert) до згенерованих тестів.

Продовження таблиці 2.11

Назва	Опис
FR-23	Задання необхідності підстановки аргументів типів для універсальних типів / членів типів Можливість використання функції “Smart Generics” при генерації тестів. Ця функція дозволяє знаходити та підставляти аргументи типів для універсальних символів, таким чином формуючи закритий універсальний тип і уникаючи помилок компіляції при генерації тестів.
FR-24	Задання необхідності попередження про генерацію кількох типів Можливість вказати, чи варто показувати попередження про генерацію кількох файлів з тестами, якщо обрані користувачем елементи містять багато типів, для яких можуть бути згенеровані тести.

	FR-1	FR-2	FR-3	FR-4	FR-5	FR-6	FR-7	FR-8	FR-9	FR-10	FR-11	FR-12	FR-13	FR-14	FR-15	FR-16	FR-17	FR-18	FR-19	FR-20	FR-21	FR-22	FR-23	FR-24
UC-1	+																							
UC-2		+	+																					
UC-3				+	+	+	+																	
UC-4						+	+	+	+															
UC-5					+	+	+			+	+	+	+	+	+									
UC-6						+	+		+		+	+	+	+	+	+								
UC-7											+													
UC-8							+																	
UC-9																	+	+	+	+	+	+	+	+

Рисунок 2.1 – Матриця трасування вимог

2.4 Розроблення нефункціональних вимог

Розширення має генерувати максимальну кількість шаблонного коду, лишаючи користувачеві дописати лише кілька рядків коду;

Розширення має бути спрямоване на генерацію коду, що завжди компілюється.

Висновки до розділу

Метою другого розділу було формування вимог до програмного забезпечення. Розділ почався зі створення діаграми варіантів використання UML, після чого зазначені варіанти використання були також описані більш детально у формі таблиць. Усього розроблюваний проєкт містить 9 варіантів використання.

Після цього були описані системні вимоги. Зважаючи на специфіку розроблюваного ПЗ (розширення для IDE), ці вимоги впливають із вимог для середовища Visual Studio 2022.

Також у розділі було сформовано загальну модель функціональних вимог у вигляді таблиці. Після цього кожна функціональна вимога була детально описана в таблиці 2.11. Усього проєкт містить 24 функціональні вимоги. Була побудована матриця трасування, що співставила варіанти використання з функціональними вимогами.

Нарешті, були описані нефункціональні вимоги.

За результатами розділу сформовано технічне завдання на розробку програмного забезпечення.

3 КОНСТРУЮВАННЯ ТА РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Архітектура програмного забезпечення

Архітектурно розширення поділене на 6 проєктів (пакетів), кожен з яких має окрему роль. Діаграма пакетів UML для розширення зображена на рисунку 3.1.

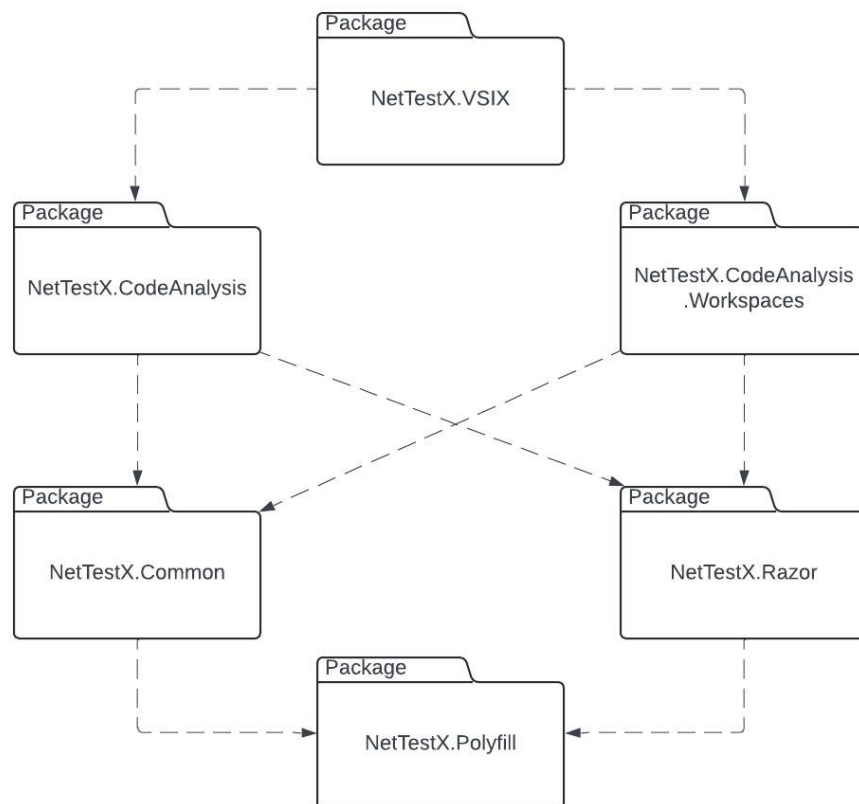


Рисунок 3.1 – Діаграма пакетів UML

Два проєкти з шести (NetTestX.Polyfill та NetTestX.Common) мають виключно допоміжну роль і не містять окремих компонентів, водночас інші чотири проєкти містять основний функціонал і виділяють набір компонентів. Діаграма компонентів для розширення наведена у графічному матеріалі, креслення 2.

Розглянемо архітектуру та призначення кожного проєкту більш детально.

NetTestX.Polyfill – невеликий проєкт, що містить необхідну інфраструктуру для підтримки розробки з використанням останньої версії C#. У зв'язку з тим, що Visual Studio використовує .NET Framework, усі бібліотеки, що складають розширення, мають бути з ним сумісні. Це означає, що проєкти не можуть використовувати .NET останніх версій (наприклад, .NET 8.0), а мають використовувати .NET Standard 2.0. Офіційно, версія мови C# 12 підтримується на .NET 8.0, при цьому використання її з .NET Standard 2.0 можливе, але деякі можливості нових версій мови можуть бути недоступні. Деякі з них недоступні через зміни до CLR (середовища виконання), а деякі – через відсутність необхідних для компілятора типів у BCL. У той час, як можливості першого типу (наприклад, реалізація членів інтерфейсів за замовченням [17]) неможливо підтримати на більш ранніх версіях .NET, можливості другого типу (наприклад, primary constructors [17]) можна підтримати. Проєкт NetTestX.Polyfill досягає саме цього – додає необхідні для компілятора типи до компіляції для підтримки нових версій C#, наприклад, ключові слова `init`, `required`, використання діапазонів (`range`), `primary constructors` [17]. Усі інші проєкти посилаються на цей проєкт для підтримки останньої версії C# (це посилання робиться автоматично через MSBuild).

NetTestX.Common – невеликий проєкт, що містить специфічні для домену типи (список бібліотек моків, тестових фреймворків) та надає API для повідомлення діагностик (використовується у вікні розширеної генерації тестів для попередження користувача, наприклад, про відсутність у типу конструктора, а отже неможливість генерації тестів для не статичних членів). Розширення підтримує тестові фреймворки xUnit, NUnit, MSTest і тестові бібліотеки NSubstitute, FakeItEasy, Moq. Загалом на цей проєкт посилаються всі інші проєкти, окрім NetTestX.Polyfill і NetTest.Razor (бо вони не залежать від домену).

NetTestX.Razor – проєкт, що містить необхідну інфраструктуру для підтримки Razor-шаблонів. Надає користувачам інтерфейс (компонент “Razor Engine”) для створення Razor-сторінок (файли `.cshtml`) та їх матеріалізації

(підстановки значень у шаблон і отримання результату). Використовується проєктами `NetTestX.CodeAnalysis` і `NetTestX.CodeAnalysis.Workspaces` для генерації файлів з юніт-тестами і файлів проєкту `.csproj`. Діаграма класів UML для проєкту наведена на рисунку 3.2.

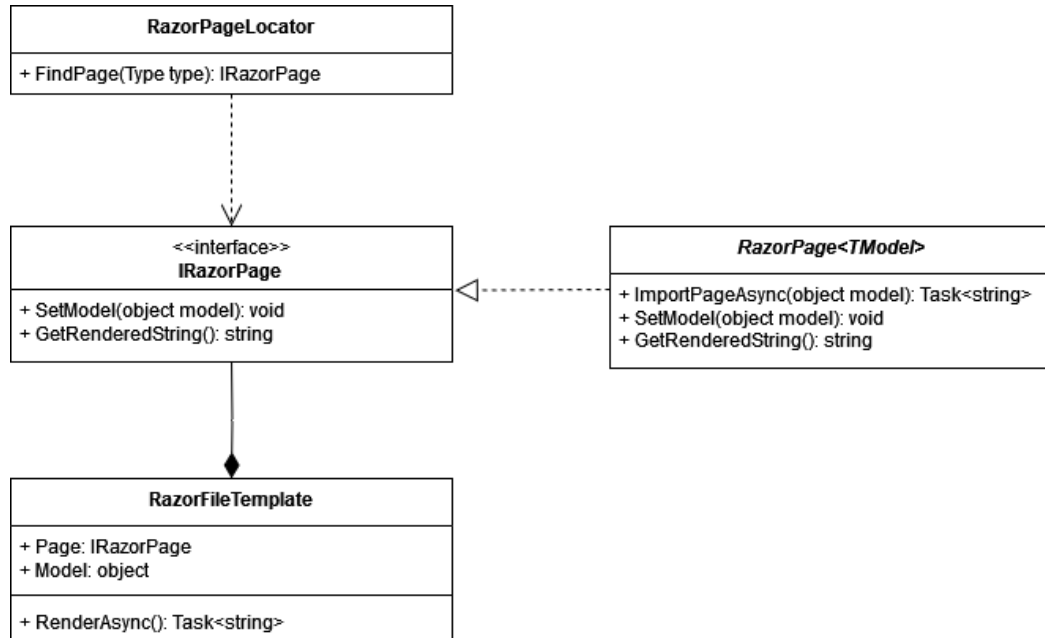


Рисунок 3.2 – Діаграма класів UML для проєкту `NetTestX.Razor`

`NetTestX.CodeAnalysis` – проєкт, що містить компоненти для синтаксичного та семантичного аналізу коду і генерації юніт-тестів, і надає інтерфейс (компонент “Test Generator Driver”) для генерації тестів для заданого символу. Компонент “Test Framework Model” надає інтерфейс, що містить специфічні для обраного тестового фреймворку анотації тестового класу / юніт-тесту для їх розпізнавання, і використовується в шаблонах `Razor` для генерації тестів. Компонент “Mock Value Provider” виконує аналогічну роль для бібліотек моків, визначаючи, як і для яких типів обрана бібліотека може створити моки. Використовується в шаблонах `Razor`. Компонент “Test Method Collection” використовується для аналізу обраного користувачем типу та пошуку всіх юніт-тестів, що можуть бути створені для цього типу. Це включає в себе не тільки тестування публічних методів типу, а й, наприклад, тестування методів `Dispose()` / `DisposeAsync()`, якщо тип реалізує інтерфейс `IDisposable` / `IAsyncDisposable`. Компонент “Smart Generics” містить підтримку функціональності з відповідною назвою. Він містить алгоритми пошуку і

підстановки аргументів типу для заданого універсального типу, якщо користувач обрав опцію “Use Smart Generics” у налаштуваннях розширення. Ці алгоритми будуть детальніше описані в п. 3.3. Нарешті, компонент “Test Generator Driver” є головним публічним інтерфейсом пакету, що отримує заданий символ (тип) для генерації тестів і параметри генерації, і генерує файл із тестами для цього символу в пам’яті. Проєкт NetTestX.CodeAnalysis використовується “основним” проєктом розширення NetTestX.VSIX. Діаграма класів UML для проєкту наведена на рисунку 3.3.

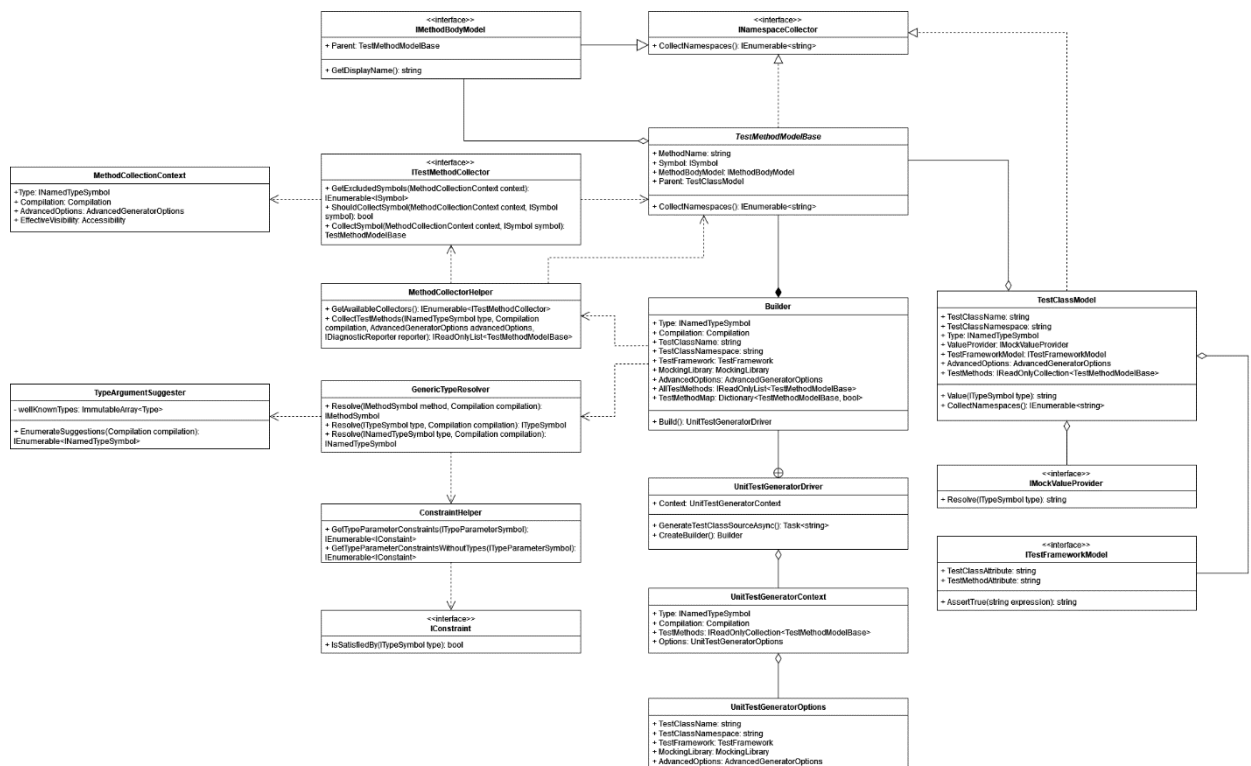


Рисунок 3.3 – Діаграма класів UML для проєкту NetTestX.CodeAnalysis

NetTestX.CodeAnalysis.Workspaces – проєкт, що містить компоненти для створення та маніпуляції проєктами та рішеннями .NET. Він використовується “основним” проєктом розширення NetTestX.VSIX для генерації тестових проєктів і модифікації файлів *.csproj. Компоненти “Testing Framework Detection” і “Mocking Library Detection” використовуються при генерації файлів з юніт-тестами для того, щоб визначити, який тестовий фреймворк і бібліотеку моків використовує обраний тестовий проєкт. Компоненти “Testing Framework Model” і “Mocking Library Model” використовуються при генерації тестового проєкту для того, щоб визначити, на які пакети додати посилання у

файлі *.csproj (або якісь інші властивості проєкту, специфічні для обраного тестового фреймворку / бібліотеки моків). Загалом цей проєкт, як і проєкт NetTestX.CodeAnalysis, широко взаємодіє із бібліотекою Roslyn, а також використовує бібліотеку Microsoft.Build, що надає доступ до API MSBuild. Діаграма класів UML для проєкту наведена на рисунку 3.4.

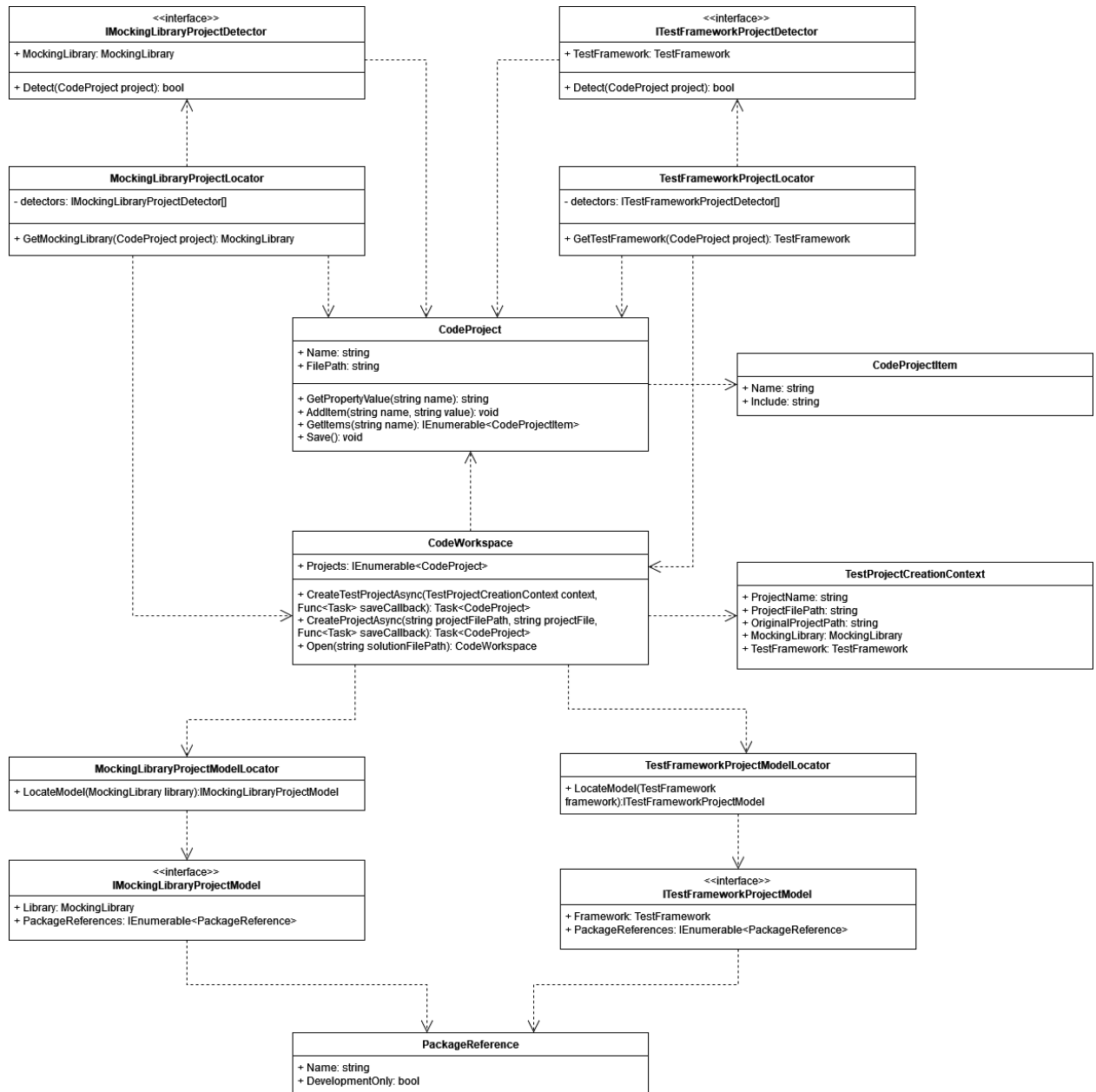


Рисунок 3.4 – Діаграма класів UML для проєкту
NetTestX.CodeAnalysis.Workspaces

NetTestX.VSIX є проєктом розширення Visual Studio. Він визначає компоненти, що взаємодіють із API Visual Studio, зокрема DTE, файл *.vsct (Visual Studio Command Table), що містить конфігурацію команд і елементів

меню, і налаштування (опції) розширення, екранні форми WPF. Саме цей проєкт у результаті генерує файл `NetTestX.vsix`, що є файлом розширення Visual Studio та може бути встановлений у середовище розробки як плагін або поширений на порталі Visual Studio Marketplace. Компоненти “VSIX Commands” і “VSIX Options” визначають відповідно логіку команд і список налаштувань розширення (доступних у вікні Tools -> Options). Команди використовують компоненти “Test Source Generation” і “Test Project Generation” для генерації відповідно тестових файлів і тестових проєктів, при цьому використовуючи API `NetTestX.CodeAnalysis` і `NetTestX.CodeAnalysis.Workspaces`. Опції використовують компонент “Variable Provider” для вирішення кінцевого значення, оскільки опції можуть застосовувати шаблони. Наприклад, значення опції “{TestName}Tests” буде замінено на “MyClassTests”. Компонент “UI” містить екранні форми WPF та застосовує шаблон MVVM, і використовує компонент “Validation” для перевірки введених значень і відповідної зміни елементів інтерфейсу. Діаграма класів UML для проєкту наведена на рисунку 3.5.

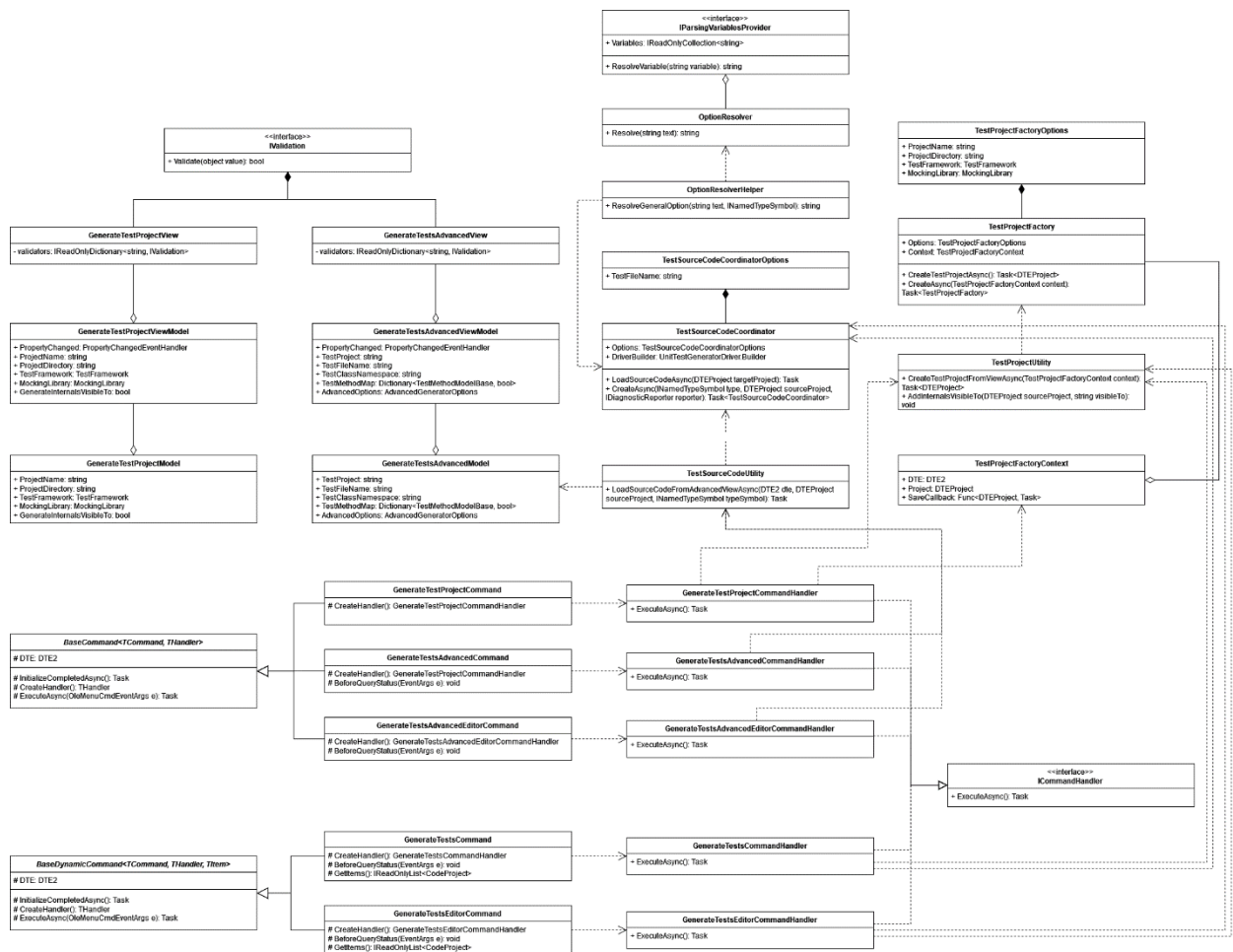


Рисунок 3.5 – Діаграма класів UML для проєкту NetTestX.VSIX

3.2 Обґрунтування вибору засобів розробки

Як було зазначено в п. 3.1, розширення для Visual Studio мають використовувати .NET Framework. Ця програмна платформа є реалізацією CLI, отже, може використовувати будь-яку мову, сумісну із CIL [16]. Канонічними мовами для цього виступають C#, F#, VB.NET і C++. Мова F# є відмінним варіантом для функціонального програмування в .NET, але для розробки розширень Visual Studio її складно налаштувати і, враховуючи, що API Visual Studio є об'єктно-орієнтованим, інші мови підійдуть для цієї задачі краще. Мова C++, аналогічно до F#, не підтримується Microsoft для розробки розширень “з коробки” і потребує додаткових налаштувань. При цьому її не можна використовувати у шаблонах Razor. Мови VB.NET і C# є прямими конкурентами, що можуть бути використані як для розробки розширень Visual Studio, так і для написання шаблонів Razor, проте C# є значно поширенішою

та більш підтримуваною мовою, і має багато прикладів написання розширень [13]. Також, якщо розробити розширення з відкритим кодом саме на C#, воно приверне увагу більшої кількості контриб'юторів. Враховуючи всі перелічені вище факти для розробки розширення було обрано мову C#.

У якості середовища розробки було обрано Visual Studio як найбільш очевидний варіант, враховуючи специфіку розроблюваного програмного забезпечення. Теоретично розробку можна було б налаштувати і в Rider, проте це значно важче, і при відлагодженні розширення все одно використовуватиметься Visual Studio.

Загалом питання розробки розширення для автоматичної генерації юніт-тестів складається з трьох частин: обрання SDK, що використовуватиметься для розробки плагіну, обрання способу аналізу коду для генерації тестів, і обрання способу генерації коду.

Visual Studio має два SDK, що можна використати для розробки розширень: VSSDK та VisualStudio.Extensibility SDK (VSExtensibility). VSSDK – це більш ранній набір засобів розробки, що наразі є де-факто стандартом і є найбільш поширеним для розробки розширень [13]. Він підтримує всі можливі варіанти використання, в тому числі взаємодію з текстовим редактором, маніпуляцію рішеннями і проєктами, створення команд і меню, створення додаткових опцій тощо. Проте VSSDK має багато недоліків. Основними з них є дуже невелика кількість документації, або взагалі її відсутність, складність API та спосіб завантаження розширень: вони виконуються в одному процесі із Visual Studio [13]. Останній пункт означає, що IDE має бути перезавантажена для встановлення розширень, а також те, що вони можуть призвести до критичних помилок і “вбити” Visual Studio. Необхідність виконання в одному процесі також значить, що розробка має вестись на старих версіях .NET Framework для сумісності з Visual Studio.

Враховуючи всі вищезазначені проблеми було вирішено з нуля розробити новий SDK, що має назву VisualStudio.Extensibility і покликаний покращити UX як для розробників розширень, так і для їх користувачів. В

першу чергу це досягається тим, що плагіни, написані з використанням VSExtensibility, виконуються в окремому процесі. Це дозволяє не перезавантажувати середовище для їх встановлення і використовувати останні версії .NET для розробки. Також новий SDK має набагато кращі API та документацію, що дозволяє значно пришвидшити розробку. Основною проблемою є те, що він все ще перебуває в розробці і не має всіх можливостей, що має VSSDK. Багато з них будуть в подальшому використані в розробці розширення для автоматичної генерації юніт-тестів. Отже, в якості SDK для розробки було обрано VSSDK. Також у якості фасаду над API Visual Studio використовується популярна бібліотека з відкритим кодом Community.VisualStudio.Toolkit [13].

Аналіз коду для генерації юніт-тестів є задачею метапрограмування і потребує отримання інформації про синтаксичну і семантичну структуру коду. Платформа .NET надає можливості використання API компілятора для проведення такого аналізу. .NET Compiler Platform, або Roslyn, є проектом з відкритим кодом від Microsoft, що містить компілятори та засоби статичного аналізу коду для мов C# і VB [18] [19] [20]. Наразі Roslyn є найбільш поширеним компілятором для цих мов, його використовує як .NET CLI, так і IDE такі як Visual Studio і JetBrains Rider [18] [19]. Ще однією перевагою Roslyn є те, що VSSDK надає вбудований спосіб отримати доступ до його API. Проте існують й інші реалізації компілятора для C#. Другим за популярністю, поширеним компілятором із відкритим кодом є mcs від Mono Project [21]. Цей проєкт є повністю незалежним від Microsoft. До API компілятора можна отримати доступ з бібліотеки Mono.CSharp. Проте він має кілька суттєвих недоліків. По-перше, mcs підтримує лише версії мови до C# 6 і частково C# 7 [21]. По-друге, mcs не містить прямої підтримки проєктів (*.csproj) / рішень, для цього використовується окрема утиліта XBuild, що ускладнює роботу. По-третє, Visual Studio не містить вбудованої підтримки компілятора Mono, що означає, що інтеграція його з розширенням буде складною. Також це може призвести до більш повільної роботи розширення, оскільки проєкт треба буде

компілювати двічі: один раз середовищу розробки, а другий раз – розширенню. Загалом, після зваження варіантів, для аналізу коду та подальшої його генерації було обрано компілятор Roslyn.

Власне генерація коду може банально виконуватись із застосуванням типу `string` чи класу `StringBuilder`, проте такий підхід є дуже незручним з кількох причин. По-перше, вся індентація та відступи мають бути задані вручну і легко можуть бути задані неправильно. По-друге, відсутня можливість вкладення шаблонів один в одного, наприклад, вкладення списку згенерованих тестових методів у тестовий клас. По-третє, змішування програмного коду та коду для генерації шаблонів значно ускладнює їх розуміння. Було б добре мати окремі файли, що містять у собі шаблони для генерації, та взаємодіяти з ними через код в об'єктно-орієнтованій парадигмі.

Для досягнення цих цілей зазвичай використовуються шаблонізатори. Шаблонізатор поєднує шаблони з моделлю даних для генерації вихідного тексту, яким може бути, наприклад, HTML-розмітка, email-лист, код тощо. Існує багато шаблонізаторів для платформи .NET, як-от `Scriban` [22], `Handlebars` [23] тощо, та з-поміж них виділяється `Razor`. `Razor` – це статичний шаблонізатор з відкритим кодом, розроблений Microsoft для платформи .NET [24]. Основними його перевагами є можливість писати C# код прямо в шаблоні, підтримка змінних, циклів, умов, функцій, вкладення, а також компіляція шаблону в момент компіляції, що значно пришвидшує подальшу генерацію коду. Синтаксис `Razor` також підтримується більшістю IDE за рахунок його популярності. Незважаючи на те, що `Razor` був розроблений здебільшого для генерації HTML-розмітки і використання в межах веб-фреймворку ASP.NET, він може бути застосований для генерації будь-яких шаблонів, зокрема для генерації коду [24]. Факт того, що компіляція шаблонів стається в момент компіляції коду, також робить `Razor` безпечним і допомагає виявляти помилки на ранній стадії, наприклад, моделі представлені у вигляді класів, і використання неіснуючих полів чи методів у шаблоні призведе до помилки компіляції. Проте він містить і кілька недоліків. Основним з них

варто виділити важкість налаштування Razor для проєктів, що не використовують ASP.NET, оскільки типи з цього фреймворка використовуються в деяких внутрішніх деталях шаблонізатора. Загалом, Razor є відмінним варіантом шаблонізатора для .NET, і він використовуватиметься для генерації коду в розширенні.

3.3 Конструювання програмного забезпечення

Почнемо з опису програмних структур і більш детального опису архітектури компонентів.

3.3.1 Опис проєкту NetTestX.Razor

Як було зазначено в п. 3.1, проєкт “NetTestX.Razor” застосовується для генерації шаблонів Razor. Він потрібен у системі через те, що вбудовані можливості .NET для генерації шаблонів Razor (пакет Microsoft.AspNetCore.Mvc) призначені конкретно для фреймворку ASP.NET і сильно зав’язані на типах з його бібліотек (наприклад, ViewContext, IActionResult тощо). У свою чергу NetTestX.Razor надає власні типи IRazorPage і RazorPage<TModel>, що виконують роль адапторів для типів з ASP.NET, таким чином надаючи користувачам простий інтерфейс, що є універсальним, а також генерує результат прямо в string. Тип RazorPage<TModel> також потребує тип моделі для заданого шаблону, тобто кожен шаблон *.cshtml співвідноситься з класом-моделлю, що надає необхідні дані для генерації контенту. Щоб зрозуміти, як досягається це співвідношення, треба зрозуміти як працюють шаблони Razor “під капотом”: із застосуванням генератора коду (Source Generator) вони транслюються у звичайні класи C# і додаються до компіляції [24]. Таким чином, кожен згенерований клас (шаблон) просто наслідується від класу RazorPage<TModel>. У синтаксисі Razor це робиться з використанням директиви @inherits, наприклад, @inherits RazorPage<MyModel>.

Останнім питанням для компоненту “Razor Engine” є API для власне генерації коду. Компонент “Razor Engine” має містити механізм, що знайде шаблон Razor для заданої моделі та згенерує з нього результат. Це досягається з використанням класу RazorPageLocator, що використовує клас AppDomain для підписки на завантаження збірок у домен та пошуку в них типів, що наслідуються від RazorPage<TModel>. Таким чином встановлюється залежність між типом моделі та типом шаблону, і необхідний шаблон можна знайти просто викликавши метод RazorPageLocator.FindPage(modelType).

3.3.2 Опис проєкту NetTestX.CodeAnalysis

Проєкт має набір компонентів, кожен з яких варто розглянути як окрему підсистему.

Основний компонент “Test Generator Driver” представлений класом UnitTestGeneratorDriver і суміжною інфраструктурою. Для створення класу використовується окремий Builder клас, що дозволяє задати чи змінити значення аргументів, що потім “матеріалізуються” в UnitTestGeneratorDriver. Це корисно, наприклад, для можливості обрання конкретних тестів, що будуть згенеровані: Builder містить перелік цих тестів, що можна змінити (наприклад, з UI), після чого при створенні драйверу цей список змінити неможливо. Загалом, компонент “Test Generator Driver” отримує контекст для генерації тестів, що містить усі налаштування драйвера, і повертає контент згенерованого тестового файлу. В подальшому цей контент можна не тільки напряму записувати у файл (це робить компонент “Test Source Code Generation” у NetTestX.VSIX), а й, наприклад, показувати користувачу перед створенням файлу для попереднього перегляду того, що буде згенеровано.

Компонент “Test Method Collection” є важливою складовою пакету і знаходить усі можливі юніт-тести для заданого типу. В подальшому вони зберігаються у драйвері та генеруються у тестовий клас. Процес “знаходження” тестів досягається інтерфейсом ITestMethodCollector та набором його реалізацій. За допомогою API Roslyn з обраного користувачем

типу для генерації тестів отримується набір символів (`ISymbol` - семантичних одиниць), після чого кожен колектор фільтує символи, обираючи ті, що можна використати для генерації конкретного типу тестів. Наприклад, колектор `AccessibleInstanceMethodCollector` буде знаходити символи `IMethodSymbol`, що є публічними, не статичними методами типу, і створювати екземпляр моделі `AccessibleInstanceMethodBodyModel` для кожного такого символу для подальшої генерації тесту за допомогою відповідного шаблону `Razor`. Розділення логіки пошуку тестів на різні реалізації інтерфейсу покращує якість коду, полегшує додання нових стратегій пошуку, а також дозволяє в майбутньому обирати лише окремі стратегії: наприклад, якщо користувач хоче виключно тестувати статичні методи, то для пошуку тестів буде використовуватись лише колектор `AccessibleStaticMethodCollector`.

Компонент “Smart Generics” містить реалізацію алгоритму, що застосовується для підстановки аргументів типу для універсальних типів. Сам алгоритм буде детально описаний у п. 3.3.5. Алгоритм має враховувати усі можливі обмеження параметрів типів, що визначені мовою C# - обмеження на базовий клас, інтерфейс, `class`, `struct`, `unmanaged`, `new` [17]. Для задання цих обмежень було використано патерн `Specification`: кожне обмеження є реалізацією інтерфейсу `IConstraint` з методом `bool IsSatisfiedBy(ISypeSymbol)`.

Також проєкт містить шаблони `Razor` для власне генерації файлів з тестами, що використовують інтерфейси `ITestFrameworkModel` і `IMockValueProvider` для задання специфічних для обраного тестового фреймворку атрибутів та інших елементів і створення моків для обраної бібліотеки моків, відповідно.

3.3.3 Опис проєкту `NetTestX.CodeAnalysis.Workspaces`

Проєкт `NetTestX.CodeAnalysis.Workspaces` використовується для перегляду проєктів і рішень (наборів проєктів), а також створення і маніпуляції проєктами. Компонент “Workspace Manipulation” є основним у проєкті – він визначає типи `CodeWorkspace`, `CodeProject` і необхідну

інфраструктуру. CodeWorkspace дозволяє переглядати список проєктів у рішенні, додавати новий проєкт до рішення. CodeProject дозволяє переглядати вміст файлу *.csproj (елементи Property та Item з MSBuild). Загалом, компонент “Workspace Manipulation” є фасадом над API Roslyn і MSBuild і полегшує взаємодію з проєктною інфраструктурою.

Компоненти “Testing Framework Detection” і “Mocking Library Detection” є допоміжними компонентами для визначення, який тестовий фреймворк і бібліотеку моків використовує заданий проєкт, відповідно. Це необхідно для коректної генерації тестів у обраному проєкті: логічно, що якщо користувач обрав, щоб тести було згенеровано в проєкт X, що використовує, наприклад, xUnit і FakeItEasy, то згенеровані тести мають використовувати xUnit і FakeItEasy.

Також проєкт містить шаблони Razor для генерації файлу проєкту *.csproj. Шаблони використовують інтерфейси ITestFrameworkProjectModel і IMockingLibraryProjectModel для задання посилань на пакети обраного тестового фреймворку і бібліотеки моків, відповідно.

3.3.4 Опис проєкту NetTestX.VSIX

Проєкт NetTestX.VSIX є основним проєктом і безпосередньо завантажується середовищем Visual Studio як розширення. Через це, на відміну від усіх інших проєктів, що використовують .NET Standard 2.0, цей проєкт використовує .NET Framework 4.8. Проєкт також використовує пакет Community.VisualStudio.Toolkit – проєкт із відкритим кодом, що значно спрощує розробку розширень.

Головним компонентом проєкту є компонент “VSIX Commands”, що визначає команди для розширення. Команда - концепція Visual Studio, що в загальному випадку визначає певну дію, що виконується при активації команди. Більша частина функціоналу розширення (кнопки у контекстних меню для генерації юніт-тестів і тестових проєктів) визначена командами. Компонент містить класи BaseCommand<TCommand, THandler> і

`BaseDynamicCommand<TCommand, THandler>`, що є базовими для всіх визначених команд, а також інтерфейс `ICommandHandler`, реалізації якого містять логіку відповідних команд. Таке розділення на власне команду й “обробник” команди сприяє розділенню відповідальності: сама команда визначає, наприклад, умови, за яких вона ввімкнена / вимкнута, присутня / відсутня в меню тощо, а обробник – безпосередньо логіку, що відбувається при активації команди. Також це дозволяє в перспективі створювати кілька різних команд з точки зору середовища розробки (наприклад, з різним місцезнаходженням), але однакових з точки зору функціональності (вони оброблюються однією реалізацією `ICommandHandler`).

Компоненти “Test Project Generation” і “Test Source Code Generation” використовуються обробниками команд для генерації тестів і файлів з тестами, відповідно. Для отримання інформації від середовища вони використовують DTE – об’єктний інтерфейс, що надається Visual Studio, після чого передають цю інформацію компонентам з `NetTestX.CodeAnalysis` і `NetTestX.CodeAnalysis.Workspaces`. Також за необхідності вони показують екранні форми чи отримують значення налаштувань.

Компонент “UI” визначає екранні форми WPF, моделі, а також іншу суміжну логіку. Екранні форми відповідають загальному оформленню Visual Studio, що значно покращує UX розширення. Цей компонент реалізований за допомогою патерну MVVM і використовує компонент “Validation” для валідації значень, які задає користувач. Якщо якесь значення не проходить валідацію, то кнопка “Continue” стає сірватою та її неможливо натиснути. Валідації реалізують інтерфейс `IValidation` із єдиним методом `bool Validate(object value)`.

Компонент “VSIX Options” визначає список налаштувань розширення. Для цього створюються типи, що наслідуються від класів `BaseOptionPage<T>` і `BaseOptionModel<T>` з пакету `Community.VisualStudio.Toolkit`. Список налаштувань (властивостей) цих типів автоматично додається до меню `Tools->Options` у Visual Studio. Компонент “Variable Provider” замінює шаблонні

змінні в опції на безпосередньо їх значення (що залежать від контексту), в результаті повертаючи кінцеве значення опції. Для цього використовується клас `OptionsResolver`, що приймає інтерфейс `IParsingVariablesProvider` і повертає підстановлений текст. Реалізація інтерфейсу `IParsingVariablesProvider`, у свою чергу, визначає список можливих змінних шаблону та логіку їх вирішення. Таким чином можливо задавати різні реалізації інтерфейсу в залежності від, наприклад, конкретної опції значення якої обчислюється.

3.3.5 Опис алгоритму підстановки аргументів типу

Функція “Smart Generics” для підстановки аргументів типу в універсальних символах використовує оригінальний алгоритм, оскільки відомих реалізацій подібної функціональності не було знайдено.

Загалом алгоритм побудований на принципі бектрекінгу, проходячи по параметрах типу та намагаючись знайти типи, що задовільняють їх обмеження. Опишемо кілька важливих концепцій, що використовуються в алгоритмі.

Відкритий тип – це тип, що містить параметри типу. Це або власне параметр типу, або універсальний тип, у якого прийнятні один з аргументів типу є відкритим типом [17]. Закритий тип – це тип, що не є відкритим [17].

Набір параметрів типу будь-якого універсального типу можна представити у вигляді графу, де вершини – параметри типу, а ребра – залежності між ними, що виражені обмеженнями на параметр типу. Наприклад, для універсального типу на рисунку 3.6 можна побудувати граф залежностей, як на рисунку 3.7.

```
class C<T1, T2, T3, T4, T5>
{
    where T1 : IEnumerable<T2>
    where T2 : IEquatable<T3>
    where T4 : class, IDictionary<T1, T2>
    where T5 : IEquatable<T3>;
}
```

Рисунок 3.6 – Приклад універсального типу із обмеженнями

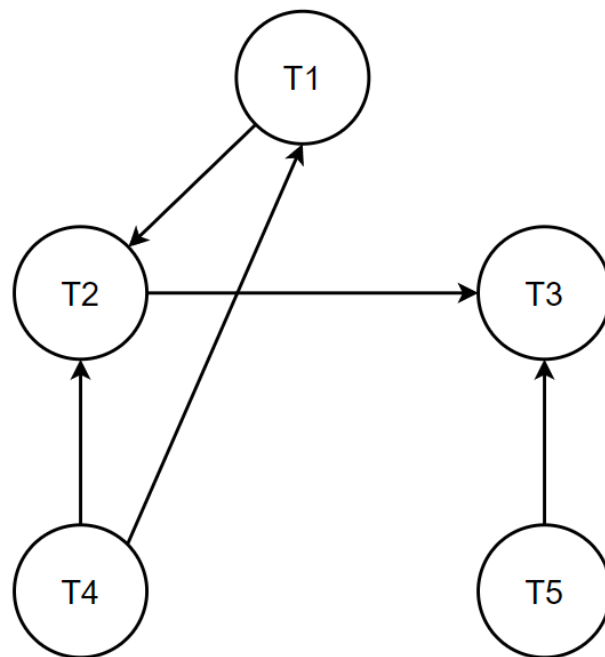


Рисунок 3.7 – Граф залежностей параметрів типу

Зрозуміло, що при побудові такого графу враховуються лише обмеження на відкритий тип. Обмеження на закритий тип чи інші обмеження не враховуються, бо не впливають на залежності між параметрами типу.

Варто зазначити, що алгоритм не підтримує циклічні залежності між параметрами типу, наприклад такі як на рисунку 3.8, оскільки використовує топологічне сортування.

```

class C<T1, T2>
    where T1 : IEnumerable<T2>
    where T2 : IEnumerable<T1>;

```

Рисунок 3.8 – Приклад циклічних залежностей між параметрами типу

Загальні кроки алгоритму наступні:

- Крок 1. Сформувати таблицю підібраних аргументів типу (спочатку пуста)
- Крок 2. Топологічно відсортувати список параметрів типу, базуючись на графі залежностей
- Крок 3. Розв'язати кожен параметр типу окремо, йдучи списком. Якщо параметр типу вже міститься у списку відвіданих, то пропустити його

Крок 4. Дозаповнити таблицю підібраних аргументів типу для параметрів типу, що в ній відсутні, типом `System.Void`

Алгоритм розв’язання параметра типу наступний:

Крок 1. Перебрати послідовність кандидатів аргумента типу. Для кожного типу-кандидата виконати крок 2

Крок 2. Перебрати список типів-обмежень на поточний параметр типу. Для кожного типу-обмеження виконати крок 3

Крок 3. Якщо тип-обмеження неможливо вирішити, обрати наступний тип-кандидат і повернутись до кроку 2

Крок 4. Якщо обране рішення типу-обмеження запроваджує такі обмеження на параметри типу, що їх не задовільняє тип-кандидат, обрати наступний тип-кандидат і повернутись до кроку 2

Крок 5. Якщо обраний тип-кандидат не є універсальним типом – закінчити розв’язання параметра типу

Крок 6. Рекурсивно викликати загальний алгоритм для типу-кандидата

Послідовність кандидатів аргумента типу наступна:

- тип `System.Object`;
- тип `System.Int32`;
- тип `System.String`;
- тип `System.Char`;
- тип `List<T>`;
- тип `HashSet<T>`;
- тип `Dictionary<TKey, TValue>`;
- усі типи зі збірки, в якій міститься тип, для якого генеруються тести;
- усі інші типи, що доступні зі збірки.

Алгоритм вирішення типу-обмеження наступний:

Крок 1. Знайти список реалізацій типу-обмеження типом-кандидатом. Для кожної реалізації виконати крок 2

Крок 2. Перебрати список аргументів типу реалізації. Для кожного аргумента типу виконати крок 3

Крок 3. Якщо поточний аргумент типу є параметром типу – оновити обмеження на поточний параметр типу відповідно до вже вирішених параметрів типу

Крок 4. Рекурсивно викликати алгоритм розв’язання параметра типу для поточного параметра типу

У результаті роботи алгоритму параметри типу заміщуються аргументами типу. Якщо якийсь параметр типу замістити неможливо, якщо, наприклад, якщо типу, що задовільняє обмеження параметра типу, не знайдено, то він заміщується типом `System.Void`.

Для прикладу роботи алгоритму оберемо тип, що показаний на рисунку 3.9. Якщо функція “Smart Generics” у налаштуваннях розширення вимкнена, то згенерований файл з тестами матиме вигляд як на рисунку 3.10. Якщо функція ввімкнена, то файл матиме вигляд як на рисунку 3.11.

```
public class C<T1, T2, T3, T4, T5>
{
    where T1 : struct, IEnumerable<T2>
    where T2 : IDictionary<T3, T5>
    where T3 : struct, ISpanFormattable
    where T4 : IEquatable<T5>
    where T5 : unmanaged

    public void M1<T, K, R>()
    {
        where T : ISet<R>
    }

    public void M2<T, K, R>()
    {
        where T : IEnumerable<K>, IStructuralEquatable
        where K : IEqualityComparer<R>
    }
}
```

Рисунок 3.9 – Приклад універсального типу з універсальними методами

```

public class CTests
{
    [Fact]
    public void TestM1()
    {
        // Arrange
        C<T1, T2, T3, T4, T5> sut = new();

        // Act
        sut.M1<T, K, R>();

        // Assert
    }

    [Fact]
    public void TestM2()
    {
        // Arrange
        C<T1, T2, T3, T4, T5> sut = new();

        // Act
        sut.M2<T, K, R>();

        // Assert
    }
}

```

Рисунок 3.10 – Згенеровані тести для типу без функції “Smart Generics”

```

public class CTests
{
    [Fact]
    public void TestM1()
    {
        // Arrange
        C<ImmutableArray<Dictionary<int, int>>, Dictionary<int, int>, int, int, int> sut = new();

        // Act
        sut.M1<HashSet<object>, object, object>();

        // Assert
    }

    [Fact]
    public void TestM2()
    {
        // Arrange
        C<ImmutableArray<Dictionary<int, int>>, Dictionary<int, int>, int, int, int> sut = new();

        // Act
        sut.M2<ImmutableArray<EqualityComparer<object>>, EqualityComparer<object>, object>();

        // Assert
    }
}

```

Рисунок 3.11 – Згенеровані тести для типу з функцією “Smart Generics”

3.3.6 Опис утиліт, бібліотек та іншого стороннього програмного забезпечення

Опис утиліт, бібліотек та іншого стороннього програмного забезпечення, що використовується у розробці наведено в таблиці 3.1.

Таблиця 3.1 – Опис утиліт

№ п/п	Назва утиліти	Опис застосування
1	Visual Studio	IDE в якій виконується розробка і для якої пишеться розширення
2	Microsoft.VisualStudio	Основний пакет для розробки розширень, що надає доступ до API Visual Studio, зокрема DTE. Надає можливість визначення команд, меню тощо
3	Microsoft.CodeAnalysis	Пакет, що містить API компілятора Roslyn для синтаксичного і семантичного аналізу коду
4	Microsoft.CodeAnalysis .Workspaces	Пакет, що містить API Roslyn для аналізу проєктів і рішень
5	Microsoft.Build	Пакет, що містить API MSBuild для аналізу проєктів і рішень, а також можливості взаємодії із вмістом файлів проєкту *.csproj
6	Community.VisualStudio .Toolkit	Допоміжний пакет, що полегшує розробку розширень для Visual Studio. Грає роль фасаду над основним API, спрощуючи багато елементів взаємодії з Visual Studio
7	Microsoft.AspNetCore .Mvc	Пакет, що містить необхідну інфраструктуру для підтримки шаблонів Razor, зокрема базові типи та генератори коду

Тексти програмного коду наведені в окремому документі «Текст програми».

3.4 Аналіз безпеки даних

Розроблюване програмне забезпечення не містить модулів, що є критичними з точки зору безпеки даних. Розширення встановлюється безпосередньо в середовище Visual Studio на машині розробника, не взаємодіє з мережею, не потребує особистих даних користувача для роботи. Наслідки будь-якого несанкційованого використання розширення несе особисто користувач.

Висновки до розділу

Третій розділ пояснювальної записки мав на меті конструювання та розроблення програмного забезпечення.

Спочатку була описана на високому рівні архітектура розроблюваного розширення: наведені діаграма пакетів і діаграма компонентів UML, опис і призначення окремих компонентів.

Були обґрунтовані засоби розробки розширення: вирішено використовувати мову C# і середовище Visual Studio, бібліотеку VSSDK для отримання доступу до API середовища, компілятор Roslyn для аналізу коду і проєктів, а також шаблонізатор Razor для генерації файлів з тестами та файлів проєкту (*.csproj).

Було детально описано дизайн і призначення кожного проєкту, описано їх ключові типи та модулі, взаємодію між ними, наведені діаграми класів. Зазначено проєктувальні патерни, що були використані при розробці: адаптер, стратегія, фасад, специфікація, а також архітектурний патерн MVVM. Також наведено та детально описано алгоритм підстановки аргументів типу в універсальних символах. Функція “Smart Generics”, що використовує цей алгоритм, була продемонстрована на прикладі універсального типу та універсальних методів.

Був проведений аналіз безпеки даних і обґрунтовано відсутність вразливостей у розроблюваному розширенні.

4 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

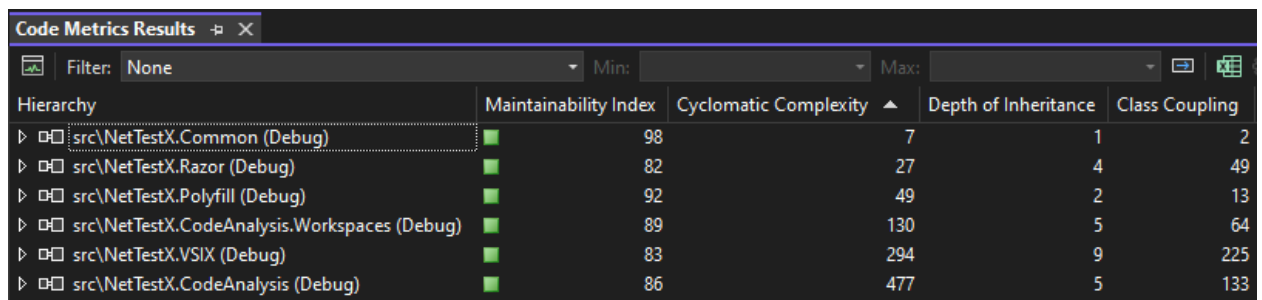
4.1 Аналіз якості ПЗ

Аналіз якості розробленого рішення складається з оцінки значення обраних метрик коду, оцінки ефективності розширення з точки зору економії часу на розробку тестів, а також з точки зору покриття коду тестами.

Для аналізу якості ПЗ було обрано наступні метрики коду:

- Maintainability Index (індекс підтримуваності)
- Cyclomatic Complexity
- Depth of Inheritance
- Class Coupling

Значення метрик обраховується статичними аналізаторами коду, вбудованими в середовище Visual Studio (рис. 4.1).



Hierarchy	Maintainability Index	Cyclomatic Complexity	Depth of Inheritance	Class Coupling
src\NetTestX.Common (Debug)	98	7	1	2
src\NetTestX.Razor (Debug)	82	27	4	49
src\NetTestX.Polyfill (Debug)	92	49	2	13
src\NetTestX.CodeAnalysis.Workspaces (Debug)	89	130	5	64
src\NetTestX.VSIX (Debug)	83	294	9	225
src\NetTestX.CodeAnalysis (Debug)	86	477	5	133

Рисунок 4.1 – Обрахунок метрик у Visual Studio

Опишемо та наведемо значення кожної метрики в пунктах 4.1.1 – 4.1.4.

4.1.1 Індекс підтримуваності

Індекс підтримуваності коду є числом, що визначає відносну легкість підтримки коду. Високе значення означає кращу підтримуваність. Значення цього індексу обчислюється за наступною формулою [25]:

$$MI = 171 - 5.2 \ln(HV) - 0.23(CC) - 16.2 \ln(LOC) \quad (4.1)$$

При цьому MI – індекс підтримуваності, HV – міра Халстеда, CC – цикломатична складність, LOC – кількість рядків коду. Наведена формула (3.1) набуває значень від 171 до мінус нескінченності. Проте чим ближче

значення до 0, тим очевидно важче підтримувати код, при цьому різниця між нулем та від’ємними значеннями не є дуже корисною. Таким чином, враховуючи непотрібність від’ємних значень і бажання мати зрозумілу метрику, від’ємні значення рахуються як 0, а додатні – діляться на 1.71, щоб отримати кінцевий діапазон від 0 до 100. Кінцева формула, що застосовується для обчислення метрики, наведена нижче (3.2):

$$MI = \text{MAX}(0, \frac{171 - 5.2\ln(HV) - 0.23(CC) - 16.2\ln(LOC)}{1.71}) \quad (4.2)$$

Значення метрики для кожного проєкту, а також загальне (середнє), наведене в таблиці 4.1.

Таблиця 4.1 – Значення метрики “індекс підтримованості”

Проект	Значення метрики
NetTestX.VSIX	83 (добре)
NetTestX.CodeAnalysis	86 (добре)
NetTestX.CodeAnalysis.Workspaces	89 (добре)
NetTestX.Razor	82 (добре)
NetTestX.Common	98 (добре)
Загально	87.6 (добре)

4.1.2 Цикломатична складність

Однією із найменш зрозумілих метрик коду є цикломатична складність. Фактично, більші значення цієї метрики є гіршими, а менші - кращими. Цикломатична складність надає уявлення про те, наскільки складним може бути певний код для тестування, підтримки чи усунення несправностей, а також для вказівки того, наскільки ймовірно код матиме баги.

Цикломатична складність розраховується шляхом обчислення кількості лінійно незалежних шляхів у коді [25]. Програма, що має складний потік управління, потребує більше тестів для високого покриття коду і важче підтримується.

Значення метрики для кожного проєкту, а також загальне (сумарне), наведене в таблиці 4.2.

Таблиця 4.2 – Значення метрики “цикломатична складність”

Проект	Значення метрики
NetTestX.VSIX	294
NetTestX.CodeAnalysis	477
NetTestX.CodeAnalysis.Workspaces	130
NetTestX.Razor	27
NetTestX.Common	7
Загально	935

4.1.3 Глибина наслідування

Вказує на максимальну глибину дерева наслідування між різними класами [26]. Метрика глибини наслідування подібна до метрики зв’язності коду, оскільки зміна в базовому класі може вплинути на будь-який з його класів-наслідників. Чим вище це число, тим глибше успадкування та вищий потенціал для модифікацій базового класу, які призведуть до критичних змін [26]. Для цієї метрики нижче значення – краще, а більше – гірше.

Значення метрики для кожного проєкту, а також загальне (середнє), наведене в таблиці 4.3.

Таблиця 4.3 – Значення метрики “глибина наслідування”

Проект	Значення метрики
NetTestX.VSIX	9
NetTestX.CodeAnalysis	5
NetTestX.CodeAnalysis.Workspaces	5
NetTestX.Razor	4
NetTestX.Common	1
Загально	4.8

4.1.4 Зв'язність коду

Зв'язність коду є мірою того, скільки класів використовує окремий клас [26]. Високе число – це погано, а низьке – зазвичай добре для цієї метрики.

Значення метрики зв'язності коду розраховується за допомогою параметрів, локальних змінних, типів, що повертаються, викликів методів, базових класів, реалізацій інтерфейсу, полів, атрибутів. Висока зв'язність вказує на те, що дизайн важко повторно використовувати та підтримувати через його багато взаємозалежностей від інших типів [26].

Значення метрики для кожного проєкту, а також загальне (сумарне), наведене в таблиці 4.4.

Таблиця 4.4 – Значення метрики “зв'язність коду”

Проект	Значення метрики
NetTestX.VSIX	225
NetTestX.CodeAnalysis	133
NetTestX.CodeAnalysis.Workspaces	64
NetTestX.Razor	49
NetTestX.Common	2
Загально	463

4.1.5 Оцінка ефективності (економія часу)

Ключовим параметром якості розробленого рішення є його ефективність в плані економії часу на розробку тестів. Для оцінки цього параметра порівнюється кількість рядків згенерованого коду та загальна кількість рядків коду, що містять юніт-тести. Кількість згенерованих рядків позначимо GLOC, кількість рядків загально – SLOC. Тоді ефективність E можна буде обчислити за формулою (4.3):

$$E = \frac{GLOC}{SLOC} * 100\% \quad (4.3)$$

У якості проєктів для генерації тестів були обрані проєкти самого розширення: Razor, CodeAnalysis, CodeAnalysis.Workspaces. Для них були

створені відповідні тестові проєкти зі згенерованими юніт-тестами, після чого тести були дописані. Обрахунок кількості рядків коду для цих проєктів у Visual Studio наведено на рисунку 4.2.

Hierarchy ▲	Lines of Source code
▷  comparison\Common\NetTestX.CodeAnalysis.Tests (Debug)	1,944
▷  comparison\Common\NetTestX.CodeAnalysis.Workspaces.Tests (Debug)	565
▷  comparison\Common\NetTestX.Razor.Tests (Debug)	118
▷  comparison\NetTestX\CodeAnalysis.Tests.NetTestX (Debug)	1,203
▷  comparison\NetTestX\CodeAnalysis.Workspaces.Tests.NetTestX (Debug)	415
▷  comparison\NetTestX\Razor.Tests.NetTestX (Debug)	74

Рисунок 4.2 – Обрахунок кількості рядків коду для тестових проєктів

Проекти, що містяться у папці Common, містять дописані (фінальні) тести. Проекти, що містяться у папці NetTestX, містять виключно згенеровані тести. Маючи дані про кількість рядків коду, можемо зробити порівняльну таблицю ефективності розширення (табл. 4.5).

Таблиця 4.5 – Оцінка ефективності розширення

Проект	GLOC	SLOC	Ефективність
Razor	74	118	62.7%
CodeAnalysis	1203	1944	61.8%
CodeAnalysis.Workspaces	415	565	73.5%
Загально	1692	2627	64.4%

Отримане загальне значення у 64.4% дозволяє зробити висновок про ефективність розробленого рішення з точки зору кількості згенерованого коду та економії часу на написання тестів.

4.1.6 Оцінка ефективності (покриття тестами)

Ще одним параметром якості розширення є покриття коду згенерованими тестами. Для цього тести, згенеровані та дописані для проєктів, наведених у таблиці 4.5, були проаналізовані на покриття коду засобами ReSharper [2]. Результати аналізу наведені на рисунку 4.3.

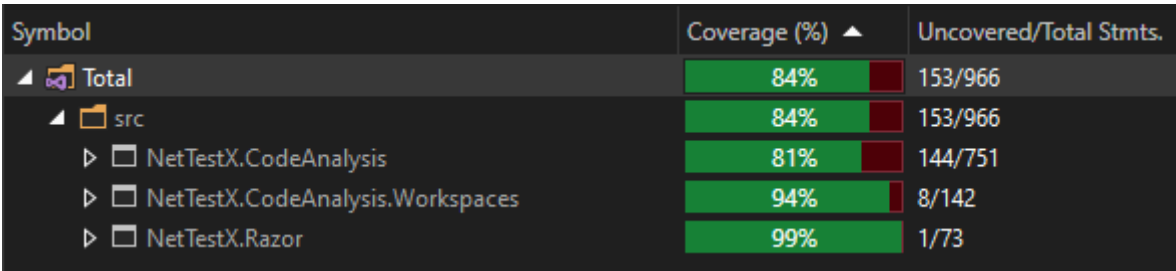


Рисунок 4.3 – Покриття коду згенерованими тестами

Отримане загальне значення покриття коду у 84% дозволяє зробити висновок про ефективність розробленого рішення з точки зору покриття коду тестами.

4.2 Опис процесів тестування

Тестування виконується згідно документу «Програма та методика тестування».

Для тестування розширення були застосовані юніт-тести, інтеграційні тести, мануальні тести.

Опис відповідних тестів, виконаних при мануальному тестуванні програмного забезпечення, наведено у таблицях 4.6 – 4.15. Усі тести виконуються в середовищі Visual Studio зі встановленим розширенням.

Таблиця 4.6 – Тест 1. Створення тестового проєкту

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить прийнятні один проєкт
Вхідні дані	Назва проєкту, директорія проєкту, тестовий фреймворк, бібліотека моків, необхідність генерації InternalsVisibleTo

Продовження таблиці 4.6

Опис проведення тесту	У вікні дослідника рішень обирається існуючий проєкт C# і для нього викликається контекстне меню. У меню обирається команда “Generate Test Project...”, після чого з’являється вікно створення проєкту. У вікні задається назва проєкту, що не має містити спеціальних символів і пробілу, директорія проєкту, що має бути валідним шляхом Windows, тестовий фреймворк і бібліотека моків з випадаючого списку, а також відмічається опція ”Set InternalsVisibleTo”
Очікуваний результат	В обраній папці генерується новий проєкт C#. До файлу *.csproj додаються посилання на обраний тестовий фреймворк і бібліотеку моків. До файлу початкового проєкту додається властивість InternalsVisibleTo, якщо відповідна опція була обрана при генерації
Фактичний результат	Збігається з очікуванням

Таблиця 4.7 – Тест 2. Генерація юніт-тестів з вікна редактора коду для простого випадку

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Публічний клас, що містить наступні члени: - Публічне поле типу int; - Публічна властивість типу string; - Публічний метод без параметрів, що повертає void; - Приватний метод без параметрів, що повертає void

Продовження таблиці 4.7

Опис проведення тесту	У вікні редактора коду для заданого класу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs із тестами для обраного класу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. Згенерований файл містить один юніт-тест для публічного методу класу. У тесті створюється екземпляр класу, після чого викликається метод
Фактичний результат	Збігається з очікуванням

Таблиця 4.8 – Тест 3. Генерація юніт-тестів з вікна дослідника рішень для простого випадку

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Публічний клас, що містить наступні члени: - Публічне поле типу int; - Публічна властивість типу string; - Публічний метод без параметрів, що повертає void; - Приватний метод без параметрів, що повертає void
Опис проведення тесту	У вікні дослідника рішень для заданого файлу з класом викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)

Продовження таблиці 4.8

Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs із тестами для обраного класу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. Згенерований файл містить один юніт-тест для публічного методу класу. У тесті створюється екземпляр класу, після чого викликається метод
Фактичний результат	Збігається з очікуванням

Таблиця 4.9 – Тест 4. Генерація юніт-тестів з вікна редактора коду для складного випадку

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Публічний клас, що містить наступні члени: - Публічний конструктор, що має 3 параметри: int, ref int, in int - Публічний метод, що має 4 параметри: string, ref string, in string, out string, і повертає void - Публічний асинхронний метод без параметрів, що повертає Task - Публічний асинхронний метод без параметрів, що повертає Task<object> - Публічний статичний метод без параметрів, що повертає void - Публічний асинхронний статичний метод без параметрів, що повертає Task

Продовження таблиці 4.9

Опис проведення тесту	У вікні редактора коду для заданого класу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs із тестами для обраного класу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. Згенерований файл містить 5 тестів для відповідних методів. Тести використовують правильний конструктор
Фактичний результат	Збігається з очікуванням

Таблиця 4.10 – Тест 5. Генерація юніт-тестів з вікна дослідника рішень для файлу з кількома типами

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Файл, що містить 5 публічних типів: class, struct, static class, record class, readonly record struct. Кожен тип містить публічний статичний метод без параметрів, що повертає void
Опис проведення тесту	У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту). На екран виводиться попередження про генерацію кількох файлів. Натискається кнопка “ОК”

Продовження таблиці 4.10

Очікуваний результат	У папці обраного тестового проєкту генерується 5 файлів *.cs із тестами для кожного типу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. Згенеровані файли містять один юніт-тест для публічного методу класу
Фактичний результат	Збігається з очікуванням

Таблиця 4.11 – Тест 6. Генерація тестів з коментарями AAA (Arrange – Act – Assert)

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Файл, що містить один публічний тип. Тип містить один публічний метод
Опис проведення тесту	У вікні налаштувань розширення (меню Tools -> Options) відмічається опція “Include AAA Comments”. У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs з одним тестом для публічного методу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. Згенерований тест містить коментарі над відповідними секціями тесту: Arrange, Act, Assert.
Фактичний результат	Збігається з очікуванням

Таблиця 4.12 – Тест 7. Генерація тестів для універсальних символів без використання Smart Generics

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Файл, що містить один універсальний публічний тип. Тип містить один публічний універсальний метод
Опис проведення тесту	У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs з одним тестом для публічного методу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. У згенерованому тесті аргументи типу для універсального типу та універсального методу не підстановлені
Фактичний результат	Збігається з очікуванням

Таблиця 4.13 – Тест 8. Генерація тестів для універсальних символів із використанням Smart Generics

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Файл, що містить один універсальний публічний тип. Тип містить один публічний універсальний метод

Продовження таблиці 4.13

Опис проведення тесту	У вікні налаштувань розширення (меню Tools -> Options) відмічається опція “Use Smart Generics”. У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	У папці обраного тестового проєкту генерується файл *.cs з одним тестом для публічного методу. Тестовий фреймворк та бібліотека моків визначаються тестовим проєктом, до якого був доданий файл. У згенерованому тесті аргументи типу для універсального типу та універсального методу підстановлені та задовільняють обмеженням (якщо вони присутні)
Фактичний результат	Збігається з очікуванням

Таблиця 4.14 – Тест 9. Валідація даних при розширеній генерації тестів

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий
Вхідні дані	Файл, що містить один публічний тип. Тип містить один публічний метод
Опис проведення тесту	У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests Advanced...”. Задаються неправильні значення “Test Class Name” і “Test Class Namespace” (наприклад, %)

Продовження таблиці 4.14

Очікуваний результат	Відповідні поля вводу даних підсвічено червоним кольором. Кнопка “Continue” вимкнена та має сірий колір, допоки помилки валідації не будуть усунуті
Фактичний результат	Збігається з очікуванням

Таблиця 4.15 – Тест 10. Генерація тестів для internal символів

Початковий стан системи	Користувач знаходиться у середовищі Visual Studio із відкритим рішенням, що містить два проєкти: звичайний і тестовий. У вікні налаштувань розширення (Tools -> Options) увімкнена опція “Test Internal Members”
Вхідні дані	Файл, що містить один публічний тип і один internal тип. Обидва типи містять два методи: один – публічний, другий – internal
Опис проведення тесту	У вікні дослідника рішень для заданого файлу викликається контекстне меню. У меню обирається команда “Generate Unit Tests” -> “In TestProject” (назва існуючого тестового проєкту)
Очікуваний результат	Згенеровано 2 файли з тестами для типів. Кожен файл містить 2 тести для обох методів типів. Якщо в початковому проєкті не було встановлене значення <InternalsVisibleTo />, то воно додається до файлу *.csproj.
Фактичний результат	Збігається з очікуванням

4.3 Опис контрольного прикладу

Контрольним прикладом є тестування кожного з аспектів функціоналу розширення. Опишемо покроково кроки прикладу.

Відкрити Visual Studio. У вікні Tools -> Options знайти налаштування розширення та змінити наступні налаштування (рис. 4.4 – 4.6):

- Test Internal Members: true
- Multiple Type Warning: false
- Use Smart Generics: true

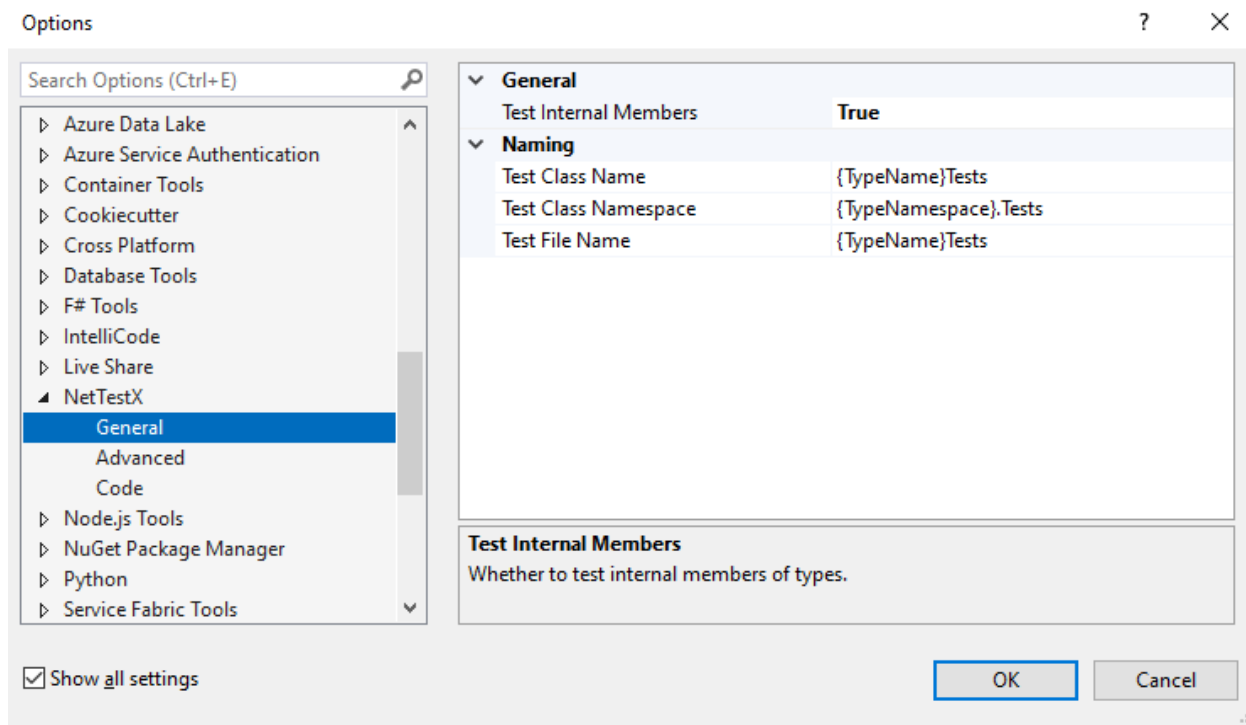


Рисунок 4.4 – Налаштування розширення, сторінка General

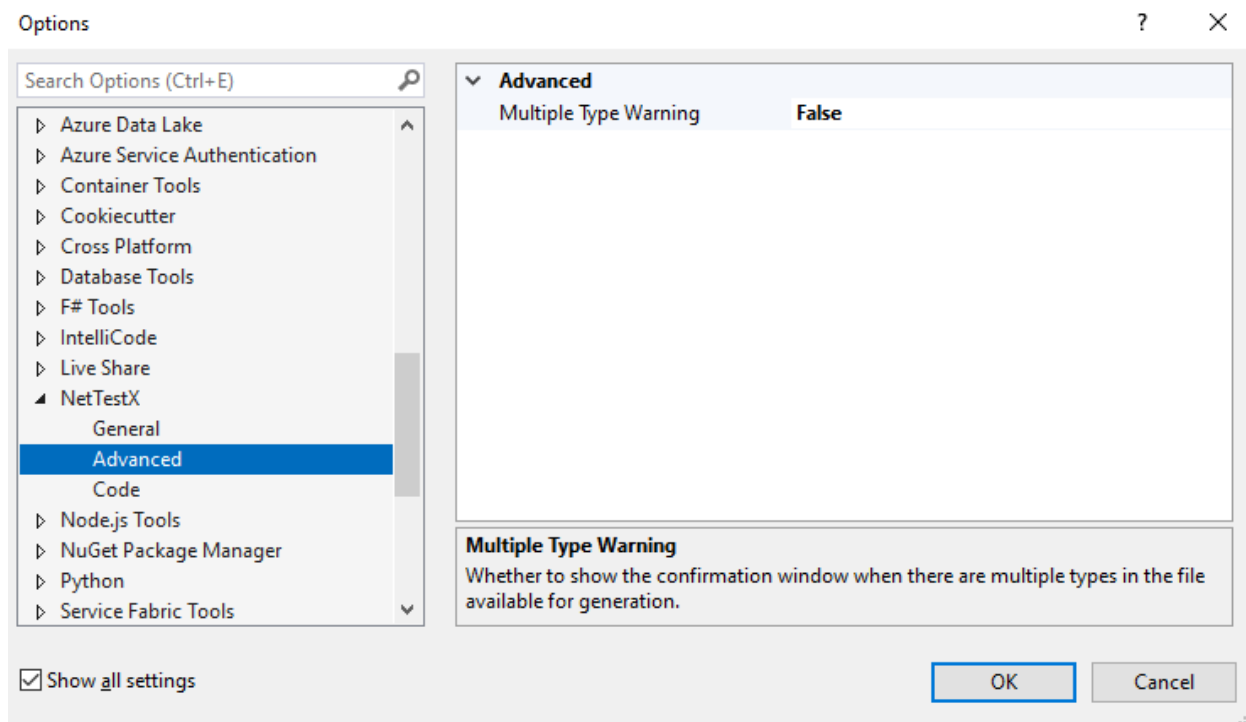


Рисунок 4.5 – Налаштування розширення, сторінка Advanced

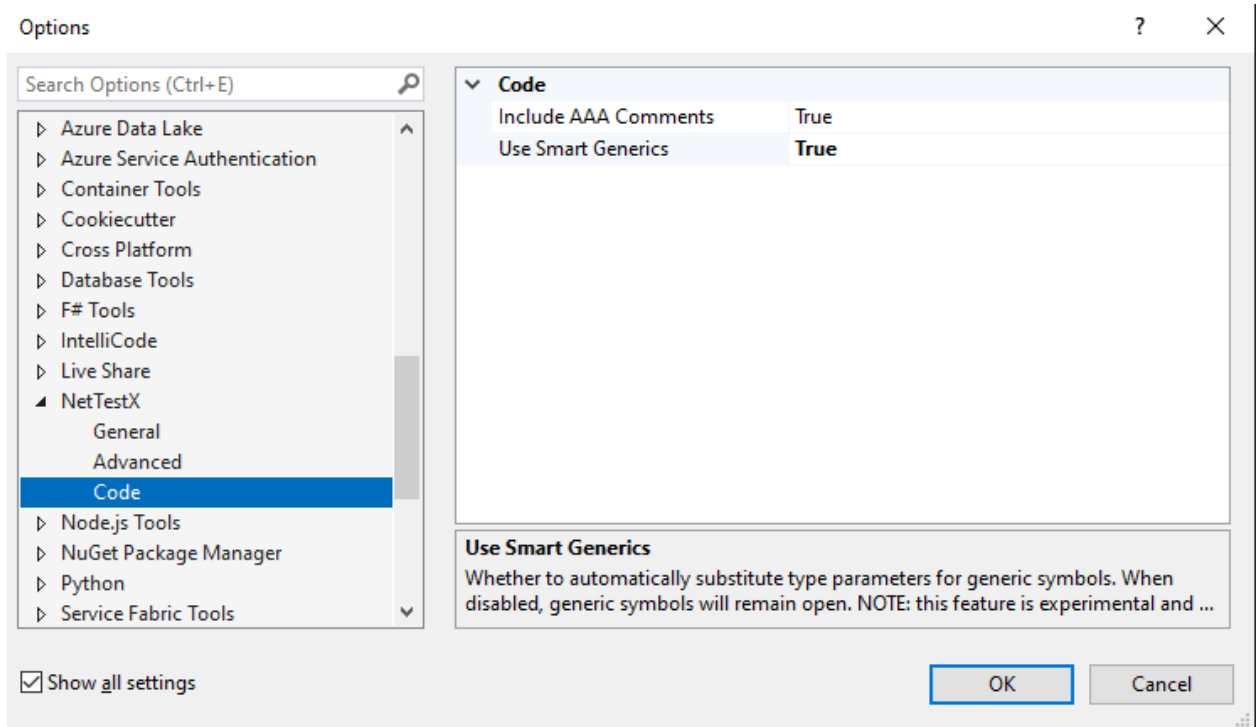


Рисунок 4.6 – Налаштування розширення, сторінка Code

Відповідні налаштування, значення яких відрізняється від значення за замовченням, виділені жирним.

Створити проєкт C# у Visual Studio, у ньому задати таку файлову структуру, як на рис. 4.7. Зміст файлів наведений на рис. 4.8 – 4.12.

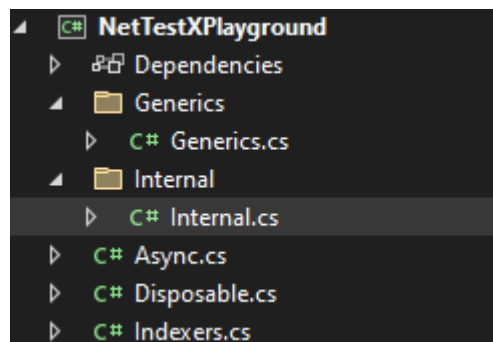


Рисунок 4.7 – Файлова структура проєкту для контрольного прикладу

```
public class Async
{
    public async Task DoAsync1(IEnumerable<int> ints) { }

    public async Task<int> DoAsync2(IEqualityComparer<Async> comparer) => 42;

    public static async Task DoStaticAsync1(IEnumerable<int> ints) { }

    public static async Task<int> DoStaticAsync2(IEqualityComparer<Async> comparer) => 42;
}
```

Рисунок 4.8 – Зміст файлу Async.cs

```
namespace NetTestXPlayground;

public class Disposable : IDisposable, IAsyncDisposable
{
    public void Dispose() { }

    public async ValueTask DisposeAsync() { }
}
```

Рисунок 4.9 – Зміст файлу Disposable.cs

```
namespace NetTestXPlayground;

public class Indexers
{
    public string this[int i, char c, object o] => "";
}
```

Рисунок 4.10 – Зміст файлу Indexers.cs

```
namespace NetTestXPlayground.Generics;

public class GenericClass<T1, T2, T3, T4>
    where T1 : class
    where T3 : class, IEquatable<T4>
{
    public void Do<K1, K2, K3>()
        where K3 : unmanaged
    { }
}

public struct GenericStruct<T1, T2>
    where T1 : IDictionary<T2, T2>
    where T2 : struct
{
    public void Do<K1, K2, K3>()
        where K3 : unmanaged
    { }
}
```

Рисунок 4.11 – Зміст файлу Generics.cs


```
namespace NetTestXPlayground.Internal;

internal class Internal
{
    public void Do() { }
}
```

Рисунок 4.12 – Зміст файлу Internal.cs

Обрати всі файли у вікні дослідника рішень, викликати контекстне меню, в ньому обрати “Generate Unit Tests -> “Generate Test Project” (рис. 4.13)

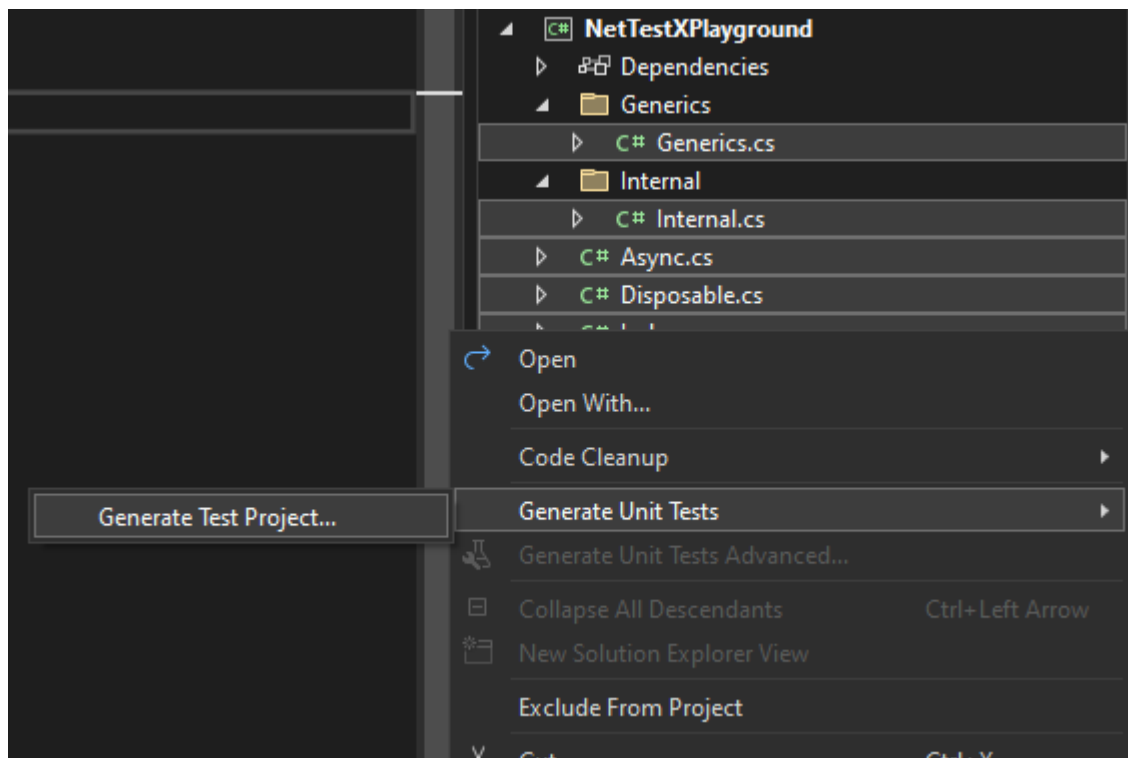


Рисунок 4.13 – Виклик команди генерації тестового проєкту

У формі, що з’явилась, додати символ “%” до назви тестового проєкту. Переконавшись, що кнопка продовження вимкнута, а проблематичне поле – підсвічено червоним (рис. 4.14).

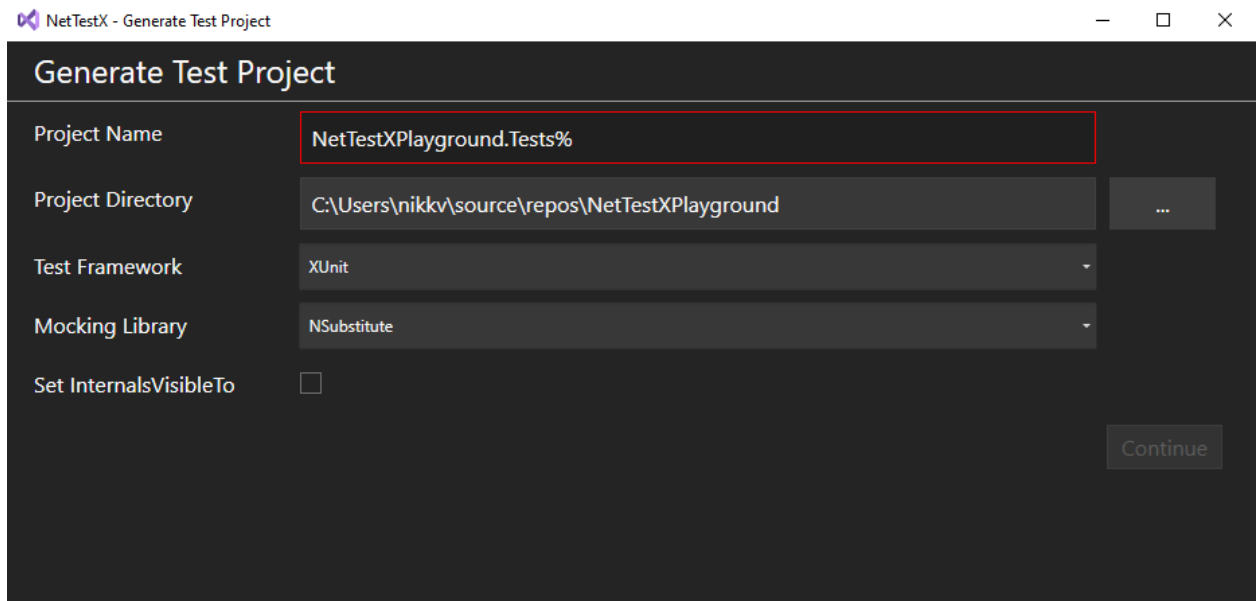


Рисунок 4.14 – Помилка валідації при генерації тестового проєкту

Прибрати проблематичний символ та натиснути кнопку “Continue”. Переконайтеся, що новий проєкт було створено і додано в рішення, а також, що до нього було додано згенеровані файли з тестами (рис. 4.15). При цьому файлова структура збігається.

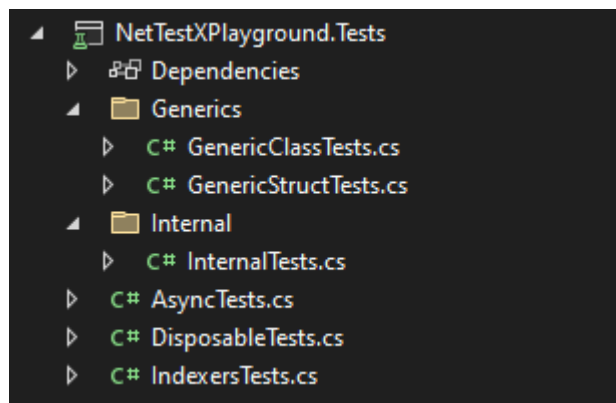


Рисунок 4.15 – Файлова структура в згенерованому проєкті

Зібрати проєкт і переконайтеся, що в ньому немає помилок компіляції. Перевірити, що зміст згенерованих файлів відповідає змісту файлів на рис. 4.16 – 4.21.

```

namespace NetTestXPlayground.Tests;

public class AsyncTests
{
    [Fact]
    public async Task TestDoAsync1()
    {
        // Arrange
        Async sut = new();

        var testInts = Substitute.For<IEnumerable<int>>();

        // Act
        await sut.DoAsync1(testInts);

        // Assert
    }

    [Fact]
    public async Task TestDoAsync2()
    {
        // Arrange
        Async sut = new();

        var testComparer = Substitute.For<IEqualityComparer<Async>>();

        // Act
        var result = await sut.DoAsync2(testComparer);

        // Assert
    }
}

[Fact]
public async Task TestDoStaticAsync1()
{
    // Arrange
    var testInts = Substitute.For<IEnumerable<int>>();

    // Act
    await Async.DoStaticAsync1(testInts);

    // Assert
}

[Fact]
public async Task TestDoStaticAsync2()
{
    // Arrange
    var testComparer = Substitute.For<IEqualityComparer<Async>>();

    // Act
    var result = await Async.DoStaticAsync2(testComparer);

    // Assert
}

```

Рисунок 4.16 – Зміст файлу AsyncTests.cs

```

namespace NetTestXPlayground.Tests;

public class DisposableTests
{
    [Fact]
    public void TestDisposable()
    {
        // Arrange
        IDisposable sut = new Disposable();

        // Act
        using (sut) { }

        // Check that the object ignores all calls to Dispose after the first one (and does not throw)
        sut.Dispose();

        // Assert
    }

    [Fact]
    public async Task TestAsyncDisposable()
    {
        // Arrange
        IAsyncDisposable sut = new Disposable();

        // Act
        await using (sut) { }

        // Check that the object ignores all calls to Dispose after the first one (and does not throw)
        var disposeTask = sut.DisposeAsync();

        // Assert
        Assert.True(disposeTask.IsCompleted);
    }
}

```

Рисунок 4.17 – Зміст файлу DisposableTests.cs

```

namespace NetTestXPlayground.Tests;

public class IndexersTests
{
    [Fact]
    public void TestintcharobjectIndexer()
    {
        // Arrange
        Indexers sut = new();

        var testI = 0;
        var testC = ' ';
        var testO = default(object);

        // Act
        var result = sut[testI, testC, testO];

        // Assert
    }
}

```

Рисунок 4.18 – Зміст файлу IndexersTests.cs

```

namespace NetTestXPlayground.Generics.Tests;

public class GenericClassTests
{
    [Fact]
    public void TestDo()
    {
        // Arrange
        GenericClass<object, object, SocketAddress, SocketAddress> sut = new();

        // Act
        sut.Do<object, object, int>();

        // Assert
    }
}

```

Рисунок 4.19 – Зміст файлу GenericClassTests.cs

```
namespace NetTestXPlayground.Generics.Tests;

public class GenericStructTests
{
    [Fact]
    public void TestDo()
    {
        // Arrange
        GenericStruct<Dictionary<int, int>, int> sut = new();

        // Act
        sut.Do<object, object, int>();

        // Assert
    }
}
```

Рисунок 4.20 – Зміст файлу GenericStructTests.cs

```
namespace NetTestXPlayground.Internal.Tests;

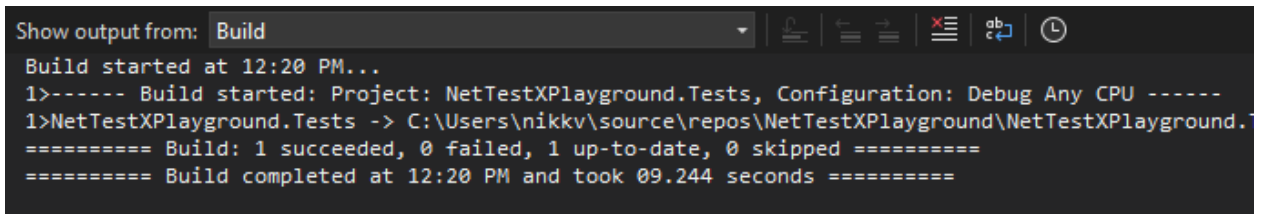
public class InternalTests
{
    [Fact]
    public void TestDo()
    {
        // Arrange
        Internal sut = new();

        // Act
        sut.Do();

        // Assert
    }
}
```

Рисунок 4.21 – Зміст файлу InternalTests.cs

Зібрати проєкт у Visual Studio. Переконатись, що компіляція пройшла успішно (рис. 4.22).



```

Show output from: Build
Build started at 12:20 PM...
1>----- Build started: Project: NetTestXPlayground.Tests, Configuration: Debug Any CPU -----
1>NetTestXPlayground.Tests -> C:\Users\nikkv\source\repos\NetTestXPlayground\NetTestXPlayground.T
===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
===== Build completed at 12:20 PM and took 09.244 seconds =====

```

Рисунок 4.22 – Компіляція проєкту зі згенерованими тестами

В результаті було задано налаштування розширення, згенеровано тестовий проєкт, а також тести до нього, при цьому були враховані різні можливі типи тестів, було перевірено, що проєкт компілюється.

Висновки до розділу

Метою четвертого розділу було тестування та оцінка якості розробленого розширення. На початку розділу було проведено детальний аналіз ПЗ за обраними метриками (індекс підтримуваності, цикломатична складність, глибина наслідування, зв'язність коду), а також описані самі метрики. Були наведені порівняльні таблиці метрик для кожного проєкту в рішенні. Загалом значення метрик можна вважати хорошими.

Після цього було проведено мануальне тестування розширення, зокрема були протестовані генерація тестового проєкту, генерація тестів з вікна редактора коду і з вікна дослідника рішень, розширена генерація тестів із заданням власних параметрів, генерація тестів для універсальних та internal типів, валідація вхідних даних. Були наведені таблиці з тестами. Результати проведеного тестування збігаються з очікуваними.

Також у розділі було наведено контрольний приклад, що тестує кожен аспект функціоналу розробленого розширення, наведені ілюстрації, що демонструють коректність роботи розширення та правильність згенерованих тестів. Окрім цього, при генерації була збережена файлова структура оригінального проєкту, а також файли з кількома типами були “розбиті” на індивідуальні файли для кожного типу окремо.

5 РОЗГОРТАННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

5.1 Розгортання програмного забезпечення

Розширення Visual Studio публікуються у вигляді файлу *.vsix [13]. Фактично цей файл є архівом, що містить необхідні файли розширення, і “особливо” сприймається Visual Studio в якості інсталятора. При відкритті цього файлу розширення буде встановлено в середовище (для цього його треба перезавантажити). Розширення можна як завантажувати з інтернету і встановлювати мануально, так і знаходити і встановлювати безпосередньо з середовища Visual Studio (вікно Extensions -> Manage Extensions).

Першим кроком у публікації розробленого розширення є заповнення файлу з метаданими *.vsixmanifest, що містить такі дані як назва, версія, опис, ліцензія, іконка тощо (рис. 5.1).

The image shows the Visual Studio .vsixmanifest file editor. At the top, there are fields for Product Name (NetTestX), Product ID (NetTestX.b68ae9dc-954c-4cdf-8bc4-ca14fc16cb2a), Author (Mykyta Kovalenko), and Version (1.0). Below these is a sidebar with tabs: Metadata (selected), Install Targets, Assets, Dependencies, and Prerequisites. The main area contains the following fields:

- Description: Unit test scaffolder for .NET
- Language: English (United States)
- License: Resources\LICENSE (with a Browse... button)
- Icon: Resources\Icon.png (with a Browse... button)
- Preview Image: Resources\Icon.png (with a Browse... button)
- Tags: (empty text box)
- Release Notes: (empty text box with a Browse... button)
- Getting Started Guide: (empty text box with a Browse... button)
- More Info URL: https://github.com/insouciance/NetTestX

At the bottom, there is a checkbox labeled "This VSIX is in preview" which is checked.

Рисунок 5.1 – Графічний вміст файлу *.vsixmanifest

Іконку рекомендовано робити розміром 90x90, а також із прозорим фоном [13].

Після налаштування файлу з метаданими проєкт NetTestX.VSIX треба зібрати в конфігурації Release, після чого файл *.vsix з’явиться в папці bin (рис. 5.2)

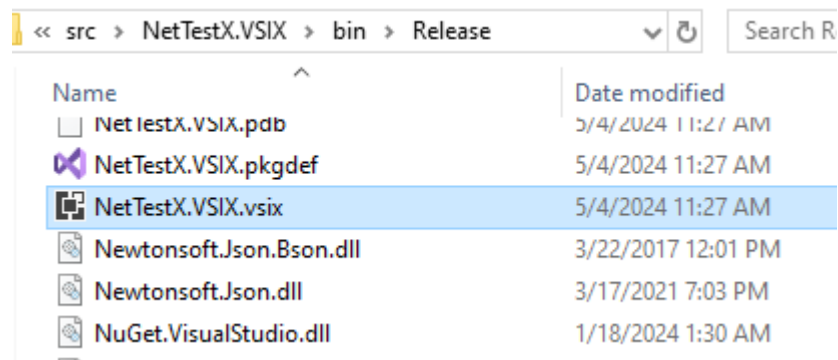


Рисунок 5.2 – Файл розширення *.vsix

Загалом розширення можуть публікуватися будь-де, оскільки для встановлення треба просто скачати і встановити файл *.vsix. Проте де-факто стандартним і найбільш популярним місцем для публікації розширень є сайт Visual Studio Marketplace [27] (рис. 5.3). Він містить тисячі різних розширень, а також має інтеграцію з Visual Studio: якщо завантажити розширення на сайт, то його можна буде встановити прямо з середовища з вікна Extensions -> Manage Extensions.

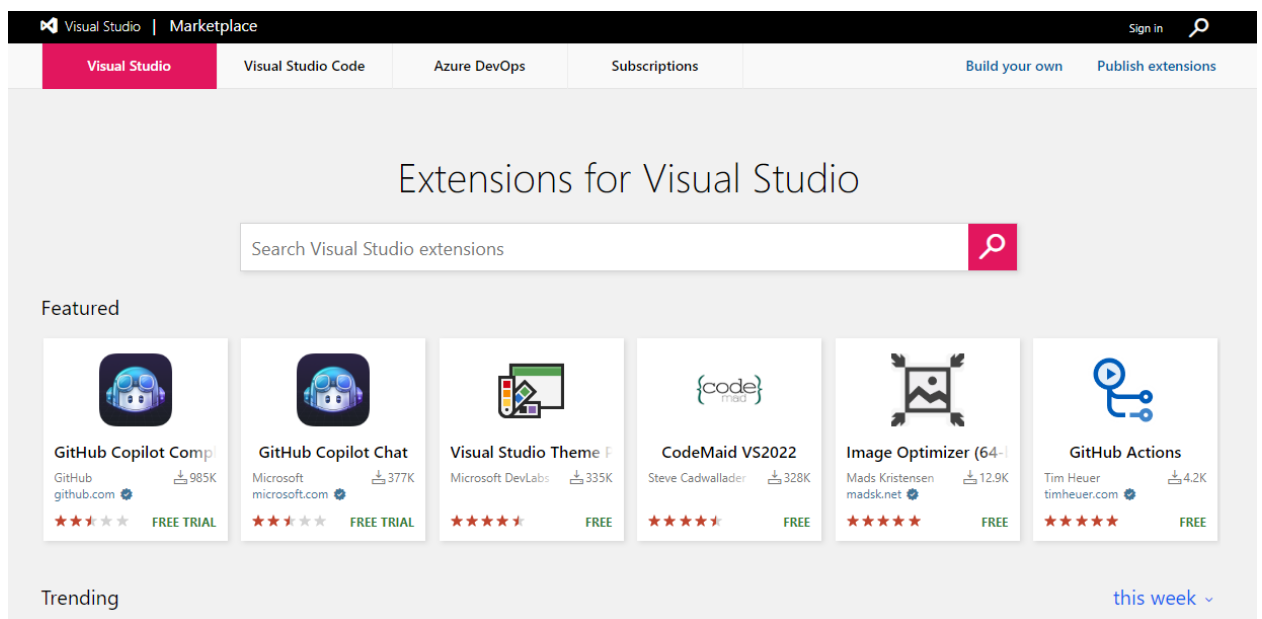


Рисунок 5.3 – Сайт Visual Studio Marketplace

Для публікації розширення на Visual Studio Marketplace треба зареєструватися, після чого натиснути “Publish Extensions” в шапці сайту. Після цього у відкритому вікні Manage Publishers & Extensions натиснути “New Extension” (рис. 5.4).

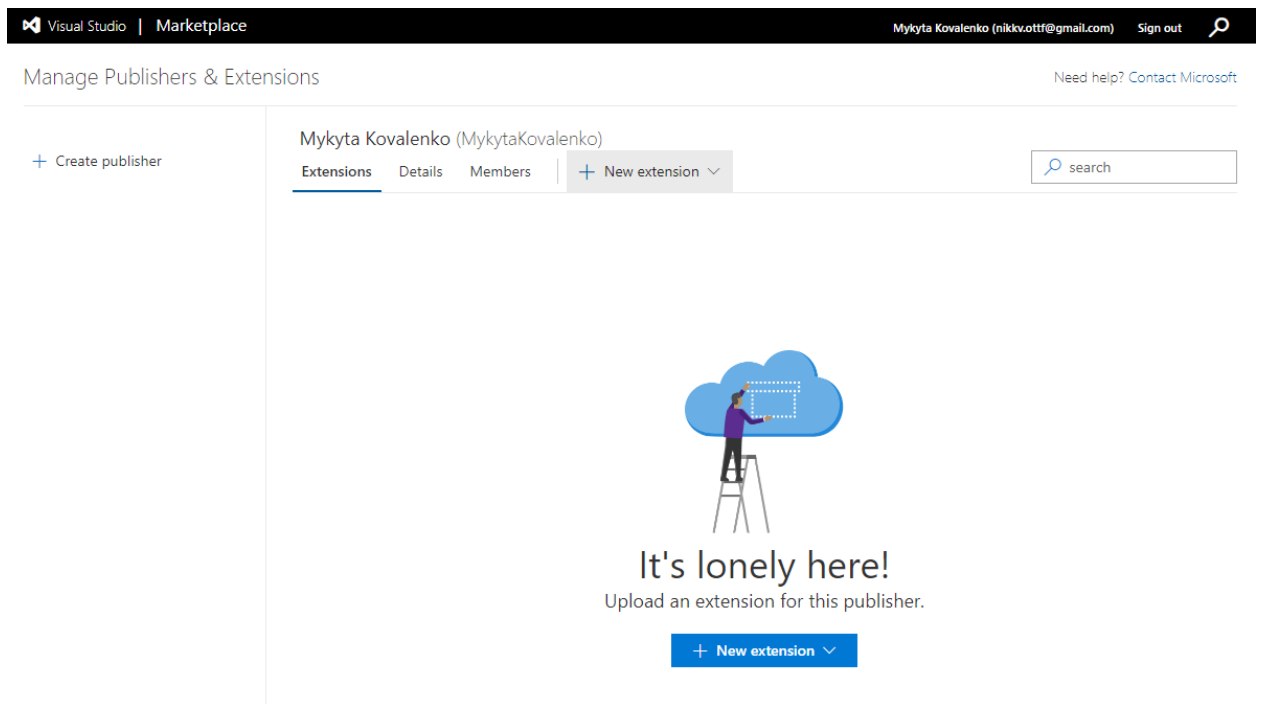


Рисунок 5.4 – Початок публікації розширення

Тепер у відкритому вікні треба власне завантажити файл розширення *.vsix (рис. 5.5):

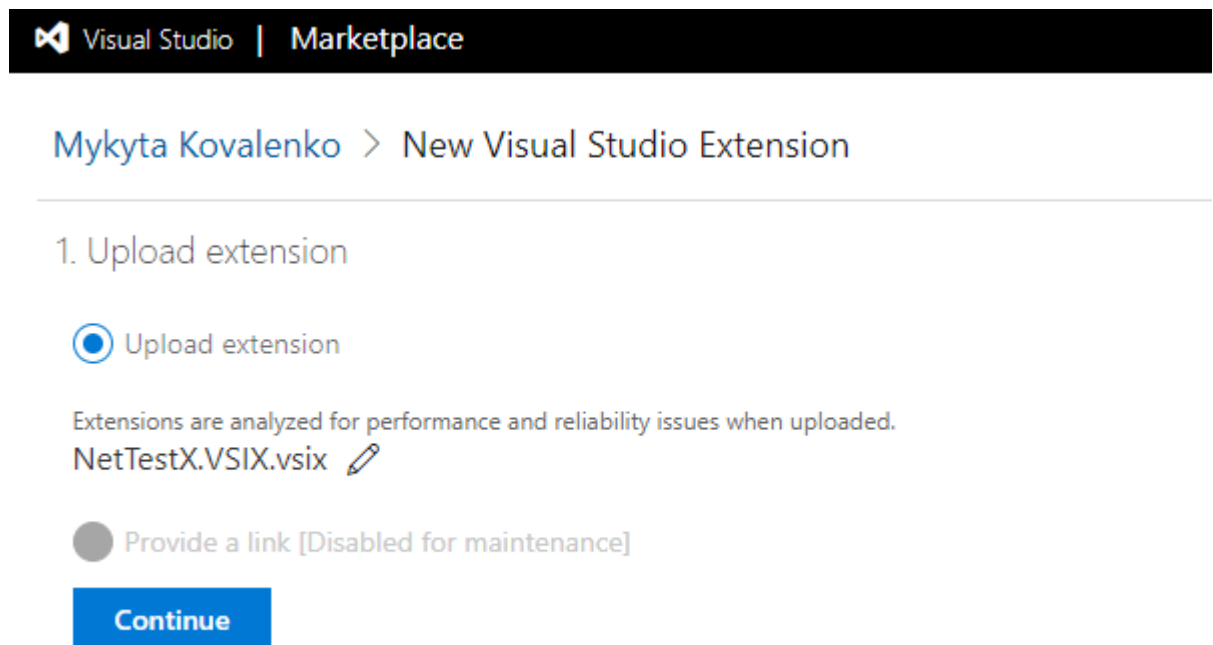


Рисунок 5.5 – Завантаження файлу розширення

Після завантаження файл буде перевірено на наявність вірусів чи шкідливого контенту (напр. zip bomb), після чого буде запропоновано заповнити детальну інформацію про розширення (рис. 5.6):

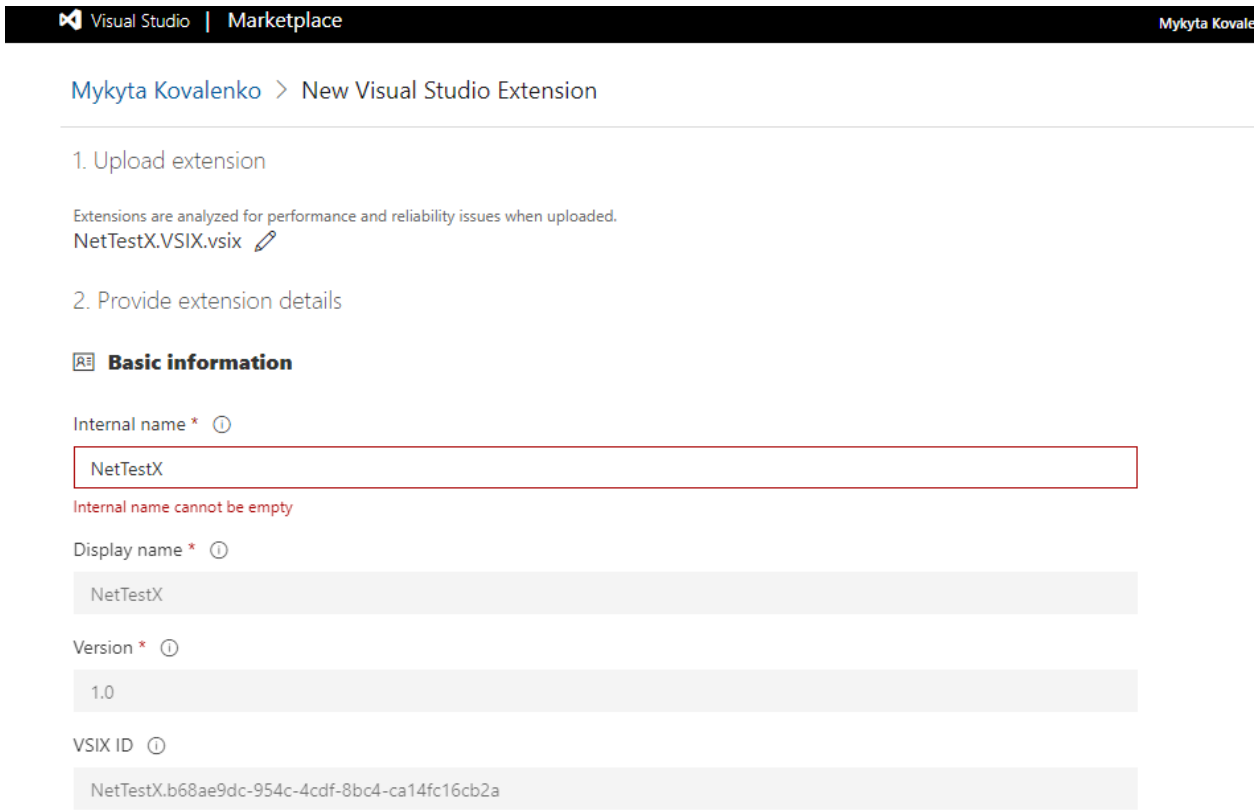


Рисунок 5.6 – Заповнення інформації про розширення

Після заповнення всієї необхідної інформації треба натиснути Save & Upload та почекати доки завантаження закінчиться. Після цього розширення буде опубліковано на сайті (рис. 5.7).

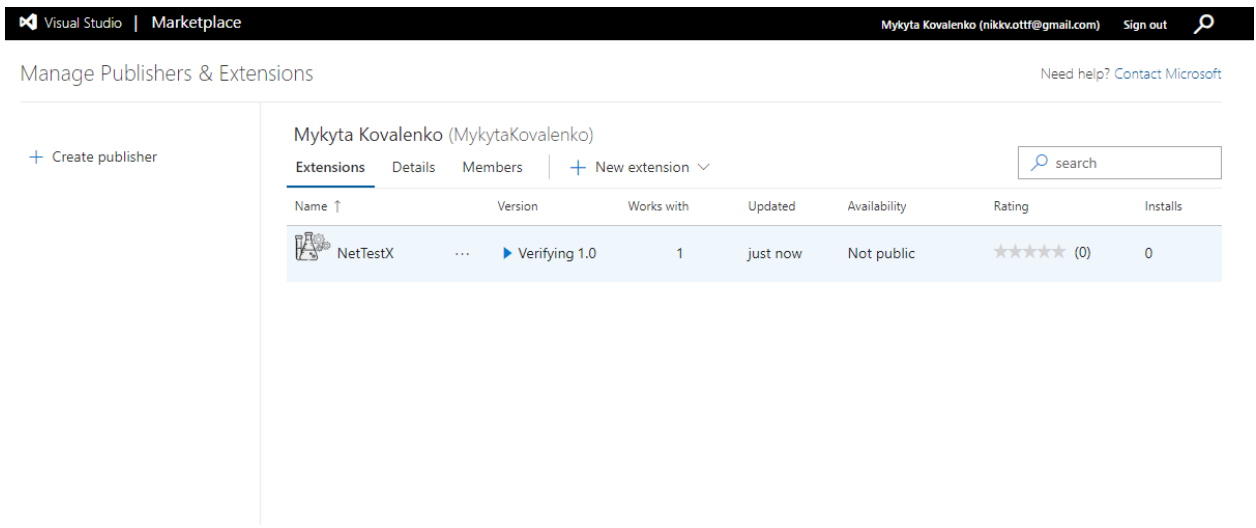
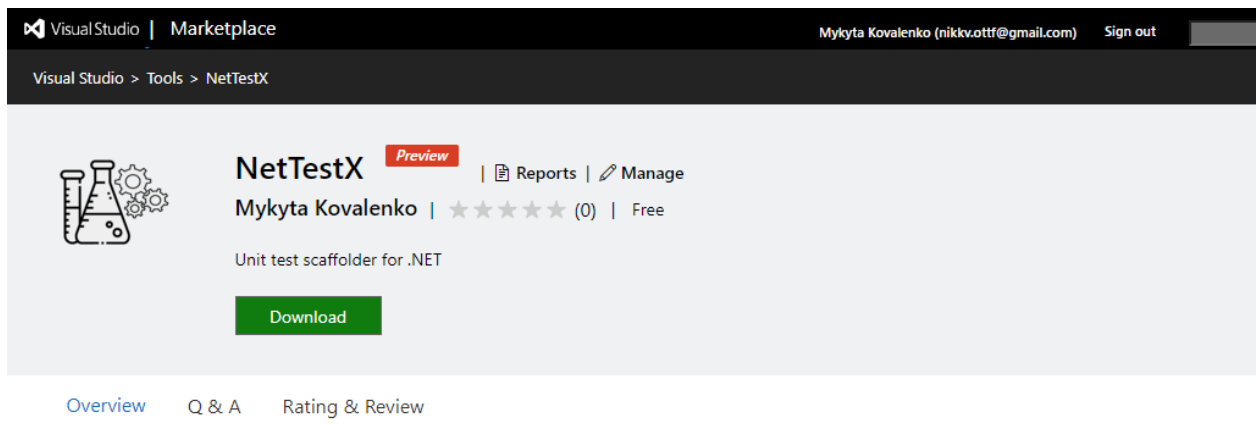


Рисунок 5.7 – Опубліковане розширення на сайті Visual Studio Marketplace

Сторінка розширення на сайті Visual Studio Marketplace зображена на рис. 5.8.



Unit test generator for .NET (C#). Supports:

- Test project generation
- Unit test generation
 - Instance methods
 - Static methods
 - Indexers
 - IDisposable / IAsyncDisposable implementations
- Options
 - Test class name
 - Test class namespace
 - Test class filename
 - AAA comments
 - Test internal members
 - [Experimental] Substitute generic type parameters

Categories

Tools Testing Scaffolding

Works with

Visual Studio 2022 (amd64)

Resources

[License](#)

[Copy ID](#)

Project Details

[insouciance/NetTestX](#)

More Info

Version 1.0

Рисунок 5.8 – Сторінка розширення на сайті Visual Studio Marketplace

Також розширення можливо завантажити напряму із середовища Visual Studio з вікна Extensions -> Manage Extensions (рис. 5.9).

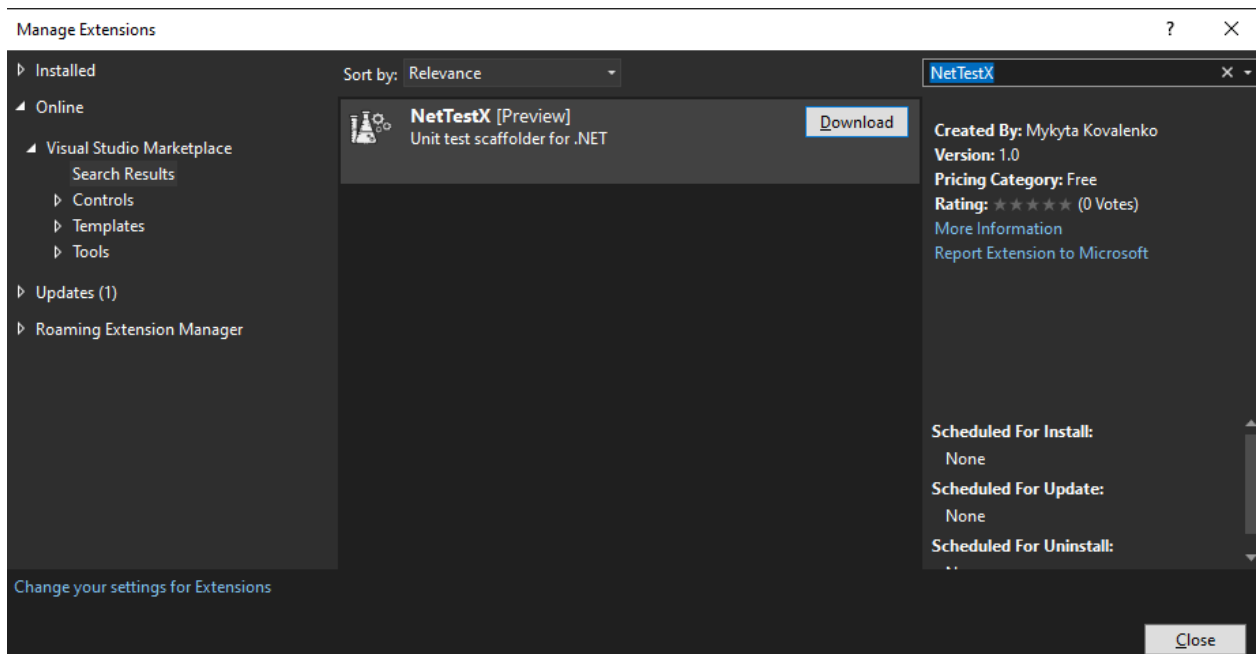


Рисунок 5.9 – Можливість завантаження розширення з IDE

5.2 Супровід програмного забезпечення

Користувачі розширення повинні мати можливість оновлювати встановлене розширення при публікації нової версії. Налаштування процесів CD дозволяє публікувати нові версії автоматично на сторінці репозиторію GitHub, а також ці версії необхідно публікувати на сторінці розширення Visual Studio Marketplace. При цьому користувачі зможуть відкрити сторінку з розширенням у Visual Studio та оновити його, не заходячи спеціально на сайт, що покращує UX і потенційно збільшує кількість користувачів розширення.

Висновки до розділу

Метою п'ятого розділу було розгортання, публікація та супровід розробленого розширення. Було наведено повний процес розгортання розширення. Описано зміст файлу метаданих *.vsixmanifest і задано всі необхідні дані: назва, версія, іконка, опис тощо. Після цього розширення було зібрано в інсталяційний файл *.vsix, а також пояснено його зміст. Для публікації розширення було обрано онлайн-портал Visual Studio Marketplace як де-факто стандарт і найпопулярніше рішення. В результаті розроблене розширення має власну сторінку на Visual Studio Marketplace, а також можливість інтеграції напряму з середовищем Visual Studio: його можна встановити безпосередньо з середовища у вікні Extensions -> Manage Extensions.

Було наведено процеси, що використовуватимуться для підтримки та супроводу розробленого ПЗ: автоматична публікація нових версій на GitHub, публікація на сторінці порталу Visual Studio Marketplace, можливість оновлення напряму з IDE.

ВИСНОВКИ

Розроблене в результаті виконання дипломного проєкту розширення для середовища Visual Studio суттєво полегшує і пришвидшує процес написання юніт-тестів, автоматизуючи рутинні складові розробки. Розширення генерує максимально можливий об'єм коду, який компілюється майже завжди (іноді компіляція неможлива через, наприклад, використання універсальних типів без опції підстановки аргументів типу або таких, що цією функцією не підтримуються). Згенерований код містить більшу частину логіки тесту, при цьому розробнику залишається дописати перевірку результату та – інколи – ініціалізацію змінних. Також розширення дозволяє генерувати тестові проєкти, при цьому додаючи необхідні посилання на пакети та проєкти до файлу *.csproj – функціонал, що містить лише один аналог.

Сформовані та обґрунтовані на початку роботи функціональні задачі були виконані повністю: можливо згенерувати юніт-тести з вікон дослідника рішень та редактора коду, при цьому опціонально задавши розширені налаштування, згенерувати тестові проєкти, при цьому процес генерації відповідає заданим у налаштуваннях розширення опціям.

Спроектоване розширення було встановлено до середовища Visual Studio та використано на практиці для розробки юніт-тестів для свого ж коду. В результаті проведених статистичних досліджень встановлено, що 64.4% від усього коду юніт-тестів було згенеровано, при цьому згенеровані тести дозволили досягнути покриття коду у 84%, що доводить ефективність розробленого рішення.

Подальший розвиток розширення включає генерацію більшої кількості видів тестів, наприклад, тести для реалізацій інтерфейсів за замовченням, тести абстрактних типів чи тести конструкторів. Можливе вдосконалення алгоритму підстановки аргументів типу, оскільки він не підтримує, наприклад, циклічні залежності в обмеженнях параметрів типів. Також можливим вдосконаленням розширення було би створення плагінів, наприклад, для

підтримки більшої кількості тестових фреймворків, бібліотек моків, розробки власних типів тестів або навіть підтримки інших .NET-сумісних мов.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) Stack Overflow Developer Survey 2023. Stack Overflow. URL: <https://survey.stackoverflow.co/2023/> (дата звернення: 10.05.2024).
- 2) JetBrains. ReSharper. Версія 2024.1. 2024. URL: <https://www.jetbrains.com/resharper> (дата звернення: 15.04.2024).
- 3) Whitfield M. Unitverse. Версія 0.202.0. 2023. URL: <https://github.com/mattwhitfield/Unitverse> (дата звернення: 15.04.2024)
- 4) Osherove R. The Art of Unit Testing. 2nd ed. Shelter Island : Manning Publications, 2014. 294 с.
- 5) Beck K. Test-Driven Development: By Example. Boston : Addison Wesley, 2002. 240 с.
- 6) Forgacs I., Kovács A. Modern Software Testing Techniques: A Practical Guide for Developers and Testers. New York : Apress, 2023. 284 с.
- 7) Rickard D. Unit Test Boilerplate Generator. URL: <https://github.com/RandomEngy/UnitTestBoilerplateGenerator> (дата звернення: 15.04.2024).
- 8) Lakshman P. Introducing Smart Unit Tests. Microsoft Developer Blogs. URL: <https://devblogs.microsoft.com/devops/introducing-smart-unit-tests/> (дата звернення: 11.05.2024).
- 9) Anaccius P. AI-Powered Unit Tests Generator. URL: <https://www.unit-test.dev/> (дата звернення: 11.05.2024).
- 10) University of Passau. Pynguin. Версія 0.36.0. 2024. URL: <https://github.com/se2p/pynguin> (дата звернення: 11.05.2024).
- 11) Agitar Technologies. AgitarOne. Версія 4.2. URL: <http://www.agitar.com/solutions/products/agitarone.html> (дата звернення: 11.05.2024).
- 12) Strauss D. Getting Started with Visual Studio 2022: Learning and Implementing New Features. 2-ге вид. Uitenhage : Apress, 2022. 332 с.

- 13) Verma R. Visual Studio Extensibility Development: Extending Visual Studio IDE for Productivity, Quality, Tooling, Analysis, and Artificial Intelligence. 2-ге вид. Hyderabad : Apress, 2023. 468 с.
- 14) Nesteruk D. Rider Succinctly. 2018. 83 с.
- 15) Johnson B. Visual Studio Code: End-to-End Editing and Debugging Tools for Web Developers. Indianapolis : Wiley, 2019. 192 с.
- 16) ECMA-335. Common Language Infrastructure (CLI). Чинний від 2012-06-01. Вид. офіц. Geneva : Ecma International, 2012. 574 с.
- 17) ECMA-334. C# language specification. Чинний від 2023-12-01. Вид. офіц. Geneva : Ecma International, 2023. 699 с.
- 18) Bock J. .NET Development using the Compiler API. Shakopee : Apress, 2016. 175 с.
- 19) Vasani M. Roslyn Cookbook. Birmingham : Packt Publishing, 2017. 475 с.
- 20) Del Sole A. Roslyn Succinctly. 2016. 127 с.
- 21) Mono Project. Mono. URL: <https://github.com/mono/mono> (дата звернення: 16.04.2024).
- 22) Mutel A. Scriban. Версія 5.10.0. 2024. URL: <https://github.com/scriban/scriban> (дата звернення: 16.05.2024)
- 23) Formaniuk O. Handlebars.Net. Версія 2.1.6. 2024. URL: <https://github.com/Handlebars-Net/Handlebars.Net> (дата звернення: 16.05.2024)
- 24) Chadwick J. Programming Razor. Sebastopol : Oreilly & Associates, 2011. 99 с.
- 25) McCabe T., Watson A. Software Complexity. The Journal of Defense Software Engineering. 1994. Т. 7, № 12. С. 5–9.
- 26) Chidamber S., Kemerer C. A metrics suite for object oriented design. IEEE Transactions on Software Engineering. 1994. Т. 20, № 6. С. 476–493.
- 27) Microsoft. Visual Studio Marketplace. URL: <https://marketplace.visualstudio.com/> (дата звернення: 16.05.2024).

ДОДАТКИ

Додаток А. Звіт подібності Unicheck



Ім'я користувача:
Лісовиченко Олег Іванович

Дата перевірки:
01.06.2024 18:00:20 EEST

Дата звіту:
01.06.2024 18:03:47 EEST

ID перевірки:
1016308153

Тип перевірки:
Doc vs Internet + Library

ID користувача:
76913

Назва документа: ІП-01_Коваленко_ПЗ

Кількість сторінок: 87 Кількість слів: 14493 Кількість символів: 112212 Розмір файлу: 2.25 MB ID файлу: 1016104620

Виявлено модифікації тексту (можуть впливати на відсоток схожості)

3.81%
Схожість

Найбільша схожість: 2.48% з джерелом з Бібліотеки (ID файлу: 1016099048)

0.78% Джерела з Інтернету	71		Сторінка 89
3.75% Джерела з Бібліотеки	149		Сторінка 89

0% Цитат

- Вилучення цитат вимкнене
- Вилучення списку бібліографічних посилань вимкнене

0%
Вилучень

Немає вилучених джерел

Модифікації

Виявлено модифікації тексту. Детальна інформація доступна в онлайн-звіті.

Підозріле форматування 16 сторінок

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗШИРЕННЯ VISUAL STUDIO ДЛЯ АВТОМАТИЧНОЇ ГЕНЕРАЦІЇ
ЮНІТ-ТЕСТІВ**

Текст програми

КПІ.ПІ-0114.045300.03.12

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Микита КОВАЛЕНКО

Київ – 2024

Посилання на репозиторій з повним текстом програмного коду

<https://github.com/xkarlin/NetTestX>

Файл GenerateTestProjectCommand.cs

Реалізація функціональної вимоги FR-1 (Створення тестового проєкту)

```
namespace NetTestX.VSIX.Commands;

/// <summary>
/// Command that generates testing projects, uses <see
cref="GenerateTestProjectCommandHandler"/>
/// </summary>
[Command(PackageIds.GenerateTestProjectCommand)]
internal class GenerateTestProjectCommand : BaseCommand<GenerateTestProjectCommand,
GenerateTestProjectCommandHandler>
{
    protected override GenerateTestProjectCommandHandler CreateHandler() =>
new(DTE);
}
```

Файл GenerateTestProjectCommandHandler.cs

Реалізація функціональної вимоги FR-1 (Створення тестового проєкту)

```
namespace NetTestX.VSIX.Commands.Handlers;

/// <summary>
/// Handler for generating testing projects
/// </summary>
public class GenerateTestProjectCommandHandler(DTE2 dte) : ICommandHandler
{
    public async Task ExecuteAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        TestProjectFactoryContext context = new()
        {
            DTE = dte,
            Project = dte.GetSelectedProjectFromSolutionExplorer()
        };

        await TestProjectUtility.CreateTestProjectFromViewAsync(context);
    }
}
```

Файл TestProjectUtility.cs

Реалізація функціональної вимоги FR-1 (Створення тестового проєкту)

```
namespace NetTestX.VSIX.Projects;

/// <summary>
/// Helper class used to generate test projects
/// </summary>
public static class TestProjectUtility
{
    private const string INTERNALS_VISIBLE_TO_ITEM_NAME = "InternalsVisibleTo";

    /// <summary>
```

```

    /// Generate a test <see cref="DTEProject"/> from the project creation view
    (<see cref="GenerateTestProjectView"/>)
    /// </summary>
    public static async Task<DTEProject>
    CreateTestProjectFromViewAsync(TestProjectFactoryContext context)
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        var testProjectFactory = await TestProjectFactory.CreateAsync(context);

        GenerateTestProjectModel model = new()
        {
            ProjectName = testProjectFactory.Options.ProjectName,
            ProjectDirectory = testProjectFactory.Options.ProjectDirectory,
            TestFramework = testProjectFactory.Options.TestFramework,
            MockingLibrary = testProjectFactory.Options.MockingLibrary,
            GenerateInternalsVisibleTo = false
        };

        GenerateTestProjectView view = new(model);
        bool? result = view.ShowDialog();

        if (result != true)
            return null;

        if (model.GenerateInternalsVisibleTo)
            AddInternalsVisibleTo(context.Project, model.ProjectName);

        testProjectFactory.Options.ProjectName = model.ProjectName;
        testProjectFactory.Options.ProjectDirectory = model.ProjectDirectory;
        testProjectFactory.Options.TestFramework = model.TestFramework;
        testProjectFactory.Options.MockingLibrary = model.MockingLibrary;

        return await testProjectFactory.CreateTestProjectAsync();
    }

    /// <summary>
    /// Add the InternalsVisibleTo <paramref name="visibleTo"/> item to the provided
    <paramref name="sourceProject"/>
    /// </summary>
    public static void AddInternalsVisibleTo(DTEProject sourceProject, string
    visibleTo)
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        CodeProject codeProject = new(sourceProject.FileName);

        if (codeProject.GetItems(INTERNALIS_VISIBLE_TO_ITEM_NAME).Any(x => x.Include
        == visibleTo))
            return;

        codeProject.AddItem(INTERNALIS_VISIBLE_TO_ITEM_NAME, visibleTo);
        codeProject.Save();
    }
}

```

Файл TestProjectFactory.cs

Реалізація функціональної вимоги FR-1 (Створення тестового проєкту)

```

namespace NetTestX.VSIX.Projects;

/// <summary>
/// Factory used to create test <see cref="DTEProject"/>s
/// </summary>
public class TestProjectFactory

```

```

{
    private readonly TestProjectFactoryContext _context;

    /// <summary>
    /// Options used by this factory during test project generation
    /// </summary>
    public required TestProjectFactoryOptions Options { get; init; }

    private TestProjectFactory(TestProjectFactoryContext context)
    {
        _context = context;
    }

    /// <summary>
    /// Create the testing <see cref="DTEProject"/> using the provided <see
    cref="Options"/>
    /// </summary>
    public async Task<DTEProject> CreateTestProjectAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        string originalProjectPath = _context.Project.FileName;

        string testProjectPath =
            $"{Options.ProjectDirectory}/{Options.ProjectName}/{Options.ProjectName}.csproj";

        var workspace = CodeWorkspace.Open(_context.DTE.Solution.FileName);

        TestProjectCreationContext ctx = new()
        {
            ProjectFilePath = testProjectPath,
            OriginalProjectPath = originalProjectPath,
            ProjectName = Options.ProjectName,
            MockingLibrary = Options.MockingLibrary,
            TestFramework = Options.TestFramework
        };

        await workspace.CreateTestProjectAsync(ctx, () =>
            SaveSolutionAsync(_context, testProjectPath));

        var targetProject =
            _context.DTE.Solution.FindSolutionProject(Options.ProjectName);

        return targetProject;
    }

    /// <summary>
    /// Create an instance of this factory using the passed <paramref
    name="context"/>
    /// </summary>
    public static async Task<TestProjectFactory>
        CreateAsync(TestProjectFactoryContext context)
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        string originalProjectPath = context.Project.FileName;
        string originalProjectName = context.Project.Name;

        return new(context)
        {
            Options = new()
            {
                ProjectName = $"{originalProjectName}.Tests",
                ProjectDirectory =
                    Path.GetDirectoryName(Path.GetDirectoryName(originalProjectPath)),
                TestFramework = TestFramework.NUnit,
            }
        };
    }
}

```

```

        MockingLibrary = MockingLibrary.NSubstitute
    }
};
}

private static async Task SaveSolutionAsync(TestProjectFactoryContext context,
string testProjectPath)
{
    await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

    string solutionPath = context.DTE.Solution.FullName;

    var project = context.DTE.Solution.AddFromFile(testProjectPath);
    context.DTE.Solution.SaveAs(context.DTE.Solution.FileName);

    if (context.SaveCallback is { } sc)
        await sc.Invoke(project);

    _ = ThreadHelper.JoinableTaskFactory.RunAsync(async () =>
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        using WaitDialogMessageLogger messageLogger = new("Creating Test
Project");

        messageLogger.LogMessage("Waiting for project initialization...");

        context.DTE.Solution.Close();
        context.DTE.Solution.Open(solutionPath);
    });
}
}

```

Файл GenerateTestProjectView.xaml

Реалізація функціональної вимоги FR-2 (Перегляд сторінки створення тестового проєкту)

```

<Window x:Class="NetTestX.VSIX.UI.Views.GenerateTestProjectView"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:toolkit="clr-
namespace:Community.VisualStudio.Toolkit;assembly=Community.VisualStudio.Toolkit"
    toolkit:Themes.UseVsTheme="True"
    mc:Ignorable="d"
    d:DesignHeight="450" d:DesignWidth="800"
    Title="NetTestX - Generate Test Project">
    <StackPanel>
        <TextBlock Text="Generate Test Project" FontSize="24" Padding="20 5 20 5" />

        <Separator />

        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="20" />
                <ColumnDefinition Width="2*" />
                <ColumnDefinition Width="6*" />
                <ColumnDefinition Width="1*" />
                <ColumnDefinition Width="20" />
            </Grid.ColumnDefinitions>

            <Grid.RowDefinitions>
                <RowDefinition Height="1*" />
            </Grid.RowDefinitions>
        </Grid>
    </StackPanel>
</Window>

```

```

        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
    </Grid.RowDefinitions>

    <TextBlock Grid.Row="0" Grid.Column="1" Text="Project Name"
FontSize="16" Padding="0 10 0 10" />
    <TextBox Name="ProjectNameInput" Grid.Row="0" Grid.Column="2"
Text="{Binding ProjectName, UpdateSourceTrigger=PropertyChanged}" FontSize="16"
Margin="0 5 0 5" />

    <TextBlock Grid.Row="1" Grid.Column="1" Text="Project Directory"
FontSize="16" Padding="0 10 0 10" />
    <TextBox Name="ProjectDirectoryInput" Grid.Row="1" Grid.Column="2"
Text="{Binding ProjectDirectory, Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}"
FontSize="16" Margin="0 5 0 5" />
    <Button Grid.Row="1" Grid.Column="3" Content="..."
Click="ProjectDirectoryButton_Click" FontSize="16" Margin="10 5 10 5" />

    <TextBlock Grid.Row="2" Grid.Column="1" Text="Test Framework"
FontSize="16" Padding="0 10 0 10" />
    <ComboBox Name="TestFrameworkInput" Grid.Row="2" Grid.Column="2"
SelectedIndex="0" SelectedValue="{Binding TestFramework}"
SelectedValuePath="Content" Margin="0 5 0 5">
        <ComboBoxItem Content="XUnit" />
        <ComboBoxItem Content="NUnit" />
        <ComboBoxItem Content="MSTest" />
    </ComboBox>

    <TextBlock Grid.Row="3" Grid.Column="1" Text="Mocking Library"
FontSize="16" Padding="0 10 0 10" />
    <ComboBox Name="MockingLibraryInput" Grid.Row="3" Grid.Column="2"
SelectedIndex="0" SelectedValue="{Binding MockingLibrary}"
SelectedValuePath="Content" Margin="0 5 0 5">
        <ComboBoxItem Content="NSubstitute" />
        <ComboBoxItem Content="FakeItEasy" />
        <ComboBoxItem Content="Moq" />
    </ComboBox>

    <TextBlock Grid.Row="4" Grid.Column="1" Text="Set InternalsVisibleTo"
FontSize="16" Padding="0 10 0 10" />
    <CheckBox Name="GenerateInternalsVisibleToInput" Grid.Row="4"
Grid.Column="2" IsChecked="{Binding GenerateInternalsVisibleTo}" Margin="0 5 0 5" />
</Grid>

    <Button Name="ContinueButton" HorizontalAlignment="Right" Content="Continue"
Click="ContinueButton_Click" FontSize="16" Margin="0 10 25 0" Padding="10 5 10 5" />
</StackPanel>
</Window>

```

Файл GenerateTestProjectView.xaml.cs

Реалізація функціональної вимоги FR-2 (Перегляд сторінки створення тестового проєкту)

```

namespace NetTestX.VSIX.UI.Views;

public partial class GenerateTestProjectView
{
    private static readonly IReadOnlyDictionary<string, IValidation> _validators =
new Dictionary<string, IValidation>
    {
        { nameof(GenerateTestProjectViewModel.ProjectName), new
ProjectNameValidation() }
    }
}

```

```

};

private readonly IReadOnlyDictionary<string, Control> _propertyControls;

private readonly HashSet<string> _invalidProperties = [];

private readonly GenerateTestProjectViewModel _viewModel;

public GenerateTestProjectView(GenerateTestProjectModel model)
{
    DataContext = _viewModel = new(model);

    InitializeComponent();

    _propertyControls = new Dictionary<string, Control>
    {
        { nameof(GenerateTestProjectViewModel.ProjectName), ProjectNameInput }
    };

    _viewModel.PropertyChanged += ValidateViewModelOnPropertyChanged;
}

private void ValidateViewModelOnPropertyChanged(object sender,
PropertyChangedEventArgs args)
{
    if (!_validators.TryGetValue(args.PropertyName, out var validation))
        return;

    object value =
typeof(GenerateTestProjectViewModel).GetProperty(args.PropertyName)?.GetValue(_viewModel);

    var element = _propertyControls[args.PropertyName];

    if (validation.Validate(value))
    {
        _invalidProperties.Remove(args.PropertyName);

        element.BorderBrush = new SolidColorBrush(Color.FromArgb(0xFF, 0x99,
0x99, 0x99));
    }
    else
    {
        _invalidProperties.Add(args.PropertyName);

        element.BorderBrush = new SolidColorBrush(Colors.Red);
    }

    ContinueButton.IsEnabled = _invalidProperties.Count == 0;
}

private void ProjectDirectoryButton_Click(object sender, RoutedEventArgs e)
{
    using var dialog = new FolderBrowserDialog();
    var result = dialog.ShowDialog();

    if (result == System.Windows.Forms.DialogResult.OK)
        _viewModel.ProjectDirectory = dialog.SelectedPath;
}

private void ContinueButton_Click(object sender, RoutedEventArgs e)
{
    DialogResult = true;
}
}

```


Файл **ProjectNameValidation.cs**

Реалізація функціональної вимоги FR-3 (Перевірка валідності назви тестового проєкту)

```
using System;

namespace NetTestX.VSIX.Options.Validation;

/// <summary>
/// Validation for .NET project names
/// </summary>
public class ProjectNameValidation : Validation<string>
{
    private const string FORBIDDEN_CHARS = "(/*+~_@&$#%){} ";

    public override bool Validate(string value)
    {
        if (string.IsNullOrEmpty(value))
            return false;

        return value.AsSpan().IndexOfAny(FORBIDDEN_CHARS.AsSpan()) == -1;
    }
}
```

Файл **GenerateTestsCommand.cs**

Реалізація функціональної вимоги FR-4 (Генерація юніт-тестів з вікна дослідника рішень), FR-5 (Перевірка обраних елементів у вікні дослідника рішень), FR-6 (Обрання тестового проєкту)

```
namespace NetTestX.VSIX.Commands;

/// <summary>
/// Command that generates tests the fast way using default options, uses <see cref="GenerateTestsCommandHandler"/>
/// </summary>
[Command(PackageIds.GenerateTestsCommand)]
internal sealed class GenerateTestsCommand :
    BaseDynamicCommand<GenerateTestsCommand, GenerateTestsCommandHandler, CodeProject>
{
    protected override GenerateTestsCommandHandler CreateHandler(CodeProject item)
    => new(DTE, item);

    protected override void BeforeQueryStatus(OleMenuCommand command, EventArgs e,
        CodeProject project)
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        bool visible = ShouldCommandBeVisible();

        Command.Visible = Command.Enabled = visible;

        command.Text = project is null ? "Generate Test Project..." : $"In
{project.Name}";
    }

    protected override IReadOnlyList<CodeProject> GetItems()
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        List<CodeProject> projects = [null];
    }
}
```

```

        if (ShouldCommandBeVisible())
        {
            var solution = CodeWorkspace.Open(DTE.Solution.FileName);

            var testProjects = solution.GetTestProjects().ToArray();
            projects.AddRange(testProjects);
        }

        return projects;
    }

    private bool ShouldCommandBeVisible()
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        var selectedItems =
            (UIHierarchyItem[])DTE.ToolWindows.SolutionExplorer.SelectedItems;

        if (selectedItems.Length == 0)
            return false;

        return selectedItems.All(x =>
        {
            ThreadHelper.ThrowIfNotOnUIThread();
            return x.Object is ProjectItem item &&
                item.FileNames[0].EndsWith(SourceFileExtensions.CSHARP_DOT);
        });
    }
}

```

Файл `GenerateTestsCommandHandler.cs`

Реалізація функціональної вимоги FR-4 (Генерація юніт-тестів з вікна дослідника рішень)

```

namespace NetTestX.VSIX.Commands.Handlers;

/// <summary>
/// Handler for generating tests the fast way using default options
/// </summary>
public class GenerateTestsCommandHandler(DTE2 dte, CodeProject testProject) :
    ICommandHandler
{
    public async Task ExecuteAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        var selectedItems = dte.GetSelectedItemsFromSolutionExplorer();

        ProjectItem[] items = selectedItems.Select(x =>
        {
            ThreadHelper.ThrowIfNotOnUIThread();
            return (ProjectItem)x.Object;
        }).ToArray();

        var projectItem = (ProjectItem)selectedItems[0].Object;
        var sourceProject = projectItem.ContainingProject;
        var roslynProject = await projectItem.FindRoslynProjectAsync();
        var compilation = await roslynProject.GetCompilationAsync();

        if (testProject is not null)
        {
            var dteProject = dte.Solution.FindSolutionProject(testProject.Name);
            await GenerateSourceFilesAsync(dteProject);
            return;
        }
    }
}

```

```

    }

    TestProjectFactoryContext context = new()
    {
        DTE = dte,
        Project = ((ProjectItem)selectedItems[0].Object).ContainingProject,
        SaveCallback = GenerateSourceFilesAsync
    };

    await TestProjectUtility.CreateTestProjectFromViewAsync(context);

    async Task GenerateSourceFilesAsync(DTEProject targetProject)
    {
        var advancedOptions = await AdvancedOptions.GetLiveInstanceAsync();

        foreach (var item in items)
        {
            string sourceFileName = item.FileNames[0];
            var syntaxTree = compilation?.SyntaxTrees.FirstOrDefault(x =>
x.FilePath == sourceFileName);
            var syntaxTreeRoot = await syntaxTree.GetRootAsync();

            var availableTypeSymbols =
SymbolHelper.GetAvailableTypeSymbolsForGeneration(syntaxTreeRoot,
compilation).ToImmutableArray();

            bool shouldShowWarning = advancedOptions.ShowMultipleTypeWarning &&
availableTypeSymbols.Length > 1;

            if (shouldShowWarning &&
!SymbolHelper.ShowMultipleTypesWarning(sourceFileName, availableTypeSymbols))
                continue;

            foreach (var typeSymbol in availableTypeSymbols)
            {
                var codeCoordinator = await
TestSourceCodeCoordinator.CreateAsync(typeSymbol, sourceProject);
                await codeCoordinator.LoadSourceCodeAsync(targetProject);
            }
        }
    }
}

```

Файл TestSourceCodeCoordinator.cs

Реалізація функціональної вимоги FR-4 (Генерація юніт-тестів з вікна дослідника рішень), FR-8 (Генерація юніт-тестів з вікна редактора коду), FR-10 (Генерація юніт-тестів розширено з вікна дослідника рішень), FR-16 (Генерація юніт-тестів розширено з вікна редактора коду)

```

namespace NetTestX.VSIX.Code;

/// <summary>
/// Class used to generate source code files inside <see cref="DTEProject"/>s
/// </summary>
public class TestSourceCodeCoordinator
{
    private readonly DTEProject _sourceProject;

    /// <summary>
    /// Options used by this coordinator
    /// </summary>
    public required TestSourceCodeCoordinatorOptions Options { get; init; }
}

```

```

    /// <summary>
    /// Unit test driver builder used by this coordinator
    /// </summary>
    public required UnitTestGeneratorDriver.Builder DriverBuilder { get; init; }

    private TestSourceCodeCoordinator(DTEProject sourceProject)
    {
        _sourceProject = sourceProject;
    }

    /// <summary>
    /// Generate and add source file with unit tests to the given <paramref
name="targetProject"/>
    /// </summary>
    public async Task LoadSourceCodeAsync(DTEProject targetProject)
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        bool typeInternal = DriverBuilder.Type.GetEffectiveAccessibility() ==
Accessibility.Internal;
        bool anySymbolInternal = DriverBuilder.TestMethodMap.Where(x =>
x.Value).Any(x => x.Key.Symbol.GetEffectiveAccessibility() ==
Accessibility.Internal);

        if (typeInternal || anySymbolInternal)
            TestProjectUtility.AddInternalsVisibleTo(_sourceProject,
targetProject.Name);

        CodeProject codeProject = new(targetProject.FileName);

        var driver = GetGeneratorDriver(codeProject);

        string testSource = await driver.GenerateTestClassSourceAsync();

        string targetFilePath = GetTargetFilePath(targetProject);

        await AddSourceFileToProjectAsync(targetProject, testSource,
targetFilePath);
    }

    /// <summary>
    /// Create an instance of <see cref="TestSourceCodeCoordinator"/> for the given
<paramref name="type"/> and <paramref name="sourceProject"/> that contains it
    /// </summary>
    public static async Task<TestSourceCodeCoordinator> CreateAsync(INamedTypeSymbol
type, DTEProject sourceProject, IDiagnosticReporter reporter = null)
    {
        RoslynProject roslynProject = await sourceProject.FindRoslynProjectAsync();
        Compilation compilation = await roslynProject.GetCompilationAsync();

        var generalOptions = await GeneralOptions.GetLiveInstanceAsync();

        var advancedGeneratorOptions = await
OptionsUtility.GetAdvancedGeneratorOptionsAsync();

        TestSourceCodeCoordinator coordinator = new(sourceProject)
        {
            Options = new()
            {
                TestFileName =
OptionResolverHelper.ResolveGeneralOption(generalOptions.TestFileName, type)
            },
            DriverBuilder = UnitTestGeneratorDriver.CreateBuilder(type, compilation,
advancedGeneratorOptions, reporter)
        };
    }

```

```

        coordinator.DriverBuilder.TestClassName =
OptionResolverHelper.ResolveGeneralOption(generalOptions.TestClassName, type);
        coordinator.DriverBuilder.TestClassNamespace =
OptionResolverHelper.ResolveGeneralOption(generalOptions.TestClassNamespace, type);
        coordinator.DriverBuilder.AdvancedOptions = advancedGeneratorOptions;

        return coordinator;
    }

    private UnitTestGeneratorDriver GetGeneratorDriver(CodeProject project)
    {
        DriverBuilder.TestFramework = project.GetProjectTestFramework();
        DriverBuilder.MockingLibrary = project.GetProjectMockingLibrary();

        UnitTestGeneratorDriver driver = DriverBuilder.Build();
        return driver;
    }

    private async Task AddSourceFileToProjectAsync(DTEProject project, string
sourceText, string fileName)
    {
        MemoryStream ms = new(Encoding.UTF8.GetBytes(sourceText));

        var targetFolder = Path.GetDirectoryName(fileName);
        Directory.CreateDirectory(targetFolder);

        await project.CreateProjectFileAsync(ms, fileName);
    }

    private string GetTargetFilePath(DTEProject targetProject)
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        string testSourceFileName =
${Options.TestFileName}.{SourceFileExtensions.CSHARP}";

        string sourceFileDirectory =
Path.GetDirectoryName(DriverBuilder.Type.DeclaringSyntaxReferences[0].SyntaxTree.Fil
ePath);
        string sourceProjectDirectory =
Path.GetDirectoryName(_sourceProject.FileName);
        string targetProjectDirectory =
Path.GetDirectoryName(targetProject.FileName);

        string targetFileDirectory =
TestSourceCodeUtility.CopyRelativePath(sourceFileDirectory, sourceProjectDirectory,
targetProjectDirectory);

        return Path.Combine(targetFileDirectory, testSourceFileName);
    }
}

```

Файл SymbolHelper.cs

Реалізація функціональної вимоги FR-7 (Попередження про генерацію кількох типів)

```

namespace NetTestX.VSIX.Commands.Helpers;

/// <summary>
/// Helper class for <see cref="ISymbol"/>s and derived types
/// </summary>
public static class SymbolHelper
{

```

```

    /// <summary>
    /// Find all available <see cref="INamedTypeSymbol"/> that can be generated from
    the given <paramref name="node"/>
    /// </summary>
    public static IEnumerable<INamedTypeSymbol>
    GetAvailableTypeSymbolsForGeneration(SyntaxNode node, Compilation compilation)
    {
        var semanticModel = compilation.GetSemanticModel(node.SyntaxTree);

        var typeDeclarations = node
            .DescendantNodesAndSelf(x => x is not TypeDeclarationSyntax)
            .Where(x => (SyntaxKind)x.RawKind is
                SyntaxKind.ClassDeclaration or
                SyntaxKind.StructDeclaration or
                SyntaxKind.RecordDeclaration or
                SyntaxKind.RecordStructDeclaration);

        var typeSymbols = typeDeclarations
            .Select(x => semanticModel.GetDeclaredSymbol((TypeDeclarationSyntax)x))
            .Where(CanGenerateTestsForTypeSymbol);

        return typeSymbols;
    }

    /// <summary>
    /// Whether the tests can be generated for the given <paramref
    name="typeSymbol"/>
    /// </summary>
    public static bool CanGenerateTestsForTypeSymbol(INamedTypeSymbol typeSymbol)
    {
        var generalOptions = GeneralOptions.Instance;

        Accessibility targetAccessibility = generalOptions.TestInternalMembers ?
        Accessibility.Internal : Accessibility.Public;

        if (typeSymbol.GetEffectiveAccessibility() < targetAccessibility)
            return false;

        if (typeSymbol.TypeKind is not TypeKind.Class and not TypeKind.Struct)
            return false;

        if (typeSymbol.IsAbstract)
            return false;

        return true;
    }

    /// <summary>
    /// Show the warning that multiple test classes will be generated for the given
    file
    /// </summary>
    public static bool ShowMultipleTypesWarning(string fileName,
    ImmutableArray<INamedTypeSymbol> availableTypes)
    {
        string typesString = string.Join("\n", availableTypes.Select(x => x.Name));
        return VS.MessageBox.ShowConfirm("NetTestX - multiple types found", $"The
        selected file {fileName} contains multiple types:\n\n{typesString}\n\nWould you like
        to proceed and generate tests for all these types?");
    }
}

```

Файл GenerateTestsEditorCommand.cs

Реалізація функціональної вимоги FR-8 (Генерація юніт-тестів з вікна редактора коду), FR-9 (Перевірка обраного символу у вікні дослідника рішень)

```
namespace NetTestX.VSIX.Commands;

/// <summary>
/// Command that generates tests the fast way from the editor window, using default
/// options, uses <see cref="GenerateTestsEditorCommandHandler"/>
/// </summary>
[Command(PackageIds.GenerateTestsEditorCommand)]
public class GenerateTestsEditorCommand :
BaseDynamicCommand<GenerateTestsEditorCommand, GenerateTestsEditorCommandHandler,
CodeProject>
{
    private INamedTypeSymbol _activeTypeSymbol;

    protected override GenerateTestsEditorCommandHandler CreateHandler(CodeProject
item) => new(DTE, _activeTypeSymbol, item);

    protected override void BeforeQueryStatus(OleMenuCommand command, EventArgs e,
CodeProject project)
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        var (visible, enabled) = ShouldCommandBeVisible();

        Command.Visible = visible;
        Command.Enabled = enabled;

        command.Text = project is null ? "Generate Test Project..." : $"In
{project.Name}";
    }

    protected override IReadOnlyList<CodeProject> GetItems()
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        List<CodeProject> projects = [null];

        var (_, enabled) = ShouldCommandBeVisible();

        if (enabled)
        {
            var solution = CodeWorkspace.Open(DTE.Solution.FileName);

            var testProjects = solution.GetTestProjects().ToArray();
            projects.AddRange(testProjects);
        }

        return projects;
    }

    private (bool Visible, bool Enabled) ShouldCommandBeVisible()
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        var textView = Package.GetActiveTextView();

        if (!textView.TryGetActiveTypeSymbol(out _activeTypeSymbol))
            return (false, false);

        if (SymbolHelper.CanGenerateTestsForTypeSymbol(_activeTypeSymbol))
            return (true, true);
    }
}
```

```

        return (true, false);
    }
}

```

Файл **GenerateTestsEditorCommandHandler.cs**

Реалізація функціональної вимоги FR-8 (Генерація юніт-тестів з вікна редактора коду)

```

namespace NetTestX.VSIX.Commands.Handlers;

/// <summary>
/// Handler for generating tests the fast way from the editor window, using default
options
/// </summary>
public class GenerateTestsEditorCommandHandler(DTE2 dte, INamedTypeSymbol
typeSymbol, CodeProject testProject) : ICommandHandler
{
    public async Task ExecuteAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        var sourceProject = dte.ActiveDocument.ProjectItem.ContainingProject;
        var targetProject = await GetTargetProjectAsync();

        var codeCoordinator = await
TestSourceCodeCoordinator.CreateAsync(typeSymbol, sourceProject);

        await codeCoordinator.LoadSourceCodeAsync(targetProject);
    }

    private async Task<DTEProject> GetTargetProjectAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        if (testProject is not null)
            return dte.Solution.FindSolutionProject(testProject.Name);

        TestProjectFactoryContext context = new()
        {
            DTE = dte,
            Project = dte.ActiveDocument.ProjectItem.ContainingProject
        };

        return await TestProjectUtility.CreateTestProjectFromViewAsync(context);
    }
}

```

Файл **GenerateTestsAdvancedCommand.cs**

Реалізація функціональної вимоги FR-10 (Генерація юніт-тестів розширено з вікна дослідника рішень)

```

namespace NetTestX.VSIX.Commands;

/// <summary>
/// Command that generates tests advanced, uses <see
cref="GenerateTestsAdvancedCommandHandler"/>
/// </summary>
[Command(PackageIds.GenerateTestsAdvancedCommand)]
internal sealed class GenerateTestsAdvancedCommand :
BaseCommand<GenerateTestsAdvancedCommand, GenerateTestsAdvancedCommandHandler>
{

```



```

        protected override GenerateTestsAdvancedCommandHandler CreateHandler() =>
        new(DTE);

        protected override void BeforeQueryStatus(EventArgs e)
        {
            ThreadHelper.ThrowIfNotOnUIThread();

            var (visible, enabled) = ShouldCommandBeVisible();

            Command.Visible = visible;
            Command.Enabled = enabled;
        }

        private (bool Visible, bool Enabled) ShouldCommandBeVisible()
        {
            ThreadHelper.ThrowIfNotOnUIThread();

            var selectedItems =
            (UIHierarchyItem[])DTE.ToolWindows.SolutionExplorer.SelectedItems;

            bool itemsValid = selectedItems.All(x =>
            {
                ThreadHelper.ThrowIfNotOnUIThread();
                return x.Object is ProjectItem item &&
                item.FileNames[0].EndsWith(SourceFileExtensions.CSHARP_DOT);
            });

            if (selectedItems.Length == 0 || !itemsValid)
                return (false, false);

            var workspace = CodeWorkspace.Open(DTE.Solution.FileName);

            if (!workspace.GetTestProjects().Any())
                return (true, false);

            return (true, true);
        }
    }
}

```

Файл **GenerateTestsAdvancedCommandHandler.cs**

Реалізація функціональної вимоги FR-10 (Генерація юніт-тестів розширено з вікна дослідника рішень)

```

namespace NetTestX.VSIX.Commands.Handlers;

/// <summary>
/// Handler for generating tests the advanced way using a dedicated window
/// </summary>
public class GenerateTestsAdvancedCommandHandler(DTE2 dte) : ICommandHandler
{
    public async Task ExecuteAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        var projectItems = dte.GetSelectedItemsFromSolutionExplorer();
        var sourceProject = ((ProjectItem)projectItems[0].Object).ContainingProject;
        var roslynProject = await sourceProject.FindRoslynProjectAsync();

        var advancedOptions = await AdvancedOptions.GetLiveInstanceAsync();

        foreach (var projectItem in projectItems)
        {
            string sourceFileName = ((ProjectItem)projectItem.Object).FileNames[0];
            var compilation = await roslynProject.GetCompilationAsync();

```

```

        var syntaxTree = compilation?.SyntaxTrees.FirstOrDefault(x => x.FilePath
== sourceFileName);
        var syntaxTreeRoot = await syntaxTree.GetRootAsync();

        var availableTypeSymbols =
SymbolHelper.GetAvailableTypeSymbolsForGeneration(syntaxTreeRoot,
compilation).ToImmutableArray();

        bool shouldShowWarning = advancedOptions.ShowMultipleTypeWarning &&
availableTypeSymbols.Length > 1;

        if (shouldShowWarning &&
!SymbolHelper.ShowMultipleTypesWarning(sourceFileName, availableTypeSymbols))
            continue;

        foreach (var typeSymbol in availableTypeSymbols)
            await TestSourceCodeUtility.LoadSourceCodeFromAdvancedViewAsync(dte,
sourceProject, typeSymbol);
    }
}

```

Файл TestSourceCodeUtility.cs

Реалізація функціональної вимоги FR-10 (Генерація юніт-тестів розширено з вікна дослідника рішень)

```

namespace NetTestX.VSIX.Code;

/// <summary>
/// Helper class used for test source files generation
/// </summary>
public static class TestSourceCodeUtility
{
    /// <summary>
    /// Load the source code from advanced view (<see
    cref="GenerateTestsAdvancedView"/>)
    /// </summary>
    public static async Task LoadSourceCodeFromAdvancedViewAsync(DTE2 dte,
DTEProject sourceProject, INamedTypeSymbol typeSymbol)
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        TestSourceCodeDiagnosticReporter reporter = new();
        var codeCoordinator = await
TestSourceCodeCoordinator.CreateAsync(typeSymbol, sourceProject, reporter);

        GenerateTestsAdvancedModel model = new()
        {
            TestFileName = codeCoordinator.Options.TestFileName,
            TestClassName = codeCoordinator.DriverBuilder.TestClassName,
            TestClassNamespace = codeCoordinator.DriverBuilder.TestClassNamespace,
            TestMethodMap = new(codeCoordinator.DriverBuilder.TestMethodMap),
            AdvancedOptions = codeCoordinator.DriverBuilder.AdvancedOptions
        };

        var workspace = CodeWorkspace.Open(dte.Solution.FileName);
        var testProjects = workspace.GetTestProjects();

        GenerateTestsAdvancedView view = new(model, reporter, testProjects.Select(x
=> x.Name));

        bool? result = view.ShowDialog();

        if (result != true)
    }
}

```

```

        return;

codeCoordinator.Options.TestFileName = model.TestFileName;
codeCoordinator.DriverBuilder.TestClassName = model.TestClassName;
codeCoordinator.DriverBuilder.TestClassNamespace = model.TestClassNamespace;
codeCoordinator.DriverBuilder.AdvancedOptions = model.AdvancedOptions;

foreach (var (modelBase, enabled) in model.TestMethodMap)
    codeCoordinator.DriverBuilder.TestMethodMap[modelBase] = enabled;

var targetProject = dte.Solution.FindSolutionProject(model.TestProject);

await codeCoordinator.LoadSourceCodeAsync(targetProject);
}

/// <summary>
/// Rebase the relative path for <paramref name="targetBase"/> same way it's
done with <paramref name="relativeBase"/> and <paramref name="sourcePath"/>
/// </summary>
public static string CopyRelativePath(string sourcePath, string relativeBase,
string targetBase)
{
    if (!sourcePath.EndsWith("/") && !sourcePath.EndsWith("\\\\"))
        sourcePath += "/";

    if (!relativeBase.EndsWith("/") && !relativeBase.EndsWith("\\\\"))
        relativeBase += "/";

    var relativePathUri = new Uri(relativeBase,
UriKind.Absolute).MakeRelativeUri(new Uri(sourcePath));
    string relativePath = relativePathUri.ToString();
    return Path.Combine(targetBase, relativePath);
}
}

```

Файл GenerateTestsAdvancedView.xaml

Реалізація функціональної вимоги FR-11 (Перегляд сторінки розширеної генерації тестів)

```

<Window x:Class="NetTestX.VSIX.UI.Views.GenerateTestsAdvancedView"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
    xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
    xmlns:toolkit="clr-
namespace:Community.VisualStudio.Toolkit;assembly=Community.VisualStudio.Toolkit"
    toolkit:Themes.UseVsTheme="True"
    mc:Ignorable="d"
    d:DesignHeight="450" d:DesignWidth="800"
    Title="NetTestX - Generate Unit Tests Advanced">
    <StackPanel Margin="20 0">
        <TextBlock Text="Generate Unit Tests Advanced" FontSize="24" Padding="0 10"
        />

        <Separator />

        <TextBlock Text="General" FontSize="20" FontWeight="DemiBold" Padding="0 5"
        />

        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="2*" />
                <ColumnDefinition Width="6*" />
            </Grid.ColumnDefinitions>
        </Grid>
    </StackPanel>
</Window>

```

```

        <ColumnDefinition Width="1*" />
    </Grid.ColumnDefinitions>

    <Grid.RowDefinitions>
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
        <RowDefinition Height="1*" />
    </Grid.RowDefinitions>

    <TextBlock Grid.Row="2" Grid.Column="0" Text="Test Project"
FontSize="16" Padding="0 10 0 10" />
    <ComboBox Grid.Row="2" Grid.Column="1" x:Name="TestProjectComboBox"
SelectedIndex="0" SelectedValue="{Binding TestProject}" Margin="0 5 0 5">
    </ComboBox>

    <TextBlock Grid.Row="3" Grid.Column="0" Text="Test File Name"
FontSize="16" Padding="0 10 0 10" />
    <TextBox Name="TestFileNameInput" Grid.Row="3" Grid.Column="1"
FontSize="16" Text="{Binding TestFileName, UpdateSourceTrigger=PropertyChanged}"
Padding="0 10 0 10" />
    <TextBlock Grid.Row="3" Grid.Column="2" Text=".cs" FontSize="16"
Padding="0 10 0 10" />

    <TextBlock Grid.Row="4" Grid.Column="0" Text="Test Class Name"
FontSize="16" Padding="0 10 0 10" />
    <TextBox Name="TestClassNameInput" Grid.Row="4" Grid.Column="1"
FontSize="16" Text="{Binding TestClassName, UpdateSourceTrigger=PropertyChanged}"
Padding="0 10 0 10" />

    <TextBlock Grid.Row="5" Grid.Column="0" Text="Test Class Namespace"
FontSize="16" Padding="0 10 0 10" />
    <TextBox Name="TestClassNamespaceInput" Grid.Row="5" Grid.Column="1"
FontSize="16" Text="{Binding TestClassNamespace,
UpdateSourceTrigger=PropertyChanged}" Padding="0 10 0 10" />
    </Grid>

    <TextBlock Text="Code" FontSize="20" FontWeight="DemiBold" Padding="0 5" />

    <StackPanel x:Name="CodeDiagnosticsPanel">
    </StackPanel>

    <StackPanel>
        <TextBlock Text="Select test methods" FontSize="16"
FontWeight="DemiBold" Padding="0 5" />
        <StackPanel x:Name="TestMethodsPanel">
        </StackPanel>
    </StackPanel>

    <Button Name="ContinueButton" HorizontalAlignment="Right" Content="Continue"
Click="ContinueButton_Click" FontSize="16" Margin="0 10 25 0" Padding="10 5 10 5" />
    </StackPanel>
</Window>

```

Файл **GenerateTestsAdvancedView.xaml.cs**

Реалізація функціональної вимоги FR-11 (Перегляд сторінки розширеної генерації тестів), FR-15 (Обрання окремих юніт-тестів для генерації)

```
namespace NetTestX.VSIX.UI.Views;
```

```
public partial class GenerateTestsAdvancedView
{
```

```

private const int MAX_DIAGNOSTICS_TO_SHOW = 10;

private static readonly IReadOnlyDictionary<string, IValidation> _validators =
new Dictionary<string, IValidation>
{
    { nameof(GenerateTestsAdvancedViewModel.TestClassName), new
TestClassNameValidation() },
    { nameof(GenerateTestsAdvancedViewModel.TestClassNamespace), new
TestClassNamespaceValidation() },
    { nameof(GenerateTestsAdvancedViewModel.TestFileName), new
TestFileNameValidation() }
};

private readonly IReadOnlyDictionary<string, Control> _propertyControls;

private readonly HashSet<string> _invalidProperties = [];

private readonly TestSourceCodeDiagnosticReporter _reporter;

private readonly GenerateTestsAdvancedViewModel _viewModel;

private readonly IReadOnlyList<string> _testProjects;

public GenerateTestsAdvancedView(
    GenerateTestsAdvancedModel model,
    TestSourceCodeDiagnosticReporter reporter,
    IEnumerable<string> testProjects)
{
    _reporter = reporter;
    DataContext = _viewModel = new(model);

    _testProjects = testProjects.ToList();

    InitializeComponent();

    _propertyControls = new Dictionary<string, Control>
    {
        { nameof(GenerateTestsAdvancedViewModel.TestClassName),
TestClassNameInput },
        { nameof(GenerateTestsAdvancedViewModel.TestClassNamespace),
TestClassNamespaceInput },
        { nameof(GenerateTestsAdvancedViewModel.TestFileName), TestFileNameInput
}
    };

    _viewModel.PropertyChanged += ValidateViewModelOnPropertyChanged;
}

private void ValidateViewModelOnPropertyChanged(object sender,
PropertyChangedEventArgs args)
{
    if (!_validators.TryGetValue(args.PropertyName, out var validation))
        return;

    object value =
typeof(GenerateTestsAdvancedViewModel).GetProperty(args.PropertyName)?.GetValue(_vie
wModel);

    var element = _propertyControls[args.PropertyName];

    if (validation.Validate(value))
    {
        _invalidProperties.Remove(args.PropertyName);

        element.BorderBrush = new SolidColorBrush(Color.FromArgb(0xFF, 0x99,
0x99, 0x99));
    }
}

```

```

    }
    else
    {
        _invalidProperties.Add(args.PropertyName);

        element.BorderBrush = new SolidColorBrush(Colors.Red);
    }

    ContinueButton.IsEnabled = _invalidProperties.Count == 0;
}

protected override void OnInitialized(EventArgs e)
{
    base.OnInitialized(e);

    InitializeTestProjectComboBox();
    InitializeDiagnosticsPanel();
    InitializeTestMethodsPanel();
}

private void ContinueButton_Click(object sender, RoutedEventArgs e)
{
    DialogResult = true;
}

private void TestMethodCheckBox_Checked(object sender, RoutedEventArgs e)
{
    var checkBox = (CheckBox)sender;
    var testMethod = (TestMethodModelBase)checkBox.DataContext;

    _viewModel.TestMethodMap[testMethod] = true;
}

private void TestMethodCheckBox_Unchecked(object sender, RoutedEventArgs e)
{
    var checkBox = (CheckBox)sender;
    var testMethod = (TestMethodModelBase)checkBox.DataContext;

    _viewModel.TestMethodMap[testMethod] = false;
}

private void InitializeTestProjectComboBox()
{
    TestProjectComboBox.ItemsSource = _testProjects;
    TestProjectComboBox.SelectedIndex = 0;
}

private void InitializeDiagnosticsPanel()
{
    foreach (var (severity, message) in
        _reporter.Diagnostics.OrderByDescending(x =>
            x.Severity).Take(MAX_DIAGNOSTICS_TO_SHOW))
    {
        TextBlock block = new()
        {
            Text = message,
            FontSize = 16
        };

        CodeDiagnosticsPanel.Children.Add(block);
    }
}

private void InitializeTestMethodsPanel()
{
    if (_viewModel.TestMethodMap.Count == 0)

```

```

    {
        TextBlock noMethodsBlock = new()
        {
            Text = "No test methods available for generation.",
            FontSize = 16
        };

        TestMethodsPanel.Children.Add(noMethodsBlock);
        return;
    }

    var orderedMethods = _viewModel.TestMethodMap
        .OrderBy(x => x.Key.Symbol.Kind)
        .ThenByDescending(x => x.Key.Symbol.GetEffectiveAccessibility());

    foreach (var (testMethod, enabled) in orderedMethods)
    {
        CheckBox box = new()
        {
            Content = testMethod.MethodBodyModel.GetDisplayName(),
            DataContext = testMethod,
            IsChecked = enabled,
            FontSize = 16
        };

        box.Checked += TestMethodCheckBox_Checked;
        box.Unchecked += TestMethodCheckBox_Unchecked;

        TestMethodsPanel.Children.Add(box);
    }
}

```

Файл TestFileNameValidation.cs

Реалізація функціональної вимоги FR-12 (Перевірка валідності назви файлу з тестами)

```

namespace NetTestX.VSIX.Options.Validation;

/// <summary>
/// Validation for Windows file names
/// </summary>
public class TestFileNameValidation : Validation<string>
{
    public override bool Validate(string value) =>
        value.IndexOfAny(Path.GetInvalidFileNameChars()) == -1;
}

```

Файл TestClassNameValidation.cs

Реалізація функціональної вимоги FR-13 (Перевірка валідності назви тестового класу)

```

namespace NetTestX.VSIX.Options.Validation;

/// <summary>
/// Validation for .NET class names
/// </summary>
public class TestClassNameValidation : Validation<string>
{
    public override bool Validate(string value) =>
        SyntaxFacts.IsValidIdentifier(value);
}

```

```
}
```

Файл TestClassNamespaceValidation.cs

Реалізація функціональної вимоги FR-14 (Перевірка валідності назви простору імен тестового класу)

```
namespace NetTestX.VSIX.Options.Validation;

/// <summary>
/// Validation for .NET namespaces
/// </summary>
public class TestClassNamespaceValidation : Validation<string>
{
    public override bool Validate(string value) =>
        Array.TrueForAll(value.Split(['.']), SyntaxFacts.IsValidIdentifier);
}
```

Файл GenerateTestsAdvancedEditorCommand.cs

Реалізація функціональної вимоги FR-16 (Генерація юніт-тестів розширено з вікна редактора коду)

```
namespace NetTestX.VSIX.Commands;

/// <summary>
/// Command that generates tests advanced from the editor window, uses <see
    cref="GenerateTestsAdvancedEditorCommandHandler"/>
/// </summary>
[Command(PackageIds.GenerateTestsAdvancedEditorCommand)]
internal sealed class GenerateTestsAdvancedEditorCommand :
    BaseCommand<GenerateTestsAdvancedEditorCommand,
        GenerateTestsAdvancedEditorCommandHandler>
{
    private INamedTypeSymbol _activeTypeSymbol;

    protected override GenerateTestsAdvancedEditorCommandHandler CreateHandler() =>
        new(DTE, _activeTypeSymbol);

    protected override void BeforeQueryStatus(EventArgs e)
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        var (visible, enabled) = ShouldCommandBeVisible();

        Command.Visible = visible;
        Command.Enabled = enabled;
    }

    private (bool Visible, bool Enabled) ShouldCommandBeVisible()
    {
        ThreadHelper.ThrowIfNotOnUIThread();

        var textView = Package.GetActiveTextView();

        if (!textView.TryGetActiveTypeSymbol(out _activeTypeSymbol))
            return (false, false);

        if (SymbolHelper.CanGenerateTestsForTypeSymbol(_activeTypeSymbol))
            return (true, true);

        return (true, false);
    }
}
```


Файл GenerateTestsAdvancedEditorCommandHandler.cs

Реалізація функціональної вимоги FR-16 (Генерація юніт-тестів розширено з вікна редактора коду)

```
namespace NetTestX.VSIX.Commands.Handlers;

/// <summary>
/// Handler for generating tests the advanced way from the editor window using a
dedicated window
/// </summary>
public class GenerateTestsAdvancedEditorCommandHandler(DTE2 dte, INamedTypeSymbol
typeSymbol) : ICommandHandler
{
    public async Task ExecuteAsync()
    {
        await ThreadHelper.JoinableTaskFactory.SwitchToMainThreadAsync();

        var sourceProject = dte.ActiveDocument.ProjectItem.ContainingProject;

        await TestSourceCodeUtility.LoadSourceCodeFromAdvancedViewAsync(dte,
sourceProject, typeSymbol);
    }
}
```

Файл GeneralOptions.cs

Реалізація функціональної вимоги FR-17 (Зміна налаштувань розширення), FR-18 (Задання шаблону назви тестового файлу), FR-19 (Задання шаблону назви класу з тестами), FR-20 (Задання шаблону назви простору імен класу з тестами), FR-21 (Задання необхідності тестування internal типів)

```
namespace NetTestX.VSIX.Options;

internal partial class OptionsProvider
{
    [ComVisible(true)]
    public class GeneralOptionsPage : BaseOptionPage<GeneralOptions> { }
}

/// <summary>
/// General options contained in the "General" options page inside VS settings
/// </summary>
public class GeneralOptions : BaseOptionModel<GeneralOptions>
{
    [Category("General")]
    [DisplayName("Test Internal Members")]
    [Description("Whether to test internal members of types.")]
    [DefaultValue(false)]
    public bool TestInternalMembers { get; set; } = false;

    [Category("Naming")]
    [DisplayName("Test File Name")]
    [Description("The file name of the test class.")]
    [DefaultValue("{TypeName}Tests")]
    public string TestFileName { get; set; } = "{TypeName}Tests";

    [Category("Naming")]
    [DisplayName("Test Class Name")]
    [Description("The name of the test class.")]
    [DefaultValue("{TypeName}Tests")]
    public string TestClassName { get; set; } = "{TypeName}Tests";

    [Category("Naming")]
```

```

[DisplayName("Test Class Namespace")]
[Description("The namespace of the test class.")]
[DefaultValue("{TypeNamespace}.Tests")]
public string TestClassNamespace { get; set; } = "{TypeNamespace}.Tests";
}

```

Файл CodeOptions.cs

Реалізація функціональної вимоги FR-17 (Зміна налаштувань розширення), FR-22 (Задання необхідності генерації AAA-коментарів у згенерованих тестах), FR-23 (Задання необхідності підстановки аргументів типів для універсальних типів / членів типів)

```

namespace NetTestX.VSIX.Options;

internal partial class OptionsProvider
{
    [ComVisible(true)]
    public class CodeOptionsPage : BaseOptionPage<CodeOptions> { }
}

/// <summary>
/// Code options contained in the "Code" options page inside VS settings
/// </summary>
public class CodeOptions : BaseOptionModel<CodeOptions>
{
    [Category("Code")]
    [DisplayName("Include AAA Comments")]
    [Description("Whether to include comments for AAA (Arrange / Act / Assert) sections in test methods.")]
    [DefaultValue(true)]
    public bool IncludeAAAComments { get; set; } = true;

    [Category("Code")]
    [DisplayName("Use Smart Generics")]
    [Description("Whether to automatically substitute type parameters for generic symbols. " +
        "When disabled, generic symbols will remain open. \n" +
        "NOTE: this feature is experimental and does not work for certain scenarios.")]
    [DefaultValue(false)]
    public bool UseSmartGenerics { get; set; } = false;
}

```

Файл AdvancedOptions.cs

Реалізація функціональної вимоги FR-17 (Зміна налаштувань розширення), FR-24 (Задання необхідності попередження про генерацію кількох типів)

```

namespace NetTestX.VSIX.Options;

internal partial class OptionsProvider
{
    [ComVisible(true)]
    public class AdvancedOptionsPage : BaseOptionPage<AdvancedOptions> { }
}

/// <summary>
/// Advanced options contained in the "Advanced" options page inside VS settings
/// </summary>
public class AdvancedOptions : BaseOptionModel<AdvancedOptions>
{
    [Category("Advanced")]
    [DisplayName("Multiple Type Warning")]

```

```
[Description("Whether to show the confirmation window when there are multiple  
types in the file available for generation.")]  
[DefaultValue(true)]  
public bool ShowMultipleTypeWarning { get; set; } = true;  
}
```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗШИРЕННЯ VISUAL STUDIO ДЛЯ АВТОМАТИЧНОЇ ГЕНЕРАЦІЇ
ЮНІТ-ТЕСТІВ**

Програма та методика тестування

КПІ. ІП-0114.045300.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Микита КОВАЛЕНКО

Київ – 2024

ЗМІСТ

1	ОБ’ЄКТ ВИПРОБУВАНЬ.....	3
2	МЕТА ТЕСТУВАННЯ	4
3	МЕТОДИ ТЕСТУВАННЯ.....	5
4	ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ	6

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є розроблене розширення для середовища Visual Studio, призначене для автоматичної генерації юніт-тестів. Мінімальною версією середовища для роботи розширення є Visual Studio 2022.

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- перевірка правильності роботи програмного забезпечення відповідно до функціональних вимог;
- аналіз якості згенерованого коду;
- покращення підтримуваності за рахунок статичного аналізу кодової бази;
- знаходження проблем, помилок і недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється програма разом з усією документацією, яка аналізується на предмет дотримання стандартів програмування;

- динамічне тестування – застосовується в процесі виконання програми. Коректність програмного засобу перевіряється на певній кількості тестів. При прогоні кожного з них збираються та аналізуються дані про проблеми та помилки в роботі програми;

- функціональне тестування – полягає у перевірці відповідності реальної поведінки програмного забезпечення очікуваній;

- системне тестування – перевіряється усе програмне забезпечення в цілому;

- мануальне тестування – тестування без використання автоматизації, тест-кейси пише особа, що тестує програмне забезпечення;

- модульне тестування – автоматизоване тестування, що полягає в тестуванні кожного окремого модуля програмного забезпечення.

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується мануально з використанням наскрізного тестування та написанням unit-тестів, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення, так і в зручності користування. Також виконується статичний аналіз коду. Для того, щоб перевірити працездатність та відмовостійкість застосунку, необхідно провести наступні тестування:

- динамічне тестування на відповідність функціональним вимогам;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування компіляції згенерованого коду;
- тестування інтерфейсу користувача;
- тестування зручності використання;
- тестування присутності команд розширення у контекстних меню середовища.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗШИРЕННЯ VISUAL STUDIO ДЛЯ АВТОМАТИЧНОЇ ГЕНЕРАЦІЇ
ЮНІТ-ТЕСТІВ**

Керівництво користувача

КПІ. ІП-0114.045300.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

Виконавець:

_____ Микита КОВАЛЕНКО

Київ – 2024

ЗМІСТ

1	ПРИЗНАЧЕННЯ ПРОГРАМИ	3
2	ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ.....	4
2.1	Системні вимоги для коректної роботи.....	4
2.2	Завантаження застосунку	4
2.3	Перевірка коректної роботи.....	5
3	ВИКОНАННЯ ПРОГРАМИ	7
3.1	Налаштування розширення.....	9
3.2	Створення тестових проєктів	11
3.3	Генерація тестів.....	12

1 ПРИЗНАЧЕННЯ ПРОГРАМИ

“NetTestX” – це розширення для середовища Visual Studio, що автоматизує процес написання юніт-тестів мовою C#. Воно генерує шаблони тестів для методів (синхронних та асинхронних, статичних і не статичних), індексаторів, а також для інших ситуацій (наприклад, реалізація інтерфейсу IDisposable). Розширення прагне генерувати код, що компілюється, що полегшує його інтеграцію в процес розробки. Також воно здатне генерувати тестові проєкти.

Розширення має відкритий код і поширюється за ліцензією MIT. Посилання на репозиторій: <https://github.com/xkarlin/NetTestX>.

Розширення поширюється у вигляді файлу NetTestX.vsix. Його можливо встановити з порталу Visual Studio Marketplace або зібрати власноруч.

2 ПІДГОТОВКА ДО РОБОТИ З ПРОГРАМНИМ ЗАБЕЗПЕЧЕННЯМ

2.1 Системні вимоги для коректної роботи

Системні вимоги впливають із вимог до середовища розробки Visual Studio 2022. Для успішної роботи розширення необхідне виконання наступних вимог:

- тип процесору: Intel Core i5;
- об'єм ОЗП: 8 Гб;
- об'єм вільного дискового простору: 20 Гб.

2.2 Завантаження застосунку

Найлегшим способом встановлення розширення є встановлення напряду з середовища Visual Studio. Відкривши вікно Extensions -> Manage Extensions, знаходимо розширення у списку та встановлюємо його (рис. 2.1). Після встановлення треба перезавантажити Visual Studio.

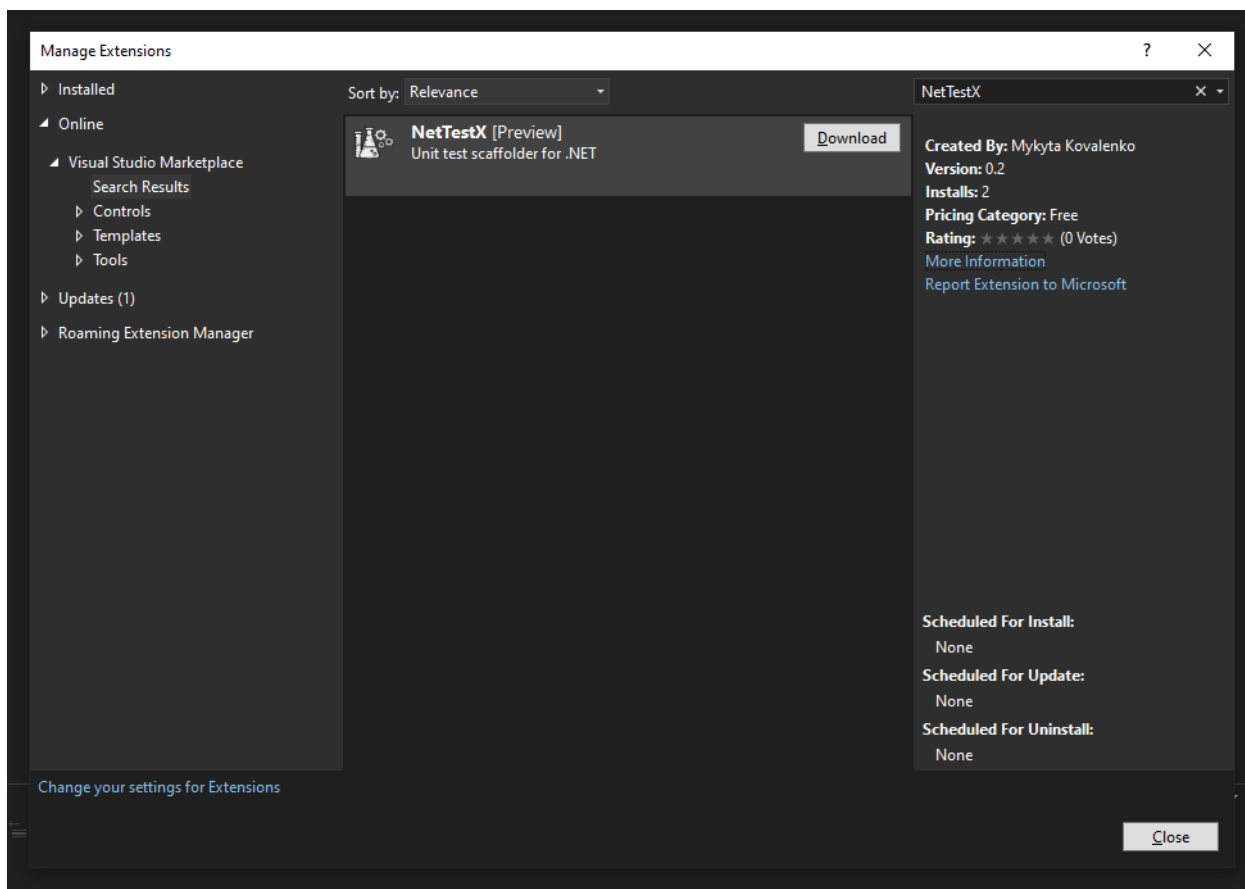


Рисунок 2.1 – Встановлення розширення з Visual Studio

Також розширення можливо встановити, завантаживши та відкривши файл NetTestX.vsix власноруч. Цей файл доступний на сторінці розширення на порталі Visual Studio Marketplace за посиланням <https://marketplace.visualstudio.com/items?itemName=MykytaKovalenko.NetTestX> (рис. 2.2).

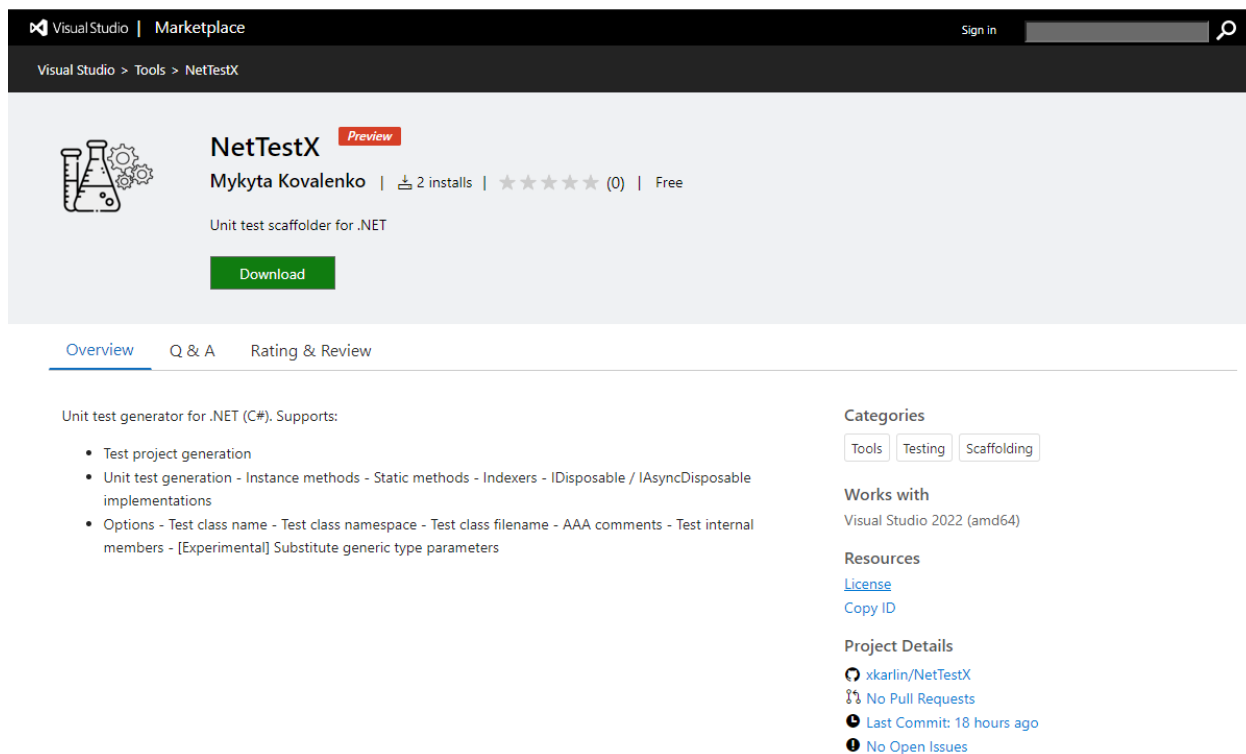


Рисунок 2.2 – Встановлення розширення з Visual Studio Marketplace

2.3 Перевірка коректної роботи

По завершенню встановлення розширення у вікні Extensions -> Manage Extensions в розділі Installed має з'явитися “NetTestX” (рис. 2.3):

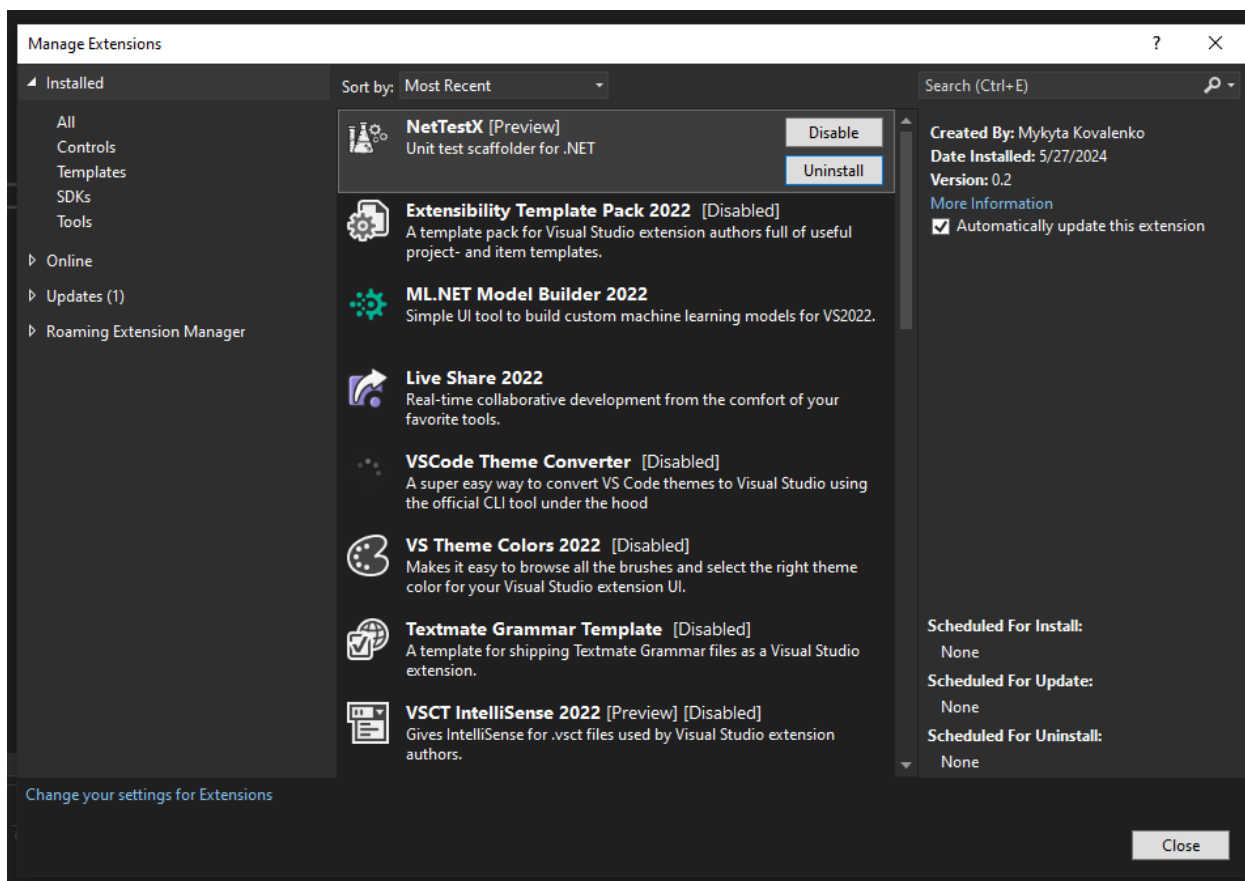


Рисунок 2.3 – Поява розширення у списку встановлених

У випадку, якщо розширення відсутнє у списку встановлених, то, ймовірно, середовище Visual Studio не було перезавантажене після завантаження розширення.

3 ВИКОНАННЯ ПРОГРАМИ

Після встановлення розширення у Visual Studio до середовища додаються такі можливості:

- команди в контекстному меню файлів *.cs у досліднику рішень для генерації тестів (рис. 3.1)

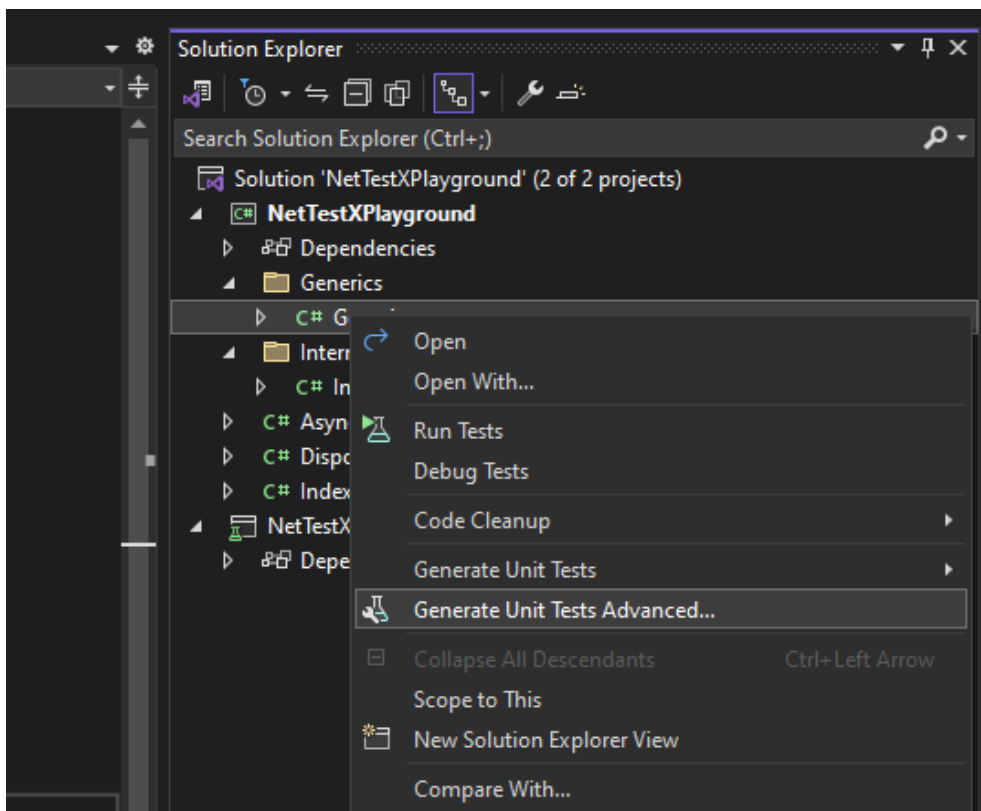


Рисунок 3.1 – Команди генерації тестів у вікні дослідника рішень

- команди в контекстному меню символу в редакторі коду для генерації тестів (рис. 3.2)

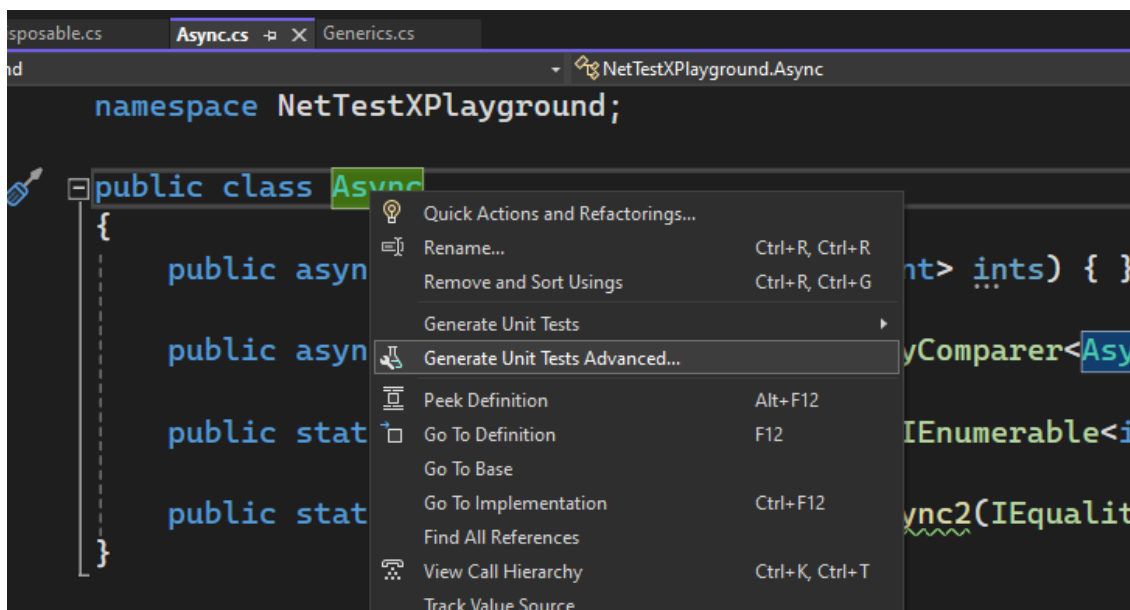


Рисунок 3.2 – Команди генерації тестів у вікні редактора коду

– команда в контекстному меню проєкту в досліднику рішень для генерації тестового проєкту (рис. 3.3)

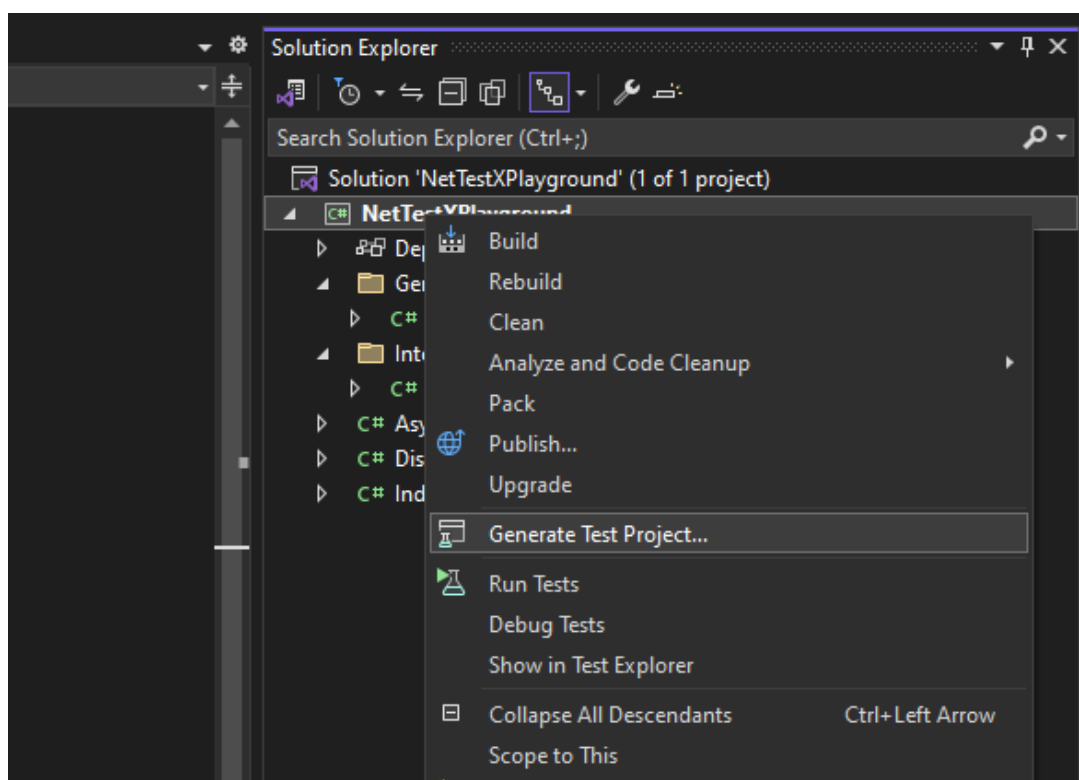


Рисунок 3.3 – Команда генерації тестового проєкту

– додаткова сторінка в меню Tools -> Options для зміни налаштувань розширення (рис. 3.4)

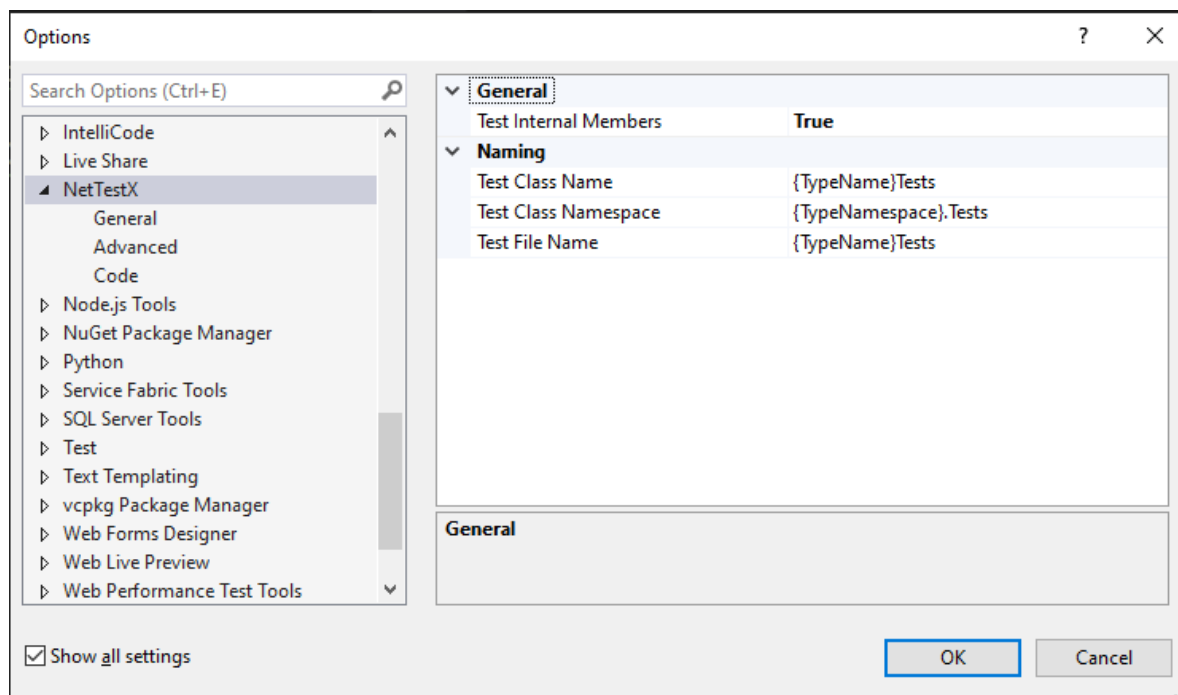


Рисунок 3.4 – Сторінка налаштувань General

3.1 Налаштування розширення

Сторінка налаштувань містить 3 розділи: General, Advanced і Code з відповідними опціями. Розділ General (рис. 3.4) надає можливості задання назви згенерованого класу, простору імен та імені файлу, а також необхідності тестування internal типів. Розділ Advanced (рис. 3.5) містить опцію попередження користувача про генерацію кількох тестових класів для одного файлу (якщо, наприклад, один файл містить кілька типів).

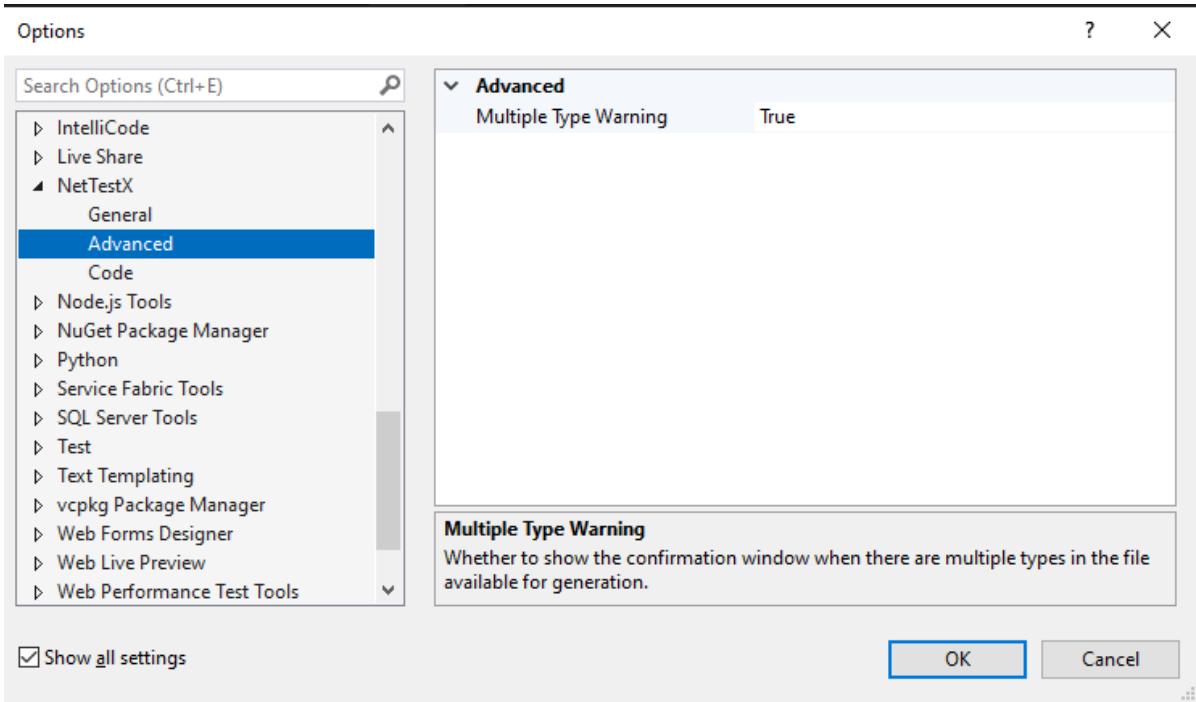


Рисунок 3.5 – Сторінка налаштувань Advanced

Сторінка Code містить налаштування згенерованого коду (рис. 3.6). Опція Include AAA Comments дозволяє додавати коментарі Arrange – Act – Assert до згенерованих тестів, а опція Use Smart Generics надає можливість автоматично підставляти аргументи типів в універсальних СИМВОЛАХ.

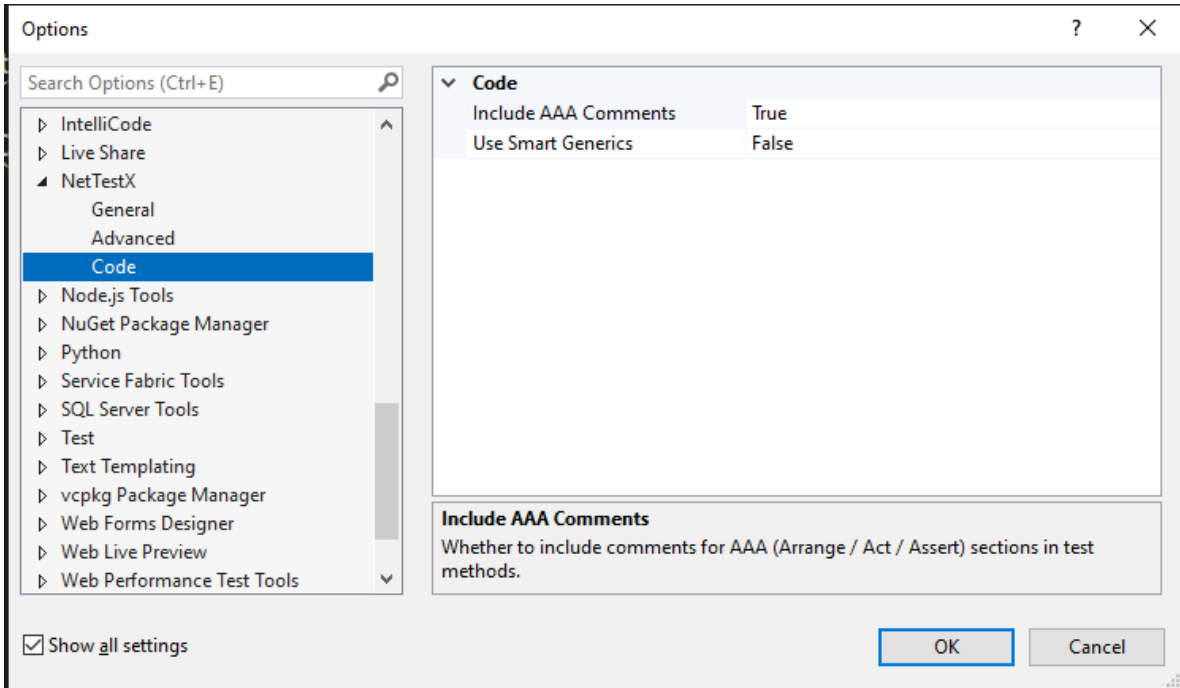


Рисунок 3.5 – Сторінка налаштувань Code

3.2 Створення тестових проєктів

Для створення тестового проєкту потрібно викликати контекстне меню для обраного проєкту та обрати команду “Generate Test Project...”, після чого сплине вікно створення проєкту (рис. 3.6). При виникненні помилки валідації відповідні поля підсвічуються червоним кольором (рис. 3.7). Після задання назви проєкту, папки з проєктом, а також тестового фреймворку та бібліотеки моків, що використовуються, і натискання Continue, до рішення буде додано згенерований тестовий проєкт (рис. 3.8).

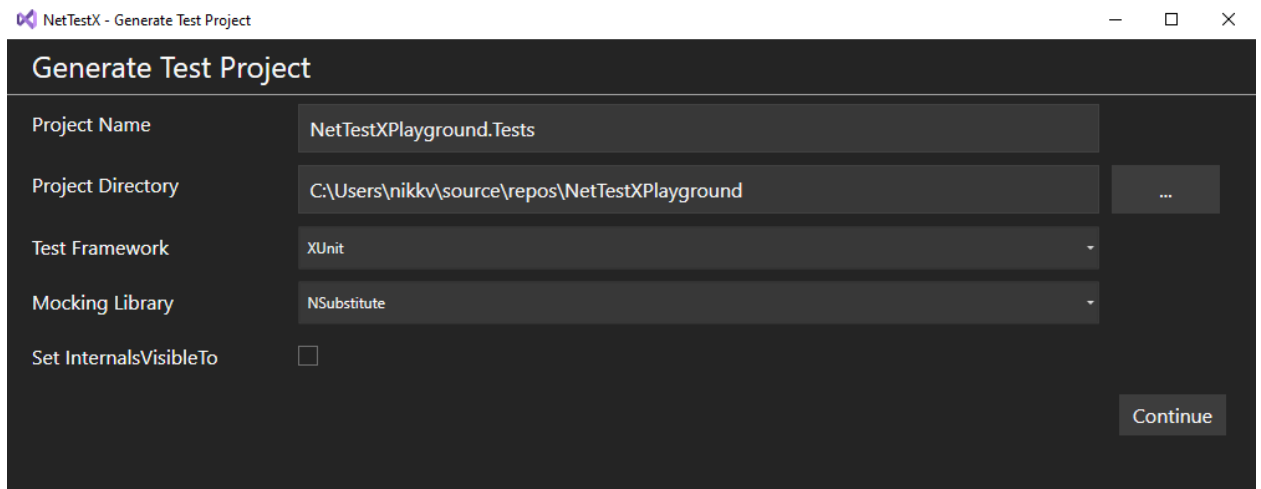


Рисунок 3.6 – Вікно створення тестового проєкту

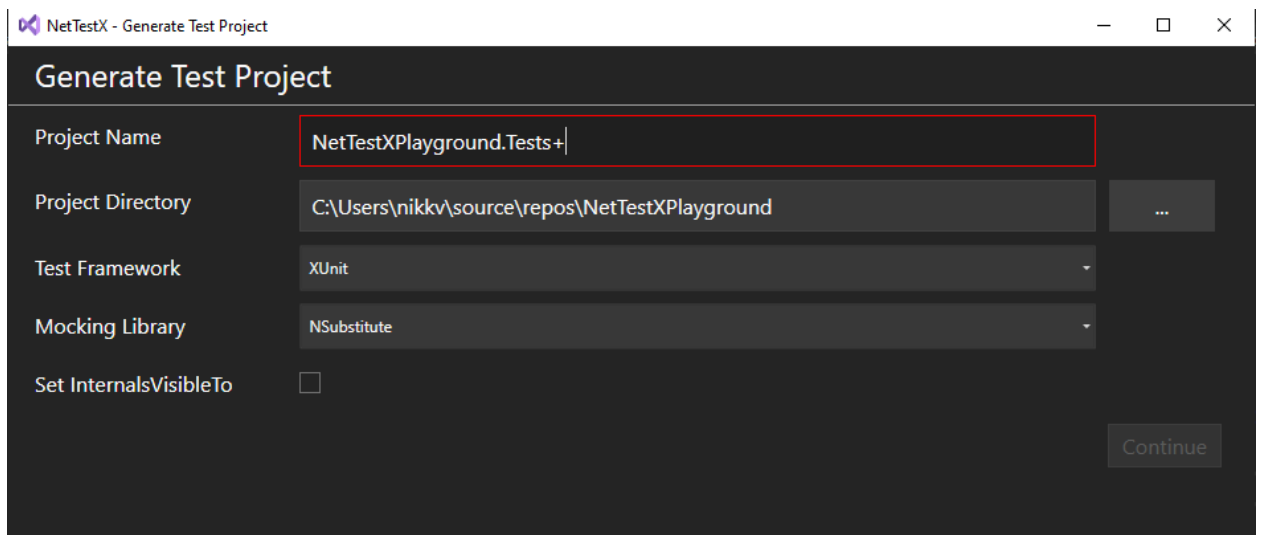


Рисунок 3.7 – Помилки валідації у вікні створення тестового проєкту

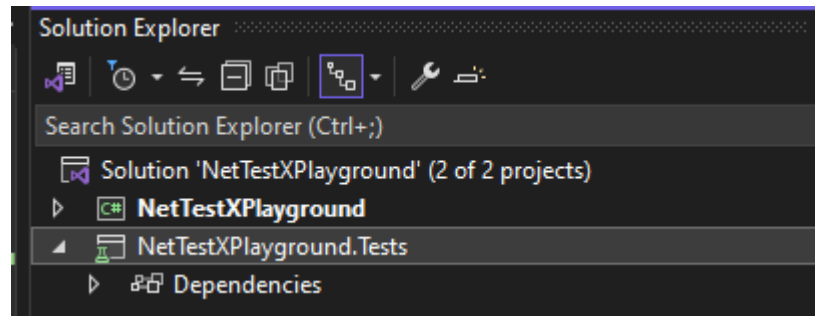


Рисунок 3.8 – Створений тестовий проєкт у вікні дослідника рішень

Створений тестовий проєкт також посилається на початковий проєкт, для якого він був створений.

3.3 Генерація тестів

Генерація тестів можлива за простим і складним (розширеним) сценарієм. При використанні простого сценарію тести генеруються, використовуючи опції за замовченням (задані в налаштуваннях розширення) та додаються до обраного тестового проєкту. Також присутня можливість створити новий тестовий проєкт (рис. 3.9).

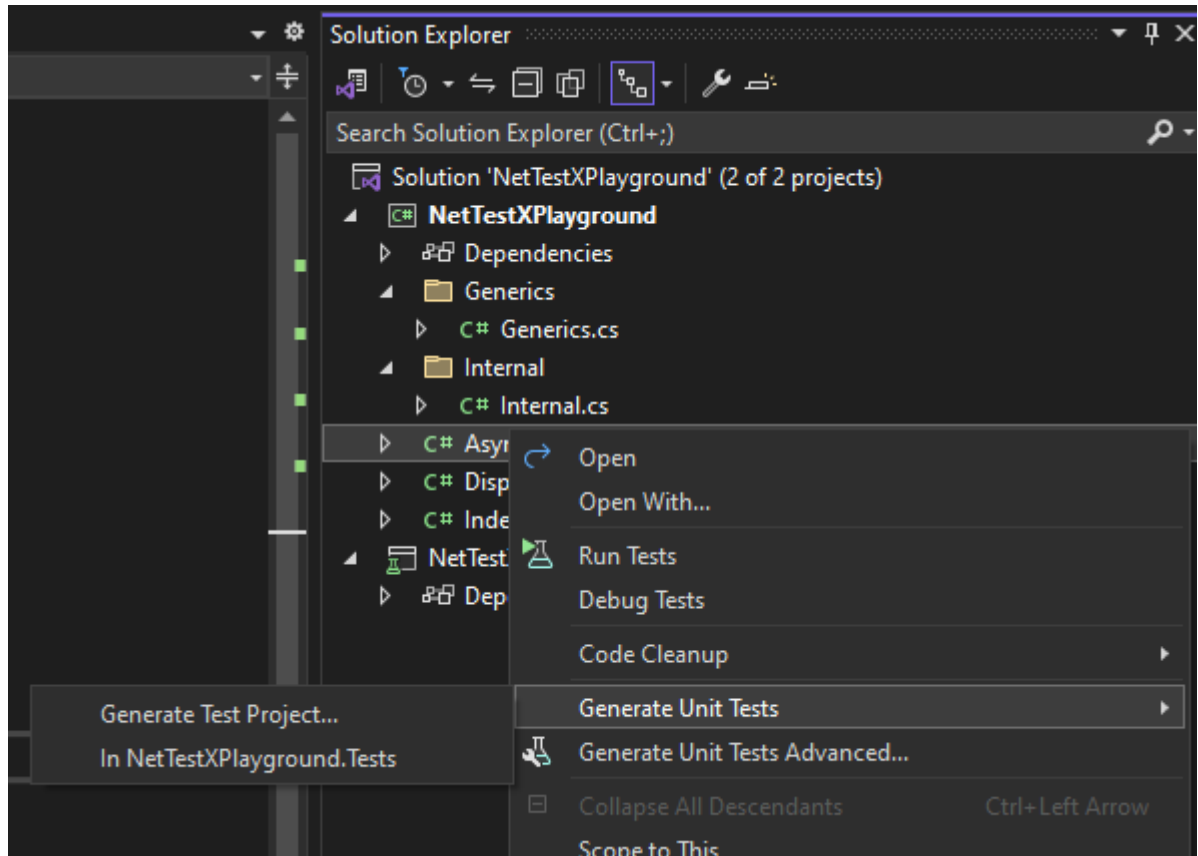


Рисунок 3.9 – Команда генерації тестів за простим сценарієм

Обравши потрібний проєкт, до якого будуть додані тести, зі списку, файл з тестами буде додано до цього проєкту (рис. 3.10).

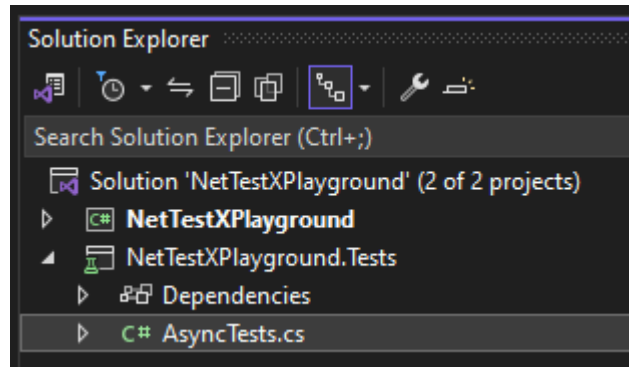


Рисунок 3.10 – Файл зі згенерованими тестами, доданий до тестового проєкту

При використанні складного сценарію спливає додаткове вікно створення тестів, у якому можливо задати специфічні налаштування (рис. 3.11), зокрема обрати окремі тести, що будуть згенеровані, а також перевизначити інші параметри. При виникненні помилки валідації відповідні поля підсвічуються червоним кольором (рис. 3.12).

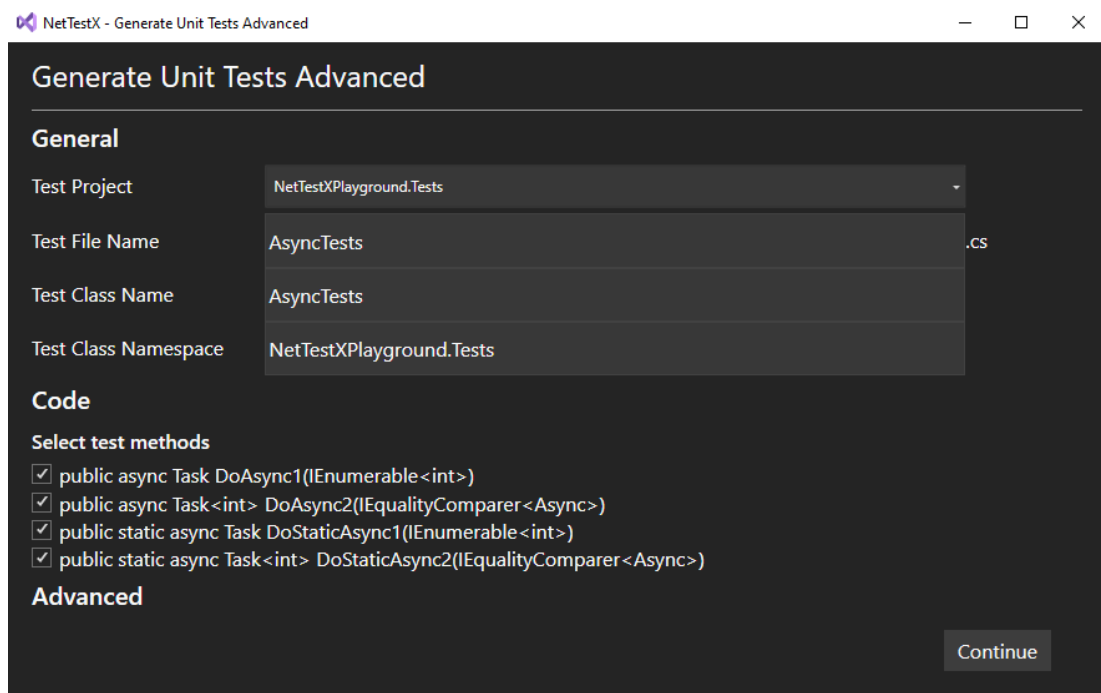


Рисунок 3.11 – Команда генерації тестів за розширеним сценарієм

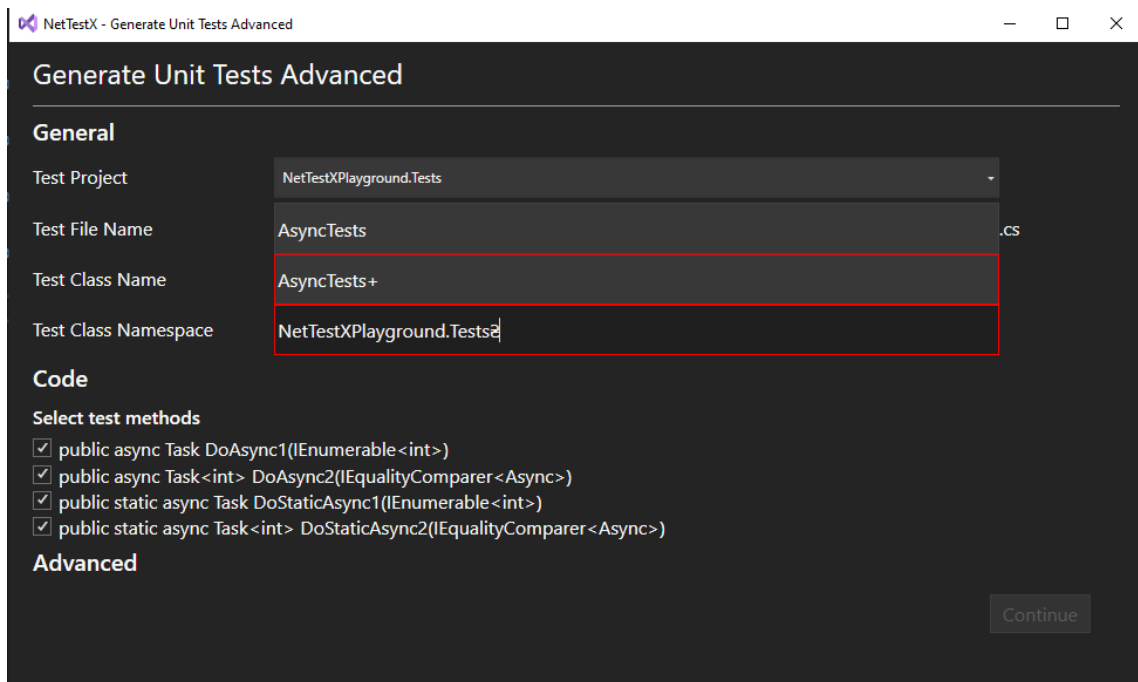


Рисунок 3.12 – Помилки валідації у вікні розширеної генерації тестів

Приклад згенерованих тестів показаний на рис. 3.13. Тіло згенерованих методів може дещо відрізнятися в залежності від символу, що тестується, та на високому рівні воно містить три секції:

- **Arrange:** ця секція створює екземпляр типу (якщо тестуються не статичні члени), а також моки для аргументів методу чи індексатора.
- **Act:** ця секція безпосередньо виконує дію, що тестується, наприклад, викликає метод. При цьому до нього передаються необхідні аргументи, визначені в секції Arrange. Якщо метод чи індексатор повертає значення, то воно записується у змінну.
- **Assert:** ця секція зазвичай дописується вручну та містить перевірку результату.

```

namespace NetTestXPlayground.Tests;

public class AsyncTests
{
    [Fact]
    public async Task TestDoAsync1()
    {
        // Arrange
        Async sut = new();

        var testInts = Substitute.For<IEnumerable<int>>();

        // Act
        await sut.DoAsync1(testInts);

        // Assert
    }
}

```

Рисунок 3.13 – Приклад згенерованих тестів

Тести також можуть бути згенеровані для кількох файлів одночасно. Для цього достатньо обрати їх у вікні дослідника рішень і викликати контекстне меню (рис. 3.14).

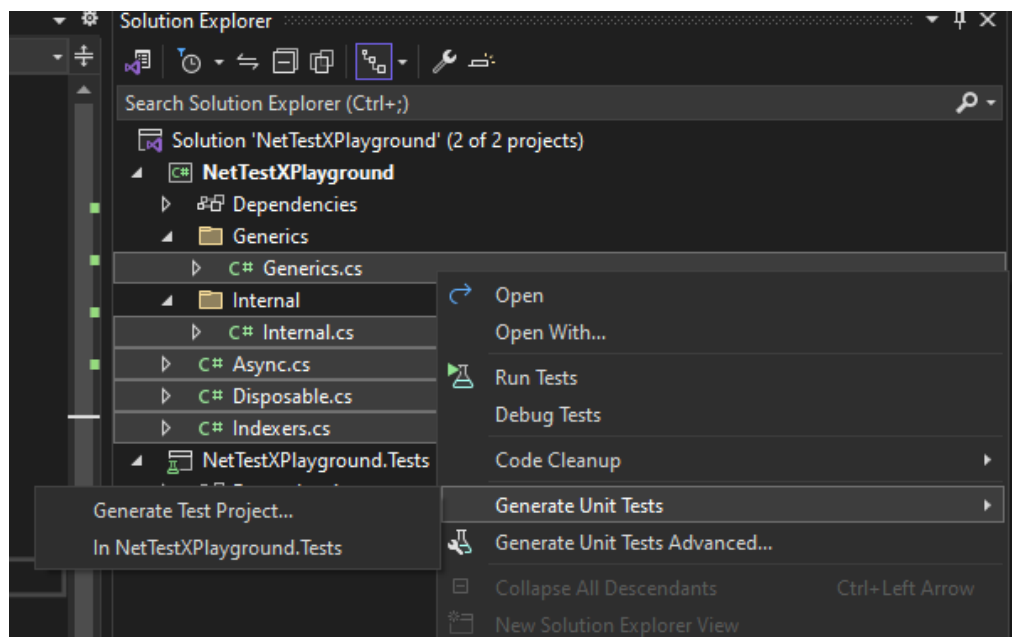


Рисунок 3.14 – Генерація тестів для кількох файлів

У результаті для кожного типу буде згенеровано окремий файл з тестами, при цьому файлова структура зберігається (рис. 3.15).

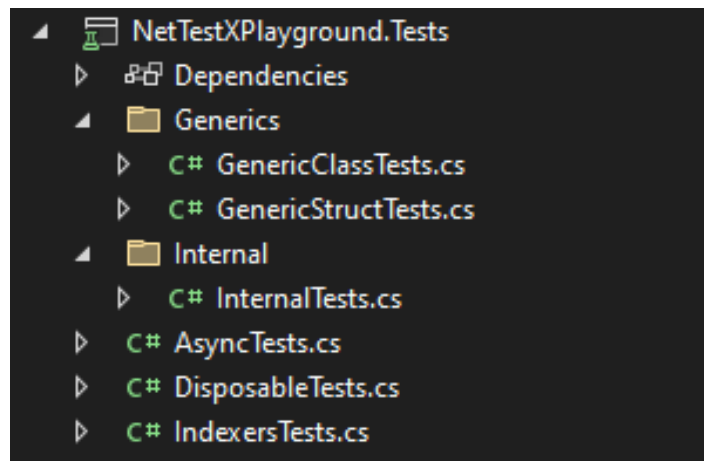


Рисунок 3.15 – Згенеровані тести для кількох файлів

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2024 р.

**РОЗШИРЕННЯ VISUAL STUDIO ДЛЯ АВТОМАТИЧНОЇ ГЕНЕРАЦІЇ
ЮНІТ-ТЕСТІВ**

Графічний матеріал

КПІ. ІП-0114.045300.06.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олександр ПАВЛОВ

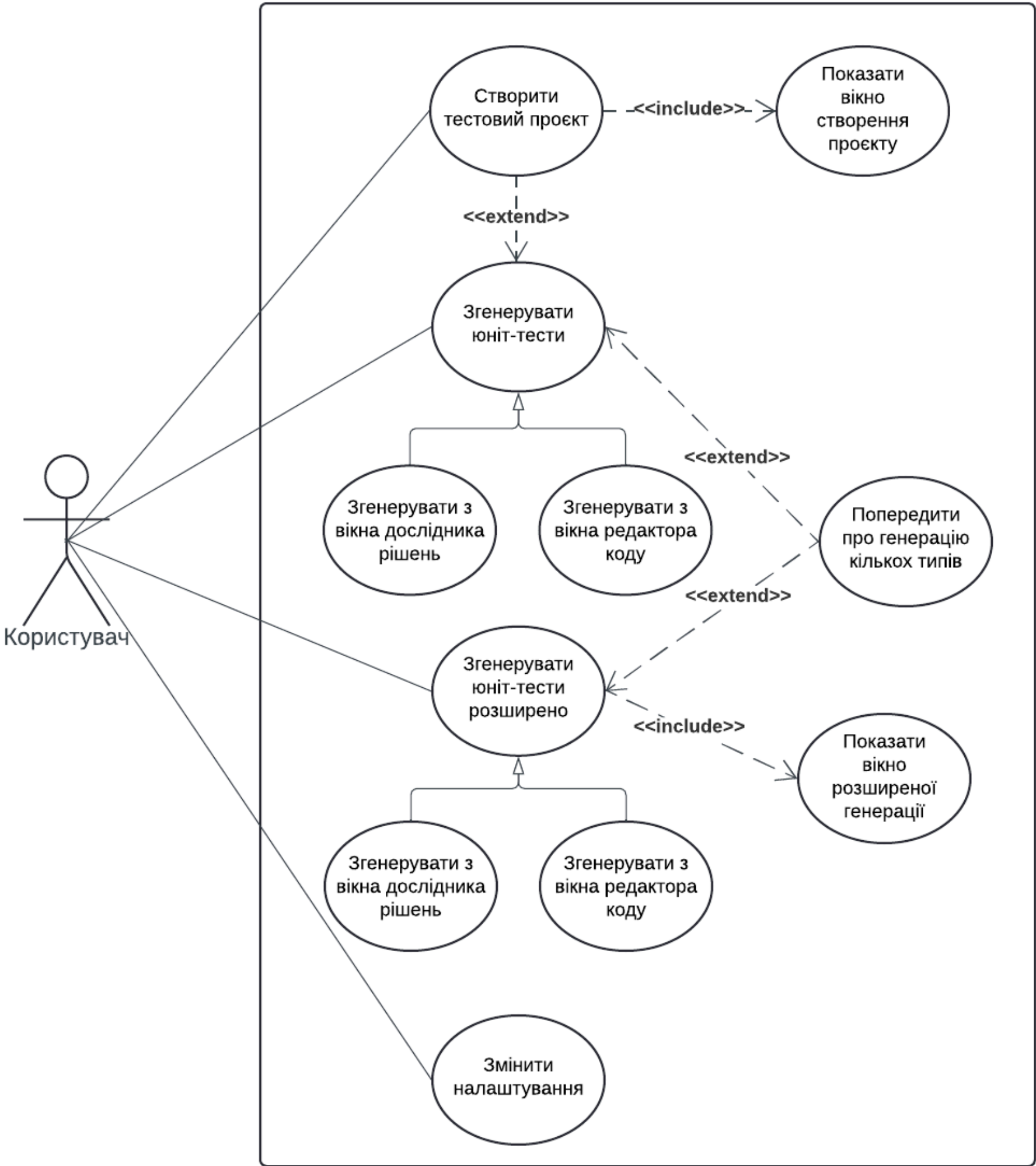
Нормоконтроль:

_____ Максим ГОЛОВЧЕНКО

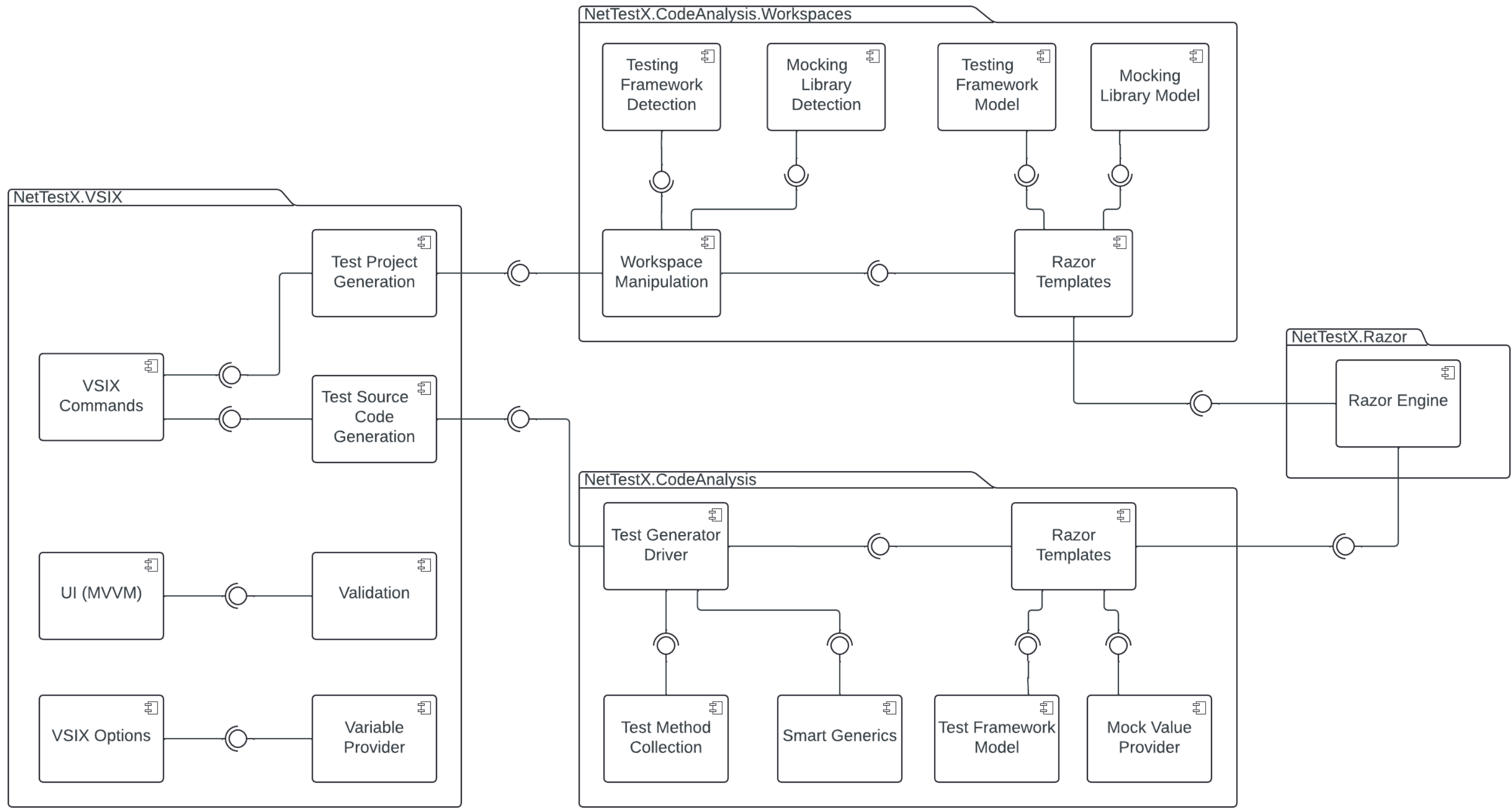
Виконавець:

_____ Микита КОВАЛЕНКО

Київ – 2024



					КПІ.ІП-0114.045300.06.99.CCB							
					Схема структурна варіантів використань			Лит.		Маса	Масштаб	
Зм.	Арк.	№ докум.	Підп.	Дата	Схема структурна варіантів використань							
Розробив		Коваленко М. А.										
Перевірив		Павлов О. А.										
Т. контр.					Розширення Visual Studio для автоматичної генерації юніт-тестів			Аркуш		Аркушів		
Н. контр.		Головченко М.М.										
Затвердив		Жаріков Е.В.						КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01				



					КПІ.ІП-0114.045300.06.99.CCM							
					Схема структурна компонентів програмного забезпечення	Літера			Маса		Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата								
Розробив	Коваленко М.А.											
Перевірив	Павлов О. А.											
Т. контр.						Аркуш			Аркушів			
					Розширення Visual Studio для автоматичної генерації юніт-тестів	КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01						
Н. контр.	Головченко М.М.											
Затвердив	Жаріков Е.В.											

NetTestX - Generate Unit Tests Advanced

Generate Unit Tests Advanced

General

Test Project

NetTestXPlayground.Tests

Test File Name

AsyncTests

.CS

Test Class Name

AsyncTests

Test Class Namespace

NetTestXPlayground.Tests

Code

Select test methods

☒ public async Task DoAsync1(IEnumerable)

☒ public async Task DoAsync2(IEqualityComparer)

☒ public static async Task DoStaticAsync1(IEnumerable)

☒ public static async Task DoStaticAsync2(IEqualityComparer)

Continue

NetTestX - Generate Test Project

Generate Test Project

Project Name

NetTestXPlayground.Tests

Project Directory

C:\Users\nikkv\source\repos\NetTestXPlayground

...

Test Framework

XUnit

Mocking Library

NSubstitute

Set InternalsVisibleTo

☐

Continue

Options

Search Options (Ctrl+E)

Cookiecutter

Cross Platform

Database Tools

F# Tools

IntelliCode

Live Share

NetTestX

General

Advanced

Code

Node.js Tools

NuGet Package Manager

Python

Service Fabric Tools

SQL Server Tools

Test

Text Templating

General

Test Internal Members

True

Naming

Test Class Name

{TypeName}Tests

Test Class Namespace

{TypeNamespace}.Tests

Test File Name

{TypeName}Tests

Test Class Name

The name of the test class.

Show all settings

OK

Cancel

					КПІ.ІП-0114.045300.06.99.КЕ					
					Креслення вигляду екранних форм	Літера		Маса	Масштаб	
Зм.	Арк.	№ документа	Підпис	Дата						
Розробив		Коваленко М.А.								
Перевірів		Павлов О. А.								
Т. контр.										
					Розширення Visual Studio для автоматичної генерації юніт-тестів	Аркуш		Аркушів		
Н. контр.		Головченко М.М.				КПІ ім.Ігоря Сікорського Кафедра ІПІ гр. ІП-01				
Затвердив		Жаріков Е.В.								